50% DEFENSE - UPDATED TRACE

FootPath Cebu - Deep Code Tracing

Verified execution paths, authorization boundaries, database traces, failure handling, and defense-ready call chains.

Revision	50% defense integrated-system trace
Source commit	6586553 - tournament squads, enforcement, and coordinator UI
Verified suites	352 Django tests | 295 Flutter tests | clean Flutter analysis | no migration drift | 3/3 CI jobs
Primary audience	Capstone defense, code review, and implementation tracing

Contents

Contents 2

50% defense implementation snapshot . . . 9
How the applications communicate . . . 9
Technology stack and purpose . . . 11
Code execution map index . . . 12
Workflow 1: Application Startup . . . 13
Workflow 2: Authentication / Session Restoration . . . 14
Workflow 3: Super Admin Club / Coordinator Provisioning . . . 15
Workflow 4: Login . . . 16
Workflow 5: Logout . . . 18
Workflow 6: Role Checking and Authorization . . . 19
Workflow 7: Navigation After Login . . . 20
Workflow 8: Player Profile Retrieval . . . 21
Workflow 9: Player Profile Update (Position) . . . 22
Workflow 10: Creating a Scouting Report . . . 24
Workflow 11: Retrieving Scouting Reports . . . 25
Workflow 12: Editing a Scouting Report . . . 26
Workflow 13: Training Session Creation . . . 27
Workflow 14: Training Attendance Recording . . . 28
Workflow 15: Coach Evaluation / Assessment . . . 30
Workflow 16: Performance Data Retrieval . . . 31
Workflow 17: Performance Dashboard Generation . . . 32
Workflow 18: Academic Eligibility Retrieval . . . 33
Workflow 19: Academic Eligibility Update . . . 34

Workflow 20: Notifications . . . 36
Workflow 21: File/Image Upload . . . 38
Workflow 22: Search / Filtering . . . 39
Workflow 23: AI / Recommendation / Chatbot . . . 40
Workflow 24: Admin / Coordinator Account Operations . . . 41
Workflow 25: Guardian PIN Unlock and Child Selection . . . 42
Workflow 26: Injury Reporting, Confirmation, Recovery Update, and Archive . . . 44
Workflow 27: Dispute Raise and Response . . . 45
Workflow 28: Player Session Confirmation . . . 46
Workflow 29: Password Reset and Change . . . 48
Workflow 30: Coordinator Creates or Updates a Completed Match . . . 49
Workflow 31: Coordinator Records or Corrects Objective Player Statistics . . . 50
Workflow 32: Player, Guardian, and Coach View Match Statistics and Trends . . . 52
Workflow 33: Coordinator Publishes a Tournament Schedule and Fixtures . . . 53
Workflow 34: Coach Adds or Clears a Subjective Match Rating . . . 54
Workflow 35: Independent-Club Academic Eligibility Suppression . . . 56
Workflow 36: Coordinator Creates and Publishes Mobile Tournament Brackets . . . 57
Workflow 37: Coach Builds and Publishes an Age-Bracket Tournament Squad . . . 58
Workflow 38: Tournament Squad Enforcement During Match Statistics Entry . . . 59
Cross-Cutting Trace: Coordinator Responsive UI Consistency . . . 61
Cross-Cutting Trace: Shared Authenticated API Client and Owner-Scoped Cache . . . 61
Repository Production Hardening Trace . . . 62

Database Query Traces **62**

QRY-1: Resolve Authenticated User . . . 62
QRY-2: Read Squad . . . 62
QRY-3: Read Own/Guardian Player Profile . . . 63

QRY-4: Save Assessment . . . 63
QRY-5: Create/List Training Session . . . 64
QRY-6: Replace Session Attendance . . . 64
QRY-7: Player Attendance History . . . 65
QRY-8: Squad Progress Aggregate . . . 65
QRY-9: Verify PIN . . . 65
QRY-10: Eligibility Change and History . . . 66
QRY-11: Injury Confirmation Workflow . . . 66
QRY-12: Dispute and Response . . . 67
QRY-13: Register Device . . . 67
QRY-14: Account Provisioning . . . 68
QRY-15: Photo Path . . . 68
QRY-16: Notification Inbox and Read State . . . 68
QRY-17: Create or Update Football Match . . . 69
QRY-18: Upsert Player Match Performance . . . 69
QRY-19: Read Player Match Summary and History . . . 70
QRY-20: Publish Tournament Schedule and Fixtures . . . 70
QRY-21: Save or Clear Coach Match Rating . . . 70
QRY-22: Live Database Selection . . . 71
QRY-23: Create, Edit, Delete, or Publish Mobile Tournament Brackets . . . 71
QRY-24: Read and Save Coach Tournament Squad . . . 72
QRY-25: Publish Coach Tournament Squad . . . 72
QRY-26: Enforce Tournament Squad During Match-Performance Save . . . 73

Variable Traces **73**

Variable: User ID / Firebase UID . . . 73
Variable: Role . . . 73

Variable: Player ID . . . 73
Variable: Coach ID . . . 74
Variable: Training ID . . . 74
Variable: Attendance Status . . . 74
Variable: Performance Score . . . 74
Variable: Match ID and Player Match Key . . . 74
Variable: Coach Rating and Trend . . . 74
Variable: Tournament Fixture ID . . . 74
Variable: Tournament and Age-Bracket ID . . . 75
Variable: Tournament Squad Eligibility . . . 75
Variable: Squad Override Reason . . . 75
Variable: Injury Review Status . . . 75
Variable: Academic Eligibility . . . 75
Variable: Guardian Unlock Token . . . 75
Variable: Evaluation ID . . . 75
Variable: Scout ID / Scouting Report ID . . . 76

Object / Model Traces **76**

Object: UserProfile . . . 76
Object: Player . . . 76
Object: TrainingSession . . . 76
Object: Attendance . . . 76
Object: FootballMatch . . . 77
Object: MatchPerformance . . . 77
Object: PlayerMatchStatistics . . . 77
Object: EligibilityChange . . . 77
Object: TournamentSchedule . . . 78

Object: TournamentSquad and TournamentRosterCandidate . . . 78
Object: InjuryRecord . . . 78
Object: Dispute . . . 79
Object: SessionConfirmation . . . 79
Object: AppNotification . . . 79

Button-to-Database/API Traces **80**

Screen-to-Screen Navigation Traces **81**

Async Code Traces **83**

Firebase/app bootstrap . . . 83
Login and restore . . . 83
API read providers . . . 83
Mutations . . . 83
Attendance synchronization . . . 83
Unawaited device registration . . . 83
Foreground and opened notifications . . . 84
Django `transaction.on_commit` . . . 84

Failure / Error Path Matrix **85**

Authorization Trace **86**

Data Lifecycle Traces **87**

1. User / Account . . . 87
2. Player Profile / Performance . . . 87
2A. Football Match / Historical Performance . . . 87

2B. Tournament Schedule / Fixture . . . 88
2C. Tournament Squad / Match Exception . . . 88
2D. Injury Report / Recovery Workflow . . . 89
3. Training Session . . . 89
4. Attendance . . . 89
5. Eligibility . . . 90
6. Notification . . . 90
Scouting Report Lifecycle . . . 90

Top 31 Important Function Call Chains **90**

Code-Tracing Defense Questions (50) **93**

Memorization Cards - Eighteen Critical Workflows **99**

Card 1: Login . . . 99
Card 2: Session Restoration . . . 100
Card 3: Player Profile Retrieval . . . 101
Card 4: Guardian Unlock . . . 102
Card 5: Create Training Session . . . 103
Card 6: Attendance Online/Offline . . . 104
Card 7: Assessment Save . . . 105
Card 8: Performance Dashboard . . . 106
Card 9: Eligibility Update and Read . . . 107
Card 10: Injury Report and Confirmation . . . 108
Card 11: Record Match and Objective Player Statistics . . . 109
Card 12: View Match Statistics and Trends . . . 110
Card 13: Flutter-to-Django API Communication . . . 111
Card 14: Publish Tournament Schedule . . . 112

Card 15: Coach Match Rating Separation . . . 113
Card 16: Mobile Tournament and Age-Bracket Management . . . 114
Card 17: Coach Tournament Squad . . . 115
Card 18: Tournament Match Squad Enforcement . . . 116

Final Trace Verification Notes **117**

This execution map follows only verified repository connections. Approximate locations use symbol names/nearby line numbers because line numbers can move. Django's default table names are stated where no explicit `db_table` overrides exist.

50% defense implementation snapshot

This reviewer describes the current repository at commit `6586553`. For the 50% defense, the defensible claim is that the implemented core is integrated and testable: authenticated role routing, Club-scoped account management, player profiles, privacy PINs, training and attendance, assessments, eligibility rules, disputes, notifications, Coordinator-managed tournament brackets, Coach-managed tournament squads, match-squad eligibility enforcement, completed-match statistics and ratings, injury confirmation, and private uploads all have executable Flutter/Django paths. The normal Flutter build uses live Firebase and Django repositories; mock repositories remain available only for isolated development and automated Flutter tests.

"Done for the 50% defense" does not mean that every possible future feature or production operation is complete. Scout reports and AI recommendations are explicitly not implemented. Real-device FCM/APNs delivery, a public deployment, production alerting, and a restore drill remain environment-dependent verification items. This wording lets the team defend what exists without claiming more than the repository proves.

Short defense answer

> Our 50% build is an integrated client-server system, not a collection of disconnected screens. Flutter obtains identity from Firebase, sends authenticated REST requests to Django, and Django applies the authoritative role, Club, object, and privacy rules before using its ORM. The current scope covers the main academy, safeguarding, tournament, and performance workflows. Automated tests and CI give repeatable evidence for the committed build, while deployment and physical-device checks remain separate validation work.

How the applications communicate

The mobile application and backend communicate through a REST-style HTTP API implemented with Django REST Framework. Flutter does not connect directly to PostgreSQL, SQLite, Redis, Firebase Admin, or Supabase Storage.

```
Flutter screen
  -> Riverpod provider/controller
  -> domain use case
  -> repository interface
  -> Api*Repository
  -> AuthenticatedApiClient (`http` package)
  -> HTTPS + JSON or multipart/form-data
  -> Django URL route
  -> DRF APIView + FirebaseAuthentication
  -> serializer validation + role/Club/object checks
  -> Django ORM transaction
  -> PostgreSQL/Supabase in live production, SQLite in tests/local development
  -> DRF JSON response
  -> Dart `fromJson()` entity
  -> Riverpod state refresh
  -> rebuilt Flutter widget
```

Authentication and request contract

1. Firebase Authentication signs in the Flutter user and issues an ID token.
2. `AuthenticatedApiClient` requests the current token and adds `Authorization: Bearer <Firebase ID token>`.
3. JSON writes add `Content-Type: application/json`; photo uploads use `multipart/form-data`.
4. `accounts.authentication.FirebaseAuthentication` verifies signature, expiry, and revocation with Firebase Admin, then maps the UID to an active Django `User`.
5. Django uses the local `User.role`, `User.club`, guardian links, privacy unlock header, and object queries for authorization. The Firebase token proves identity but does not choose permissions.
6. Serializers validate input and shape JSON output. Server-owned fields such as Club, actor, player, creator, and recorder are derived from the authenticated request or a scoped URL lookup.

Typical calls include `GET /api/auth/me/`, `GET /api/tournament-schedules/`, `POST /api/tournament-schedules/{id}/brackets/`, `PUT /api/tournament-brackets/{id}/squad/`, `GET /api/matches/{id}/roster/`, and `PUT /api/matches/{matchId}/performances/{playerId}/`. Successful reads usually return HTTP 200; creates normally return 201; successful deletes return 204; validation, authentication, permission, missing-object, conflict, and server failures use 400, 401, 403, 404, 409, and 5xx responses respectively.

Why use an API boundary

- Security: Flutter never receives database credentials, Firebase Admin credentials, or the Supabase service key.
- Central authorization: Django enforces the same role and Club rules even if a mobile request is modified.
- Validation and integrity: serializers, model validation, transactions, foreign keys, unique constraints, and database checks protect writes.
- Maintainability: presentation and domain code depend on interfaces instead of HTTP details.
- Testability: live repositories can be replaced by mocks/fakes without rewriting screens or use cases.
- Multi-client support: Flutter, the Django portal, admin, and future clients can share the same server-owned data model.
- Resilience: the shared client standardizes token attachment, timeouts, typed errors, owner-scoped GET caching, and the attendance offline outbox.

Technology stack and purpose

Area	Technology	Purpose in FootPath Cebu
Mobile application	Flutter and Dart	Cross-platform Coach, Coordinator, Player, and Guardian client
Mobile architecture	Clean Architecture with presentation, domain, and data layers	Separates UI, business operations, and infrastructure dependencies
State and dependency wiring	Riverpod	Async state, controllers, provider families, invalidation, and dependency injection
Mobile HTTP	Dart `http` and `http_parser`	JSON requests, Bearer headers, and multipart uploads to Django
Mobile identity	Firebase Authentication	Email/password sign-in, persisted session, ID-token issuance, and reauthentication
Push transport	Firebase Cloud Messaging	Best-effort device alerts; Django remains the source for the persistent inbox
Mobile local resilience	`sqflite` and `connectivity_plus`	Owner-scoped GET cache, attendance outbox, and reconnect-driven replay
Backend	Python, Django 5.2, Django REST Framework 3.16	API routing, serializers, permissions, web portal, admin, services, and ORM
Backend identity bridge	Firebase Admin SDK	Verifies Firebase ID tokens and sends FCM notifications from trusted server code
Web interfaces	Django templates, JavaScript, self-hosted Tailwind CSS	Coordinator/School Staff portal and emergency admin console
Relational persistence	Django ORM with PostgreSQL/Supabase for live deployment	Authoritative users, Clubs, academy records, constraints, transactions, and migrations
Local/test persistence	SQLite	Zero-setup local development and isolated automated tests
Private object storage	Supabase Storage REST through server-side `httpx`	Player photos and tournament documents with short-lived signed URLs
Cache and lockout state	Redis in production; local-memory cache in development/tests	Shared cache, signed-URL caching, rate/lockout consistency, and revocation windows
Web/session security	Django sessions, Argon2id, django-axes, CORS/CSRF controls	Protects portal authentication and browser requests
Production serving	Gunicorn, WhiteNoise, Docker Compose	WSGI serving, static files, reproducible application packaging, Postgres, and Redis
Monitoring	Optional Sentry plus structured console logging	Error reporting and production diagnostics without logging secrets or request bodies

Supabase clarification

Supabase has two server-side roles here. PostgreSQL/Supabase can host the relational database through Django's PostgreSQL driver, and Supabase Storage holds private files. Flutter never uses Supabase Auth or Supabase Row Level Security and never receives a service key. In non-test production, Django refuses to start without `DB_HOST`, so live execution cannot silently fall back to a device-only or local SQLite database. SQLite remains intentionally available for development and automated tests.

Code execution map index

Required map	Location in this document
Application startup trace	Workflow 1
Authentication trace	Workflows 2, 4, 5, and Authorization Trace
Navigation trace	Workflow 7 and Screen-to-Screen Navigation Traces
Role authorization trace	Workflow 6 and Authorization Trace
Player workflow trace	Workflows 8, 9, 16, 18, 25, 26, 28, 32, 35, and 37
Coach workflow trace	Workflows 13-17, 27, 31, 32, 34, 37, and Button-to-Database Traces
Coordinator workflow trace	Workflows 24, 26, 30, 31, 33-36, 38, and Button-to-Database Traces
Scout workflow trace	Workflows 10-12: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
Scouting-report trace	Workflows 10-12: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
Training trace	Workflows 13, 14, and 28
Attendance trace	Workflow 14 and attendance lifecycle/query/variable traces
Performance trace	Workflows 15-17, 30-34, and 38
Tournament bracket/squad trace	Workflows 33, 36-38; QRY-23 through QRY-26; tournament lifecycle
Academic eligibility trace	Workflows 18-19
AI trace	Workflow 23: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
File-upload trace	Workflow 21
Database query traces	Database Query Traces section
Variable traces	Variable Traces section
Object/model traces	Object / Model Traces section
Async traces	Async Code Traces section
Failure/error traces	Failure / Error Path Matrix and workflow failure paths
Top 31 important function call chains	Top 31 Important Function Call Chains section
Shared HTTP/offline-read trace	Cross-Cutting Trace: Shared Authenticated API Client and Owner-Scoped Cache
Production hardening trace	Repository Production Hardening Trace
Flutter/Django API communication	How the applications communicate and Cross-Cutting Trace

Required map	Location in this document
Technology stack	Technology stack and purpose
50% defense status	50% defense implementation snapshot and Final Trace Verification Notes

Workflow 1: Application Startup

Trigger

The OS launches Flutter and invokes `main()` in `footpath_cebu/lib/main.dart` near line 15.

Step 1 - UI

There is no user input yet. `WidgetsFlutterBinding.ensureInitialized()` prepares plugins; `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` runs. `runApp(ProviderScope(child: FootPathApp(...)))` creates the widget tree.

Step 2 - State / Controller Layer

`ProviderScope` enables Riverpod. `FootPathApp.build()` creates `MaterialApp`; its `home` is `SessionBootstrapScreen` unless Firebase setup threw, in which case `_SetupErrorScreen` displays the caught message.

Step 3 - Service / Repository Layer

Startup uses Firebase Core and, in live mode, `_FootPathAppState.initState()` subscribes to foreground messages, opened-message events, and FCM token refresh. Session work is deferred to `SessionBootstrapScreen`. Dependency factories live in `core/di/providers.dart` and are lazily read.

Step 4 - Backend / API

No backend request is made by `main()`. The first API request occurs during session restoration.

Step 5 - Database

No database record is touched by `main()`.

Step 6 - Backend Response

Not applicable. Firebase initialization either completes or throws locally/plugin-side.

Step 7 - Model Conversion

No domain model conversion occurs.

Step 8 - State Update

The caught setup error becomes the immutable `FootPathApp.setupError` constructor value. Riverpod state begins inside `ProviderScope`; successful live startup retains the three messaging subscriptions until `dispose()` cancels them.

Step 9 - UI Update

Flutter renders `SessionBootstrapScreen` or `_SetupErrorScreen` through normal widget build.

Call chain: `main()` -> `ensureInitialized()` -> `Firebase.initializeApp()` -> `runApp()` -> `ProviderScope` -> `_FootPathAppState.initState()` -> FCM foreground/open/token listeners -> `FootPathApp.build()` -> `MaterialApp` -> `SessionBootstrapScreen`.

Failure path: Firebase initialization exception -> `setupError` string -> `_SetupErrorScreen`; no login/profile call is attempted.

Workflow 2: Authentication / Session Restoration

Trigger

`_SessionBootstrapScreenState.initState()` in `presentation/screens/session_bootstrap_screen.dart` schedules `_restore()` after widget initialization.

Step 1 - UI

`SessionBootstrapScreen` initially renders a progress indicator. `_restore()` calls `ref.read(restoreSessionProvider)()`.

Step 2 - State / Controller Layer

`restoreSessionProvider` exposes the `RestoreSession` use case from `core/di/providers.dart`. The screen awaits `UserProfile?`; it retains local `_profile`, `_completed`, and error state and checks `mounted` before UI mutations.

Step 3 - Service / Repository Layer

`RestoreSession.call()` invokes `AuthRepository.restoreSession()`. In live mode, `FirebaseAuthRepository.restoreSession()` waits for `FirebaseAuth.instance.authStateChanges().first`. If a user exists it calls `user.getIdToken()` and then authenticated `GET /api/auth/me/`.

Step 4 - Backend / API

HTTP: `GET /api/auth/me/`, header `Authorization: Bearer <Firebase ID token>`. DRF runs `FirebaseAuthentication.authenticate()`, which calls Firebase Admin token verification with `check_revoked=True`, finds `User(firebase_uid=decoded['uid'], is_active=True)`, and rejects a non-admin without a club. `accounts.views.MeView.get()` serializes the local user.

Step 5 - Database

Table `accounts_user`: query filter `firebase_uid=<decoded uid>` and active status. Primary key `id`; `club_id` is checked for non-admin. This is read-only.

Step 6 - Backend Response

Success: HTTP 200 JSON from `UserSerializer` containing local identity/role/club profile fields. Missing/invalid/revoked token yields authentication error; missing/inactive/local-unscoped user yields 401/403. Other network/server failures propagate as `AuthException`.

Step 7 - Model Conversion

JSON map -> `UserProfile.fromJson()` in `domain/entities/user_profile.dart` -> typed Dart profile and role enum.

Step 8 - State Update

On a non-null profile, `_restore()` also starts `registerDeviceProvider` without blocking and calls `attendanceSyncServiceProvider?.start()`. It assigns `_profile` and marks completion with `setState`. No profile means logged-out completion. A 401/403 path signs Firebase out inside the repository; a non-auth failure becomes the screen error.

Step 9 - UI Update

`build()` returns `HomeScreen(profile: _profile!)` on success, `LoginScreen` when null, or `_RestoreErrorScreen` with retry/continue behavior on a recoverable error.

Call chain: `initState()` -> `_restore()` -> `restoreSessionProvider` -> `RestoreSession.call()` -> `FirebaseAuthRepository.restoreSession()` -> `authStateChanges().first` -> `getIdToken()` -> `GET /api/auth/me/` -> `FirebaseAuthentication.authenticate()` -> `MeView.get()` -> `UserSerializer` -> `UserProfile.fromJson()` -> `setState()` -> `HomeScreen`.

Variable Trace: Firebase UID and role

Firebase session user -> ID token -> decoded backend `uid` -> `accounts_user.firebase_uid` lookup -> `User.role` -> serialized role string -> `UserProfile.role` -> `HomeScreen` switch.

Failure path: no Firebase user/token -> null -> login; 401/403 -> Firebase sign-out -> auth error/null path; connection/5xx -> `AuthException` -> restore error screen, allowing retry rather than silently authorizing cached data.

Workflow 3: Super Admin Club / Coordinator Provisioning

Trigger

No public registration exists. An authenticated Super Admin submits the protected Club or Coordinator API/admin form.

Step 1 - UI

Super Admin enters Club details and selects School or Independent, then selects that valid Club while creating its one Coordinator.

Step 2 - State / Controller Layer

There is no Flutter controller. `AdminClubSerializer` and `AdminCoordinatorCreateSerializer` validate protected input, including active Club existence and coordinator uniqueness.

Step 3 - Service / Repository Layer

The Club endpoint saves the existing affiliation fields. The Coordinator endpoint calls `provision_club_coordinator`, which fixes the role and selected Club server-side.

Step 4 - Backend / API

`POST /api/admin/clubs/` and `POST /api/admin/coordinators/` require `IsAdmin`. Unauthorized roles receive 403; invalid or duplicate Club selection receives 400.

Step 5 - Database

Insert into `accounts_club` and `accounts_user` with role `COORDINATOR`, the exact selected `club_id`, and a hashed Django portal password. A conditional database constraint permits only one Coordinator per Club.

Step 6 - Backend Response

Success returns 201 and a one-time portal credential. Invalid selection or authorization fails without partial relational state.

Step 7 - Model Conversion

No Dart model. DRF serializer data becomes Django model instances.

Step 8 - State Update

The created Club/Coordinator is immediately visible to Super Admin and can be activated/deactivated through protected lifecycle controls.

Step 9 - UI Update

The protected console/admin surface refreshes; the public signup page remains informational and non-mutating.

Call chain: Super Admin request -> protected serializer/view -> Club save -> `provision_club_coordinator()` -> role/Club validation -> Coordinator save -> response.

Failure path: non-admin -> 403; invalid/inactive Club or duplicate Coordinator -> 400; transaction error -> no partial Coordinator row.

> Mobile/public self-registration is not implemented. Normal Club accounts are provisioned by the authenticated Club Coordinator.

Workflow 4: Login

Trigger

The `ElevatedButton.onPressed` in `presentation/screens/login_screen.dart` near line 147 invokes `_handleSignIn()` unless `LoginState.isLoading` is true.

Step 1 - UI

`_handleSignIn()` reads `_emailController.text.trim()` and `_passwordController.text`, then calls `ref.read(loginControllerProvider.notifier).signIn(email: ..., password: ...)`. The button disables and shows progress from watched controller state.

Step 2 - State / Controller Layer

`LoginController.signIn()` in `presentation/providers/auth_controllers.dart` sets `isLoading=true` and clears the previous error. It calls `ref.read(signInProvider)(email: email, password: password)`. Success stores no global role itself; it returns `UserProfile`. Failure stores a user-facing error in `LoginState` and returns null. This layer isolates async UI state from authentication mechanics.

Step 3 - Service / Repository Layer

`SignIn.call()` calls `FirebaseAuthRepository.signInAndFetchProfile(email,password)`. The repository calls `FirebaseAuth.instance.signInWithEmailAndPassword`, obtains the current user ID token, calls `/api/auth/me/`, and converts the local profile. Firebase exceptions are mapped by `_friendlyAuthMessage()`.

Step 4 - Backend / API

Operation 1: Firebase email/password sign-in. Operation 2: `GET /api/auth/me/` with Bearer ID token. `FirebaseAuthentication` performs token/user/club checks; `MeView.get()` returns `UserSerializer(request.user).data`.

Step 5 - Database

Read `accounts_user` by `firebase_uid`; verify `is_active` and required `club_id`. No password is read from Django for mobile login.

Step 6 - Backend Response

HTTP 200 user profile on success. Firebase invalid credentials, token failure, network failure, missing local user, inactive user, or forbidden club state return exceptions. The repository also signs the Firebase user back out if application-profile authorization fails after Firebase login.

Step 7 - Model Conversion

`jsonDecode(response.body)` -> `Map<String,dynamic>` -> `UserProfile.fromJson()`.

Step 8 - State Update

`LoginController` sets loading false. `_handleSignIn()` receives the profile, starts device registration and attendance sync, then checks `mounted`.

Step 9 - UI Update

`Navigator.of(context).pushReplacement(MaterialPageRoute(builder: (_) => HomeScreen(profile: profile)))`. On failure, the watched controller error is rendered on the login screen; the button re-enables.

Call chain: login button -> `_handleSignIn()` -> `LoginController.signIn()` -> `SignIn.call()` -> `FirebaseAuthRepository.signInAndFetchProfile()` -> Firebase `signInWithEmailAndPassword()` -> `getIdToken()` -> `/api/auth/me/` -> `UserProfile.fromJson()` -> `HomeScreen`.

Variable Trace: Email/password

`_emailController.text.trim()` + `_passwordController.text` -> named parameters of `LoginController.signIn` -> `SignIn` -> `signInAndFetchProfile` -> Firebase credential call. The password is not sent to Django or stored in a Dart model.

Failure path: no explicit empty-field screen validation -> Firebase rejects -> mapped `AuthException` -> controller `error` -> login error UI. This is a client UX limitation; the backend authorization boundary remains intact.

Workflow 5: Logout

Trigger

Portal app-bar/profile sign-out actions call role-screen `_signOut(...)` or `HomeScreen._signOut()`; examples are `home_screen.dart`, `player_dashboard_screen.dart`, and `guardian_dashboard_screen.dart`.

Step 1 - UI

The icon/button `onPressed` invokes the screen's async `_signOut` method.

Step 2 - State / Controller Layer

The method first awaits `ref.read(unregisterDeviceProvider)()` and then awaits `ref.read(signOutProvider)()`. Player/Guardian paths also clear the in-memory privacy-unlock store before leaving the portal.

Step 3 - Service / Repository Layer

`UnregisterDevice.call()` delegates to `ApiDeviceRepository.unregisterCurrentDevice()`: it obtains the current FCM token with a five-second timeout and asks the authenticated API to delete that token. The repository catches/logs token or HTTP failures so cleanup can never block logout. `SignOut.call()` then delegates to `FirebaseAuthRepository.signOut()`, which signs out of Firebase and clears that owner's durable GET cache.

Step 4 - Backend / API

Before Firebase sign-out, Flutter sends `DELETE /api/devices/` with JSON `{token}` and the current Bearer token. `DeviceRegisterView.delete()` scopes deletion to `user=request.user`; it cannot remove another account's token association. No separate Django session-logout endpoint is used, and server-side Firebase token revocation is not part of ordinary local logout.

Step 5 - Database

On successful unregister, the matching current-user row in `academy_devicetoken` is deleted. No training, player, attendance, assessment, or other academy domain row is changed. If token lookup or transport fails, the row may remain until later cleanup, but sign-out still proceeds.

Step 6 - Backend Response

The unregister endpoint returns HTTP 204 whether or not the scoped token row existed. Flutter accepts 200/204 and swallows/logs unregister failures; Firebase sign-out then returns `Future<void>`.

Step 7 - Model Conversion

None.

Step 8 - State Update

Firebase local auth state becomes signed out; guardian unlock tokens are cleared from the memory store. Screen awaits the operation and checks `context.mounted` where used.

Step 9 - UI Update

`Navigator` removes/replaces the authenticated route with `LoginScreen`, so back navigation does not reopen the portal.

Call chain: sign-out button -> `_signOut()` -> `unregisterDeviceProvider` -> `UnregisterDevice.call()` -> `ApiDeviceRepository.unregisterCurrentDevice()` -> FCM token -> `DELETE /api/devices/` -> scoped `DeviceToken.delete()` -> `signOutProvider` -> `SignOut.call()` -> Firebase sign-out/current-owner cache clear -> clear unlock store -> Navigator to `LoginScreen`.

Failure path: token absence, timeout, or unregister HTTP failure is logged and does not prevent Firebase sign-out. A Firebase sign-out exception can still interrupt navigation; the exact screen methods do not all expose a rich recovery message. Only the current user's matching device-token row is eligible for deletion.

Workflow 6: Role Checking and Authorization

Trigger

Every authenticated API request triggers DRF authentication; every endpoint method then performs its role/object rules. `HomeScreen.build()` also uses `profile.role` for UX routing.

Step 1 - UI

Flutter hides/chooses role interfaces using `UserProfile.role`, but this is not the trusted enforcement. A modified client can still construct requests.

Step 2 - State / Controller Layer

`UserProfile` carries the serialized role. Controllers do not grant permission; they only initiate calls and surface errors.

Step 3 - Service / Repository Layer

API repositories attach only the current Firebase ID token, not a claimed role/club. This deliberately leaves authority to Django.

Step 4 - Backend / API

`FirebaseAuthentication.authenticate()` maps token UID to `accounts.User`. Views compare `request.user.role` to `Roles.*`, filter/query by `request.user.club_id`, derive player from `request.user` where appropriate, and call guardian-link/unlock helpers. `accounts/permissions.py` supplies reusable role permissions for admin endpoints.

Step 5 - Database

`accounts_user.role` and `club_id`, `accounts_guardianlink`, target object foreign keys, and `academy_playerprivacypin`/signed token state participate. There is no RLS query because Django is the only database client.

Step 6 - Backend Response

Allowed requests return serialized data; invalid identity returns 401; authenticated but disallowed role/object/club returns 403 (or a scoped 404 in selected cases). Serializer errors return 400; PIN lock can return 423.

Step 7 - Model Conversion

Successful role is parsed into the Dart role enum; errors become repository exceptions rather than domain objects.

Step 8 - State Update

Controller/provider becomes data or error. No client-side state can convert a 403 into authorized data.

Step 9 - UI Update

Home routing displays the role portal; feature widgets show error states when the backend rejects access.

Call chain: Firebase user -> ID token -> API repository header -> `FirebaseAuthentication.authenticate()` -> local `User` -> view role check -> club/object/link/unlock check -> ORM query/response.

Defense answer - player pretending to be coach: Changing a Flutter enum or payload does not change `request.user.role`. Django obtains UID from a verified token, loads `accounts_user.role`, and coach-only endpoints such as `PlayerAssessmentView.put` and `SessionAttendanceView.post` reject any non-coach.

Workflow 7: Navigation After Login

Trigger

Successful `_handleSignIn()` pushes `HomeScreen(profile: profile)`; restored sessions directly build the same destination.

Step 1 - UI

`HomeScreen.build()` switches on `profile.role`.

Step 2 - State / Controller Layer

The typed `UserProfile` came from server JSON. No additional controller is used for routing.

Step 3 - Service / Repository Layer

No new service call is needed to choose the initial portal; portal children then watch their own providers.

Step 4 - Backend / API

Role came from `/api/auth/me/`, not from a route parameter.

Step 5 - Database

Role came from `accounts_user.role` during the preceding profile read.

Step 6 - Backend Response

The response's role string is the routing input.

Step 7 - Model Conversion

`UserProfile.fromJson()` converts the wire role to the domain role enum.

Step 8 - State Update

No route-state framework; `HomeScreen` returns a different widget.

Step 9 - UI Update

`COORDINATOR` -> `CoordinatorPortalScreen(profile)`; `COACH` -> `CoachPortalScreen(profile)`; `PLAYER` -> `PlayerPortalScreen`; `GUARDIAN` -> `GuardianPortalScreen`; Admin/School Staff use their supported web/admin surface or authenticated fallback. Feature transitions use `Navigator.push/pushReplacement` with `MaterialPageRoute`; adaptive tab shells use `IndexedStack` with bottom navigation or navigation rail.

Call chain: `/api/auth/me/` role -> `UserProfile.fromJson()` -> `HomeScreen.build()` switch -> role portal -> portal tab `IndexedStack`.

Failure path: unexpected/non-mobile role does not get coach capabilities; it receives a generic screen and can sign out.

Workflow 8: Player Profile Retrieval

Trigger

`PlayerDashboardScreen.build()` watches `myProfileProvider` after a player passes the privacy setup gate. Coach roster and guardian detail use separate providers but converge on `ApiPlayerRepository` and `Player.fromJson()`.

Step 1 - UI

Player dashboard consumes an `AsyncValue<Player>` and renders loading/error/data. Guardian unlocked content watches `selectedChildDetailsProvider(playerId)`; coach squad/profile uses `squadProvider`/selected `Player`.

Step 2 - State / Controller Layer

`myProfileProvider` calls `ref.watch(getMyProfileProvider)()`. It exists so widgets depend on typed async state rather than transport code.

Step 3 - Service / Repository Layer

`GetMyProfile.call()` calls `ApiPlayerRepository.fetchMyProfile()`, which delegates `_get('/api/players/me/')` to the shared `AuthenticatedApiClient`. The client resolves the current Firebase UID/token, applies the common timeout/error policy, and performs the GET. Guardian detail calls `fetchPlayerDetails(playerId, unlockToken)` and adds `X-Player-Unlock`; that protected response is deliberately excluded from durable caching.

Step 4 - Backend / API

Player: `GET /api/players/me/` -> `MyProfileView.get()` -> role must be `PLAYER` -> get `PlayerProfile(user=request.user)`. Guardian: `GET /api/players/{id}/profile/` -> `PlayerDetailView.get()` -> `_guardian_may_read` + `_require_unlock_when_pin_exists`. Coach roster: `GET /api/players/` -> `SquadListView.get()` and club filter.

Step 5 - Database

Read `academy_playerprofile` joined to `accounts_user`; nested fields include current ratings, eligibility, position, notes, and `photo_path`. Primary profile relation is one-to-one with player user. Storage helper may turn `photo_path` into a signed URL in serialization.

Step 6 - Backend Response

`PlayerSerializer` returns a player JSON object/list. Missing profile returns 404; wrong role/object returns 403; guardian may receive an unlock-related error.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> nested `PlayerRatings.fromJson()`, `AgeTierInfo.fromWire()`, `PlayerPositionInfo.fromWire()`, and `EligibilityStatusLabel.fromWire()`.

Step 8 - State Update

Riverpod resolves the `FutureProvider` to `AsyncData<Player>`; exception becomes `AsyncError`. `autoDispose` releases state when no longer watched.

Step 9 - UI Update

`PlayerDashboardScreen`/guardian content uses cards, rating/eligibility widgets, and image fallback. Riverpod rebuilds consumers when the async value changes.

Call chain: `PlayerDashboardScreen` -> `myProfileProvider` -> `GetMyProfile.call()` -> `ApiPlayerRepository.fetchMyProfile()` -> `_get('/api/players/me/')` -> `MyProfileView.get()` -> `PlayerSerializer` -> `Player.fromJson()` -> `AsyncData` -> dashboard.

Object: Player

Created from `PlayerSerializer` JSON by `Player.fromJson`; stored transiently in Riverpod async state; consumed by dashboards, player cards, assessment/position screens; displays identity, tier, position, 12 ratings, eligibility, signed photo URL, and coach notes.

Workflow 9: Player Profile Update (Position)

The repository has no general player self-profile editor. The verified coach-controlled profile update is position assignment; assessment is Workflow 15.

Trigger

The coach opens a player profile/position picker and selects a `PlayerPosition`; the calling screen invokes the position controller's save action (provider in `presentation/providers/player_position_controller.dart`).

Step 1 - UI

`position_picker_sheet.dart` returns the chosen enum to the caller. The player ID comes from the selected `Player`; the wire value comes from `PlayerPosition.wire`.

Step 2 - State / Controller Layer

`PlayerPositionController` enters async loading and invokes `savePlayerPositionProvider(playerId, position)`. Success invalidates squad/player-derived state; failure stores `AsyncError`.

Step 3 - Service / Repository Layer

`SavePlayerPosition.call()` delegates to `ApiPlayerRepository.savePosition(playerId, position)`, which sends JSON `{'position': position.wire}` after obtaining an ID token.

Step 4 - Backend / API

`PUT /api/players/{playerId}/position/` with Bearer token and JSON. `PlayerPositionView.put()` requires `COACH`, loads `PlayerProfile`, verifies equal `club_id`, validates `PlayerPositionSerializer`, saves, and audits.

Step 5 - Database

Update `academy_playerprofile.position` for the row whose one-to-one user ID equals `playerId`; add `academy_auditlog` action `position.changed` with actor FK.

Step 6 - Backend Response

HTTP 200 `PlayerSerializer(profile).data`; 403 for non-coach/cross-club, 404 missing player, 400 invalid position. Repository maps non-200 to `PlayerRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `Player.fromJson()` with `PlayerPositionInfo.fromWire()`.

Step 8 - State Update

Controller clears loading, returns updated `Player`, and invalidates roster state.

Step 9 - UI Update

The sheet/screen closes or updates, and rebuilt roster/profile cards display the new position.

Call chain: position selection -> `PlayerPositionController` -> `SavePlayerPosition.call()` -> `ApiPlayerRepository.savePosition()` -> `PUT /api/players/{id}/position/` -> `PlayerPositionView.put()` -> `PlayerPositionSerializer.save()` -> `PlayerSerializer` -> `Player.fromJson()` -> provider invalidation -> position label.

Failure path: no selection/cancel -> no request; invalid/cross-club/non-coach -> backend error -> repository/controller error -> UI remains unchanged.

Workflow 10: Creating a Scouting Report

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No scouting-report button, form, widget, event handler, or `SCOUT` role exists.

Step 1 - UI

No scouting-report screen/file exists.

Step 2 - State / Controller Layer

No scouting controller/provider/use case exists.

Step 3 - Service / Repository Layer

No scouting repository/service exists.

Step 4 - Backend / API

No scouting endpoint or method exists.

Step 5 - Database

No scouting table/model/migration exists.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

No scouting Dart model exists.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable. The coach dispute flow must not be presented as scouting.

Workflow 11: Retrieving Scouting Reports

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No report-list trigger exists.

Step 1 - UI

No scouting list/card exists.

Step 2 - State / Controller Layer

No scouting read provider exists.

Step 3 - Service / Repository Layer

No retrieval method exists.

Step 4 - Backend / API

No scouting GET route exists.

Step 5 - Database

No scouting records exist.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable.

Workflow 12: Editing a Scouting Report

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No scouting edit action exists.

Step 1 - UI

No edit form.

Step 2 - State / Controller Layer

No edit controller.

Step 3 - Service / Repository Layer

No update repository method.

Step 4 - Backend / API

No PUT/PATCH scouting endpoint.

Step 5 - Database

No scouting table to update.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable.

Workflow 13: Training Session Creation

Trigger

The add/schedule button in `presentation/screens/training_schedule_screen.dart` calls `_openScheduleForm()`, which pushes `ScheduleSessionScreen`. Its save `ElevatedButton.onPressed` near line 324 calls `_submit()`.

Step 1 - UI

`ScheduleSessionScreen` holds title/location controllers and selected `_date`, `_startTime`, `_endTime`, `_selectedTiers`, `_focus`. `_submit()` rejects empty fields, missing tiers, and invalid date conditions, then constructs an immutable `TrainingSession` draft. The server remains authoritative for the paired time-window invariant.

Step 2 - State / Controller Layer

`ScheduleSessionController.submit(draft)` in `training_schedule_providers.dart` calls `_run`, sets `AsyncLoading`, awaits `scheduleTrainingSessionProvider(draft)`, and stores `AsyncData` or `AsyncError`. It prevents duplicate save through `isLoading` UI disabling.

Step 3 - Service / Repository Layer

`ScheduleTrainingSession.call(draft)` calls `TrainingRepository.createSession`. `ApiTrainingRepository.createSession()` converts `draft.toJson()` and delegates the authenticated POST to `AuthenticatedApiClient`.

Step 4 - Backend / API

`POST /api/training-sessions/`; headers Bearer token + JSON content type. Payload: `id`, `title`, `ageTiers`, ISO date-only `date`, display-string `startTime`, `endTime`, `location`, uppercase `focus`, and `attendeeCount`. `TrainingSessionListCreateView.post()` requires coach; `TrainingSessionSerializer.validate()` calls `TrainingSession.validate_time_window()` so start/end must be supplied together, match the 12-hour wire format, and end later on the same day. The model repeats this through `clean()`/`save()` for non-API writes. The view assigns `created_by=request.user`, `club=request.user.club`.

Step 5 - Database

Insert `academy_trainingsession`: generated `id` PK, title/date/start/end/location/focus, JSON age tiers, `created_by_id` FK to `accounts_user`, `club_id` FK to `accounts_club`, timestamps. Insert an `academy_auditlog` event. On commit, notification code reads related club users/device tokens.

Step 6 - Backend Response

HTTP 201 with `TrainingSessionSerializer(session).data`. Invalid form/payload -> 400; non-coach/cross-auth -> 403/401; DB failure -> 5xx/rollback. Flutter accepts 200/201 and otherwise throws `TrainingRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `TrainingSession.fromJson()`: string ID, parsed date, tier enum set, focus enum, attendee count.

Step 8 - State Update

Controller resolves; the caller/provider invalidates session data (through its workflow wiring) and `_submit()` receives success. Error is read from controller state.

Step 9 - UI Update

Success `Navigator.pop(true)` returns to schedule and refreshed providers rebuild session lists/cards. Failure shows a `SnackBar`; the form remains.

Call chain: add button -> `_openScheduleForm()` -> `ScheduleSessionScreen` -> save button -> `_submit()` -> `ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()` -> `ApiTrainingRepository.createSession()` -> `TrainingSession.toJson()` -> `POST /api/training-sessions/` -> `TrainingSessionListCreateView.post()` -> serializer/model/audit/on-commit FCM -> `TrainingSession.fromJson()` -> pop/refresh.

Variable Trace: Training ID

New draft uses a client placeholder/empty ID -> server ignores/generates model PK -> serializer returns `id` -> `TrainingSession.fromJson` stores it as String -> edit/cancel/log-attendance routes use `session.id`.

Failure path: UI validation prevents an incomplete request; missing token throws "Not signed in"; malformed/unpaired/reversed times return field-specific 400 errors; a transport failure maps to the shared reach-server error; another non-success status remains an HTTP error; controller `AsyncError` -> SnackBar.

Workflow 14: Training Attendance Recording

Trigger

Coach taps a session card in `training_schedule_screen.dart`; `_logAttendance(session)` pushes `LogAttendanceScreen`. In that screen, `_FinalizeBar` button near line 915 calls `_finalize(roster)` when the session window is open and at least one mark exists.

Step 1 - UI

`_marks: Map<String, AttendanceStatus?>`, effort values, and note fields hold inputs. `_finalize()` checks the two-day attendance window, handles unmarked-player confirmation, and builds `List<Attendance>`. Effort/note are included only for `PRESENT`; non-present records clear them.

Step 2 - State / Controller Layer

`AttendanceLogController.save(session.id, records)` sets `AsyncLoading`, awaits `logSessionAttendanceProvider(sessionId,records)`, invalidates `sessionAttendanceProvider(sessionId)` on success, and stores `AsyncError` on failure. This separates transient field state from mutation state.

Step 3 - Service / Repository Layer

`LogSessionAttendance.call()` invokes the injected `AttendanceRepository`. Live composition is `OfflineFirstAttendanceRepository`, which first calls `ApiAttendanceRepository.saveSessionAttendance()`. The API adapter creates `{'records': records.map(toJson)}`. Only `AttendanceNetworkException` causes the decorator to require `_ownerUid`, enqueue JSON, and return optimistic records.

Step 4 - Backend / API

`POST /api/attendance/session/{sessionId}/`, Bearer + JSON. `SessionAttendanceView.post()` requires `COACH`, same-club session, and days-since 0-2. `SessionAttendanceRecordSerializer(many=True)` validates each row. It verifies every `playerId` belongs to the coach's club.

Step 5 - Database

Within `transaction.atomic()`, for each validated row: `Attendance.objects.update_or_create(player_id, session, defaults={status, effort, note, recorded_by})`. Table `academy_attendance`; PK `id`; FKs `player_id`, nullable `session_id`, `recorded_by_id`; unique pair `(player,session)`. Existing rows for the session whose player IDs were omitted are deleted. Offline branch instead inserts `outbox_attendance(owner_uid,session_id,records_json,created_at,retry_count,last_error)` on device.

Step 6 - Backend Response

HTTP 200 list from `AttendanceSerializer`. Possible 400: window/record validation; 401/403: identity/role/club; network: typed network exception; 5xx: generic repository error. Offline queueing occurs only on network exception, not HTTP errors.

Step 7 - Model Conversion

JSON list -> `_decodeRecords()` -> `Attendance.fromJson()` for each row. Offline optimistic objects are the same `Attendance` instances created by the UI; queued JSON later becomes `Attendance.fromJson()` during replay.

Step 8 - State Update

Online: controller becomes data and invalidates session attendance. Offline: decorator returns records so controller treats it as accepted-for-sync; outbox persists pending state. `AttendanceSyncService.start()` listens/initiates replay; it processes owner rows oldest first, deletes success, increments retry/error and stops on failure.

Step 9 - UI Update

Success/queued acceptance pops `LogAttendanceScreen(true)` and shows caller feedback. Refetched session/player attendance cards display saved status/effort/note after server sync. Failure keeps the screen and shows controller error through a SnackBar.

Call chain: session card -> `_logAttendance()` -> `LogAttendanceScreen` -> finalize button -> `_finalize()` -> `AttendanceLogController.save()` -> `LogSessionAttendance.call()` -> `OfflineFirstAttendanceRepository.saveSessionAttendance()` -> `ApiAttendanceRepository.saveSessionAttendance()` -> POST -> `SessionAttendanceView.post()` -> atomic upsert/prune -> `AttendanceSerializer` -> `Attendance.fromJson()` -> invalidate/pop.

Offline call chain: API transport failure -> `AttendanceNetworkException` -> `AttendanceOutbox.enqueue()` -> local `outbox_attendance` -> `AttendanceSyncService` -> live repository retry -> Django endpoint -> success -> outbox delete.

Variable Trace: Attendance status

Selector enum `AttendanceStatus` -> `_marks[player.id]` -> `Attendance(status: ...)` -> `Attendance.toJson()['status']` uppercase -> request `records[]` -> serializer choice -> `academy_attendance.status` -> `AttendanceSerializer.status` -> `AttendanceStatusWire.fromWire()` -> chip/summary UI.

Failure path: unmarked/closed window -> SnackBar/dialog, no API; missing token -> repository error, no queue; connection failure -> queue if owner exists, otherwise "Not signed in"; backend 400/403/500 -> no queue and controller error; replay failure -> retry metadata retained.

Workflow 15: Coach Evaluation / Assessment

Trigger

Coach opens `EditPerformanceDataScreen` for a player and presses its save `ElevatedButton.onPressed` near line 246, invoking `_save()`.

Step 1 - UI

The screen's rating values/controllers and coach-notes controller contain the input. `_save()` constructs `PlayerRatings` for the six outfield and six goalkeeper attributes and reads trimmed notes, then calls the controller with the selected `Player`.

Step 2 - State / Controller Layer

`EditPerformanceController.submit(player, ratings, coachNotes)` sets `AsyncLoading`, calls `savePlayerAssessmentProvider`, stores `AsyncData` and invalidates squad state on success, or `AsyncError` and returns null on failure.

Step 3 - Service / Repository Layer

`SavePlayerAssessment.call(player.id, ratings, coachNotes: ...)` -> `ApiPlayerRepository.saveAssessment()`. It JSON-encodes `{'ratings': ratings.toJson(), 'coachNotes': coachNotes}` and delegates the authenticated PUT to `AuthenticatedApiClient`.

Step 4 - Backend / API

`PUT /api/players/{playerId}/assessment/`. `PlayerAssessmentView.put()` requires `COACH`, loads the profile, compares player/coach `club_id`, runs `AssessmentSerializer(profile,data,partial=True)`, saves, audits `assessment.saved`, and schedules `notify_assessment_saved(profile)` on commit.

Step 5 - Database

Update the existing `academy_playerprofile` row: `pace`, `shooting`, `passing`, `dribbling`, `defending`, `physical`, `diving`, `handling`, `kicking`, `reflexes`, `speed`, `positioning`, and `coach_notes`. Add `academy_auditlog`. No assessment-history row is created; the current values overwrite.

Step 6 - Backend Response

HTTP 200 `PlayerSerializer(profile).data`. Serializer rejects out-of-range ratings (0-99 contract) or malformed data. Wrong role/club yields 403; missing player 404; transport/non-200 becomes `PlayerRepositoryException`.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> nested `PlayerRatings.fromJson()` and related enums.

Step 8 - State Update

Controller leaves loading, returns updated `Player`, and invalidates the squad provider so all consumers can reload authoritative data.

Step 9 - UI Update

`_save()` receives non-null `saved`, calls `Navigator.pop(saved)`, and previous player/card screens update from the return/refetch. Failure reads controller error and displays a SnackBar.

Call chain: save evaluation button -> `_save()` -> `EditPerformanceController.submit()` -> `SavePlayerAssessment.call()` -> `ApiPlayerRepository.saveAssessment()` -> `PUT /api/players/{id}/assessment/` -> `PlayerAssessmentView.put()` -> `AssessmentSerializer.save()` -> profile/audit/on-commit push -> `PlayerSerializer` -> `Player.fromJson()` -> provider invalidation/pop.

Variable Trace: Performance score

Rating UI integer -> `PlayerRatings.<field>` -> `ratings.toJson()` -> nested `ratings` request -> `AssessmentSerializer` validated integer 0-99 -> `PlayerProfile.<field>` -> serializer nested ratings -> `PlayerRatings.fromJson()` -> `Player.overall` averages applicable six -> card number/radar UI.

Failure path: malformed/range input should be stopped by UI/serializer; network/non-200 -> controller `AsyncError` -> SnackBar; database rollback prevents response and on-commit push. There is no old assessment record to recover except external backup/audit metadata.

Workflow 16: Performance Data Retrieval

Trigger

Coach/player/guardian screens watch a player provider; for example `PlayerDashboardScreen` watches `myProfileProvider`, and coach roster screens watch `squadProvider`.

Step 1 - UI

A `ConsumerWidget` reads an `AsyncValue<Player>` or `AsyncValue<List<Player>>`; current ratings and coach notes are rendered on player cards/profile/radar widgets.

Step 2 - State / Controller Layer

`myProfileProvider` -> `getMyProfileProvider`; `squadProvider` -> `getSquadProvider`; guardian full detail -> `selectedChildDetailsProvider` -> `getPlayerDetailsProvider`. These `FutureProvider`s hold loading/data/error state.

Step 3 - Service / Repository Layer

The corresponding use cases call `ApiPlayerRepository.fetchMyProfile()`, `fetchSquad()`, or `fetchPlayerDetails()`. The repository adds Firebase auth and, for guardians, an unlock header.

Step 4 - Backend / API

`GET /api/players/me/`, `GET /api/players/`, or `GET /api/players/{id}/profile/`. `MyProfileView`, `SquadListView`, or `PlayerDetailView` applies role/club/link/PIN rules and returns `PlayerSerializer`.

Step 5 - Database

Read the current row in `academy_playerprofile` and related `accounts_user`; columns include the 12 ratings and `coach_notes`. No historical assessment query exists.

Step 6 - Backend Response

One player map or list of maps. Null/missing optional position/photo/notes are normalized by serializer/model defaults. 401/403/404/network errors become provider error states.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> `PlayerRatings.fromJson()`; `Player.overall` deterministically averages the relevant six attributes based on position group.

Step 8 - State Update

Riverpod `FutureProvider` resolves to `AsyncData`; refetch occurs after assessment/position provider invalidation.

Step 9 - UI Update

Player card, attribute radar/stat widgets, overall value, eligibility badge, and coach-note text rebuild from the typed `Player`.

Call chain: player consumer -> player `FutureProvider` -> get-player use case -> `ApiPlayerRepository` GET -> Django player view -> `PlayerProfile/PlayerSerializer` -> `Player.fromJson()/PlayerRatings.fromJson()` -> `AsyncData` -> card/radar/text.

Failure path: auth/network/server exception -> `AsyncError` -> dashboard-state error/retry UI; a missing optional photo uses avatar fallback rather than failing the whole player.

Workflow 17: Performance Dashboard Generation

Trigger

Opening `presentation/screens/coach_progress_screen.dart` causes `CoachProgressScreen.build()` to watch `squadProgressProvider`.

Step 1 - UI

The `ConsumerWidget` renders loading, error, or a squad summary plus `_PlayerProgressCard` list.

Step 2 - State / Controller Layer

`squadProgressProvider` in `progress_providers.dart` calls `ref.watch(getSquadProgressProvider)()` and owns `AsyncValue<List<PlayerProgress>>`.

Step 3 - Service / Repository Layer

`GetSquadProgress.call()` -> `ApiProgressRepository.fetchSquadProgress()` -> shared `AuthenticatedApiClient` -> authenticated GET.

Step 4 - Backend / API

`GET /api/progress/squad/`. `SquadProgressView.get()` allows Coach/Super Admin. It filters profiles and attendance to `request.user.club_id` only for a Coach; the Super Admin keeps the all-club querysets. It uses ORM `Count` filters plus `Avg('effort')`.

Step 5 - Database

Read `academy_playerprofile` joined with `accounts_user`; aggregate `academy_attendance` grouped by `player_id`. Returned values: present/absent/excused counts and average effort; no row is inserted.

Step 6 - Backend Response

HTTP 200 list of `{id,name,position,ageTier,present,absent,excused,avgEffort}`. Players with no attendance receive zeros/null average. Non-coach/admin gets 403.

Step 7 - Model Conversion

JSON list -> `.map(PlayerProgress.fromJson)` -> typed immutable progress objects.

Step 8 - State Update

Provider resolves `AsyncData<List<PlayerProgress>>`; error becomes `AsyncError` and refresh can rerun the future.

Step 9 - UI Update

`_SquadSummary` and each `_PlayerProgressCard` calculate/display the returned deterministic metrics.

Call chain: Progress tab -> `CoachProgressScreen` -> `squadProgressProvider` -> `GetSquadProgress.call()` -> `ApiProgressRepository.fetchSquadProgress()` -> `SquadProgressView.get()` -> ORM Count/Avg -> `PlayerProgress.fromJson()` -> cards.

Failure path: network/non-200 -> error UI. Role branching is explicit: Coach is club-scoped; Super Admin receives genuine all-club progress rather than filtering on the Admin's normally-null club.

Workflow 18: Academic Eligibility Retrieval

Trigger

Player/guardian taps the eligibility tile; `Navigator.push` opens `EligibilityHistoryScreen`, whose `build()` watches `eligibilityHistoryProvider(playerId)`.

Step 1 - UI

`EligibilityHistoryScreen` receives `playerId` and uses a `ConsumerWidget` to render loading/error/list of `_ChangeCard` records.

Step 2 - State / Controller Layer

The family `FutureProvider` invokes `getEligibilityHistoryProvider(playerId)`; guardian token state is injected through the repository composition.

Step 3 - Service / Repository Layer

`GetEligibilityHistory.call()` -> `ApiEligibilityHistoryRepository.fetchHistoryForPlayer(playerId)` -> ID token + optional `X-Player-Unlock` -> HTTP GET.

Step 4 - Backend / API

`GET /api/players/{playerId}/eligibility-history/`. `EligibilityHistoryView.get()` calls `_may_read_eligibility`, verifies the player, applies PIN unlock when configured, and serializes ordered history.

Step 5 - Database

Read `academy_eligibilityhistory` filtered `player_id`, joined to optional `changed_by_id`; player FK references `accounts_user`. Current status itself is also present on `academy_playerprofile.eligibility`.

Step 6 - Backend Response

HTTP 200 history list; empty list is valid. Role/link/PIN failure -> 403/validation response; missing privileged target may be 404. Actor display is shaped for the viewer and may be null.

Step 7 - Model Conversion

Each JSON map -> `EligibilityChange.fromJson()` with old/new status and timestamp.

Step 8 - State Update

Riverpod family provider becomes data/error for that player ID.

Step 9 - UI Update

List cards show chronological eligibility transitions. Eligibility badge/current status elsewhere comes from the player profile.

Call chain: eligibility tile -> `Navigator.push(EligibilityHistoryScreen)` -> `eligibilityHistoryProvider(id)` -> `GetEligibilityHistory.call()` -> API repository -> GET -> `EligibilityHistoryView.get()` -> history queryset/serializer -> `EligibilityChange.fromJson()` -> list.

Failure path: guardian without valid in-memory unlock receives denial; provider renders error/retry. No grades are retrieved because no grade data exists.

Workflow 19: Academic Eligibility Update

Trigger

School staff submits the server-rendered form handled by `backend/portal/views.py:staff_eligibility(request)` near line 314.

Step 1 - UI

The Django portal page posts selected club player and new eligibility status using a CSRF-protected form. There is no Flutter eligibility-edit button.

Step 2 - State / Controller Layer

No Flutter controller. Django form validation checks the supplied choice/player; portal decorators/session middleware ensure authenticated staff role.

Step 3 - Service / Repository Layer

The view calls `portal.services.set_player_eligibility(staff=request.user, player_profile=..., new_status=...)`. `_assert_same_club`/service logic prevents cross-club updates, sets the actor marker, and saves the profile.

Step 4 - Backend / API

Session-authenticated portal POST, CSRF protected. It is not a public DRF PUT. Django role decorator and service enforce `SCHOOL_STAFF` and club scope.

Step 5 - Database

Update `academy_playerprofile.eligibility`. `signals.stash_previous_eligibility` reads the previous value before save. `fire_eligibility_changed` inserts `academy_eligibilityhistory(player_id,changed_by_id,old_status,new_status,changed_at)` plus `academy_auditlog` and schedules FCM after commit.

Step 6 - Backend Response

Valid update redirects/rerenders with a success message. Form, permission, or service validation rerenders/rejects; transaction/save failure prevents a normal success response.

Step 7 - Model Conversion

No immediate Dart conversion in the portal. Later mobile retrieval converts history with `EligibilityChange.fromJson()` and current status with `Player.fromJson()`.

Step 8 - State Update

Portal messages/session response update. Mobile providers see the new data on refresh/refetch; no real-time stream automatically patches the screen.

Step 9 - UI Update

Portal shows success/current value. Refreshed player/guardian mobile badge and history screen display the updated status/event.

Call chain: staff form submit -> `staff_eligibility()` -> form/role/club validation -> `set_player_eligibility()` -> `PlayerProfile.save()` -> `stash_previous_eligibility()` -> `fire_eligibility_changed()` -> history/audit + on-commit FCM -> redirect -> later mobile GET/refetch.

Variable Trace: Eligibility status

Form choice -> cleaned `new_status` -> service -> `PlayerProfile.eligibility` -> pre/post signal old/new -> `EligibilityHistory.old_status/new_status` -> serializer -> `EligibilityChange/Player.eligibility` -> badge/cards.

Failure path: unauthorized role/club -> forbidden; invalid choice/player -> form error; unchanged value -> no new change event; notification failure is designed not to invalidate the saved status.

Workflow 20: Notifications

Trigger

Three linked triggers exist: (a) successful login/restore calls `registerDeviceProvider`; (b) schedule/assessment/eligibility mutations call notification helpers, usually through `transaction.on_commit`; and (c) a bell tap, inbox-row tap, foreground FCM **View** action, opened push, or initial push enters the notification read/routing flow.

Step 1 - UI

Registration is initiated without blocking the login/bootstrap UI. `NotificationBell` watches the unread-count provider, displays a badge, and pushes `NotificationInboxScreen`. In live mode, a foreground message shows a SnackBar with a **View** action. That action, an opened/initial push, and an inbox-row tap use the same destination policy: `session_*` enters the role portal's schedule tab; `assessment_saved` enters the Player's own or Guardian's authorized child profile tab; `eligibility_changed` opens eligibility history for the Player or authorized child; unknown/unsupported events stay in a focused inbox. Player payload IDs are ignored in favor of the current profile, and a Guardian payload ID is only a hint that must match the linked-child list; protected destinations still enforce PIN/privacy gates.

Step 2 - State / Controller Layer

`RegisterDevice` is invoked after auth. `notificationsProvider`, `notificationUnreadCountProvider`, and `notificationActionsProvider` own inbox loading and read-state refresh. `notificationNavigationControllerProvider` re-fetches the server-authoritative current profile, resolves a role-appropriate `NotificationDestination`, suppresses duplicate active callbacks, and marks the represented inbox record read best-effort. Mutation controllers do not send notifications directly; backend business events do.

Step 3 - Service / Repository Layer

Flutter `ApiDeviceRepository` obtains `FirebaseMessaging.instance.getToken()`/platform data and POSTs through the `DeviceRepository` contract. `ApiNotificationRepository` uses `AuthenticatedApiClient` for list/count/mark-read operations and converts JSON to `AppNotification`. Backend `academy.notifications._send_to_users()` resolves active recipients, persists one `NotificationRecord` per recipient, and then performs best-effort Firebase Admin fanout; invalid tokens can be removed.

Step 4 - Backend / API

Registration: `POST /api/devices/` JSON `{token, platform}` with Bearer auth -> `DeviceRegisterView.post()`. Logout cleanup uses `DELETE /api/devices/` JSON `{token}` -> `DeviceRegisterView.delete()`, scoped to the current user. Inbox APIs are `GET /api/notifications/`, `GET /api/notifications/unread-count/`, `PATCH /api/notifications/{id}/read/`, and `POST /api/notifications/read-all/`; every query is scoped to `request.user`. Sending is triggered for session scheduled/updated/cancelled, assessment saved, and eligibility changed. For cancellation, the view snapshots recipients, deletes the session, and schedules `notify_session_cancelled(...)` with `transaction.on_commit`, so a failed delete cannot emit a false cancellation.

Step 5 - Database

`academy_devicetoken`: registration uses `update_or_create(token=..., defaults={user,platform})`; logout deletes only a row matching both the authenticated user and token. The token is unique and references `accounts_user`. `academy_notificationrecord` stores recipient, event type, title, body, neutral JSON data, nullable `read_at`, and `created_at`, indexed by user/read/time. Notification events create inbox rows before attempting FCM.

Step 6 - Backend Response

Device registration and unregister return HTTP 204. Inbox endpoints return the current user's newest records/count/read result. FCM failures are logged/swallowed after inbox persistence, so the business write and notification history survive unavailable Firebase; exact device delivery still depends on Firebase, Apple/Android configuration, OS permission, and device state.

Step 7 - Model Conversion

Inbox JSON is converted by `AppNotification.fromJson()` to a Dart object containing ID, type, title, body, data, read state, and timestamp. An inbox row or FCM data map becomes `NotificationOpenRequest`; current role plus event type becomes a `NotificationDestination`. IDs remain routing hints rather than authorization grants. Device registration still uses primitive token/platform strings.

Step 8 - State Update

Device-token and token-refresh registration keep delivery endpoints current. Foreground/opened handlers invalidate inbox/count providers; mark-one/mark-all actions also invalidate those providers after success. Opening a notification marks the exact ID, or the newest matching type/domain-ID row, read best-effort. An active-request key prevents duplicate navigation callbacks for the same delivery while its route operation is in progress.

Step 9 - UI Update

Foreground pushes display a SnackBar; the bell shows the inbox with loading, retry, pull-to-refresh, unread styling, and mark-read actions. Known opened/initial/foreground/inbox-row events enter the schedule, profile, or eligibility destination. Guardian child selection is resolved from the authenticated linked-child provider, and Player/Guardian privacy gates remain in force. Unknown events and profile-resolution failures fall back to the focused current-user inbox.

Call chains: login success -> `registerDeviceProvider` -> `RegisterDevice` -> `ApiDeviceRepository.registerCurrentDevice()` -> Firebase messaging token -> `POST /api/devices/` -> `DeviceRegisterView.post()` -> `DeviceToken.update_or_create()`.

Business mutation -> transaction commit -> `notify_*` helper -> `_send_to_users()` -> `NotificationRecord.bulk_create()` -> device-token queryset -> best-effort Firebase Admin FCM send.

Bell -> `NotificationInboxScreen` -> notification providers/repository -> current-user inbox endpoints -> `NotificationRecordSerializer` -> `AppNotification.fromJson()` -> badge/list/read-state UI.

Opened/initial/foreground **View**/inbox row -> `NotificationOpenRequest` -> `NotificationNavigationController.open()` -> re-resolve `/api/auth/me/` -> `resolveNotificationDestination()` -> best-effort mark-read + duplicate suppression -> `NotificationDestinationScreen` -> role portal initial tab/protected detail, or focused inbox fallback.

Failure path: token unavailable/request failure does not block login because registration is unawaited; invalid/stale FCM tokens can be cleaned. FCM/permission/device-delivery failure does not remove the persisted inbox record. Inbox network/server errors render retryable UI. A missing current profile or unknown event falls back to the inbox; an untrusted child ID cannot bypass linked-child lookup or privacy gates. Inbox/router behavior is covered by local mocked tests, while real Firebase/APNs transport and notification presentation still require a signed physical-device verification.

Workflow 21: File/Image Upload

Trigger

There are two UI entry points: a Coordinator portal file form posts to `portal.views.player_photo(request, player_id)`, and a Coach viewing a roster player taps **Update player photo** in `PlayerProfileScreen`, which invokes `_pickAndUploadPhoto()`. The protected DRF route is `POST /api/players/{id}/photo/` with the older `/api/admin/players/{id}/photo/` alias handled by the same `PlayerPhotoUploadView.post()`.

Step 1 - UI

The Coordinator selects an image in the server-rendered portal. In Flutter, the Coach uses `ImagePicker` to select a gallery image; the screen accepts JPEG/PNG/WebP content types, reads bytes, disables the action while uploading, and displays success or failure feedback.

Step 2 - State / Controller Layer

Flutter calls `PlayerPhotoController.submit()`, which invokes the `UploadPlayerPhoto` use case through `PlayerPhotoWriter`, stores loading/error state, and invalidates `squadProvider` after success. Django receives `request.FILES['photo']` through multipart parsing/form POST.

Step 3 - Service / Repository Layer

`ApiPlayerRepository.uploadPhoto()` delegates an authenticated multipart `photo` part to `AuthenticatedApiClient.postMultipart()`. The portal/API then calls `academy.storage.validate_photo_upload(upload)`, reads bytes, and invokes `upload_photo(user_id, content, content_type)`. Storage configuration comes from server environment only.

Step 4 - Backend / API

Portal POST is session/CSRF/Coordinator/club protected. `PlayerPhotoUploadView` permits a same-Club Coach or the Super Admin and rejects a cross-club Coach. Django then calls Supabase Storage REST using the server service credential. Flutter never receives that credential and never calls Storage directly.

Step 5 - Database

After storage success, update `academy_playerprofile.photo_path` with the private object path. `PlayerSerializer` later calls `signed_photo_url(photo_path)` to expose a time-limited URL.

Step 6 - Backend Response

Portal redirects/messages; API returns HTTP 200 serialized player. Missing file, invalid signature/MIME, over 25 MB, unconfigured Supabase credentials, or upload failure becomes a visible validation/controller error. Signed-read failure may return no URL so UI can use a fallback.

Step 7 - Model Conversion

API/profile JSON `photoUrl` -> `Player.fromJson()` `String? photoUrl`. The private service key and object bytes never become a Dart model.

Step 8 - State Update

Portal updates DB and redirects. Flutter replaces its local `_player` with the serialized response immediately and invalidates the squad for the next roster read.

Step 9 - UI Update

Portal feedback confirms the upload; Flutter shows a success SnackBar and the returned avatar immediately. Player avatars use `NetworkImage(url)` when nonempty; otherwise a default/initial avatar is rendered.

Call chains: Coach update action -> `_pickAndUploadPhoto()` -> `ImagePicker` -> `PlayerPhotoController.submit()` -> `UploadPlayerPhoto.call()` -> `ApiPlayerRepository.uploadPhoto()` -> `AuthenticatedApiClient.postMultipart()` -> `PlayerPhotoUploadView.post()` -> `validate_photo_upload()` -> Supabase REST -> `PlayerProfile.photo_path` -> `PlayerSerializer` -> `Player.fromJson()` -> local avatar/squad refresh.

Portal upload submit -> `player_photo()` -> `validate_photo_upload()` -> `upload_photo()` -> Supabase Storage REST -> `PlayerProfile.photo_path` save -> redirect -> later `PlayerSerializer` -> `signed_photo_url()` -> `Player.fromJson()` -> `NetworkImage`.

Failure path: picker cancellation -> no request; unsupported client type -> SnackBar; auth/cross-club/validation/storage error -> no `photo_path` update and visible form/API error; signed URL failure -> null photo URL/fallback. The upload path requires configured Supabase server credentials; implementation alone is not proof of a successful live storage upload.

Workflow 22: Search / Filtering

Trigger

Typing into the coach roster search/filter controls updates local filter state and causes the roster widget/provider selection to recompute `RosterFilter.apply`.

Step 1 - UI

Search text and selected tier/position are captured by text/control callbacks in the squad/coach UI.

Step 2 - State / Controller Layer

The roster filtering helper/provider receives the already loaded `List<Player>` plus criteria and returns a filtered list. State is local/Riverpod depending on the control; no mutation controller is needed.

Step 3 - Service / Repository Layer

Initial data came from `GetSquad/ApiPlayerRepository.fetchSquad()`. Changing search does not call the repository again.

Step 4 - Backend / API

Only initial `GET /api/players/` occurs. `SquadListView.get()` returns same-club players for coach. No `?search=` query is generated.

Step 5 - Database

Initial query reads `academy_playerprofile/accounts_user` scoped by club. Filter keystrokes do not query the DB.

Step 6 - Backend Response

The roster list is returned once per provider fetch; local filter returns a Dart list synchronously.

Step 7 - Model Conversion

Initial JSON maps become `Player` objects before `RosterFilter.apply` examines name/tier/position.

Step 8 - State Update

Search/filter state changes cause provider/widget recomputation; original loaded list remains available.

Step 9 - UI Update

The list/grid rebuilds with matching `PlayerCards`. Empty matches render the local empty state.

Call chain: roster screen load -> `squadProvider` -> GET squad -> `List<Player>` -> user types/selects -> filter state change -> `RosterFilter.apply(players,criteria)` -> filtered cards.

Failure path: initial GET failure renders provider error; an empty query/filter is not an error and returns all/applicable players. Limitation: no server pagination/scalable search.

Workflow 23: AI / Recommendation / Chatbot

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No AI/recommendation/chatbot button, lifecycle callback, or job exists.

Step 1 - UI

No AI prompt, recommendation card, or chatbot widget.

Step 2 - State / Controller Layer

No AI provider/controller.

Step 3 - Service / Repository Layer

No inference/model API repository.

Step 4 - Backend / API

No AI endpoint or third-party AI SDK call.

Step 5 - Database

No prompt, embedding, prediction, recommendation, or model-output table.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable. Player overall scores and squad progress are fixed arithmetic, not AI.

Workflow 24: Admin / Coordinator Account Operations

Trigger

Coordinator submits the portal create-account form handled by `portal.views.create_account(request)`; protected admin clients can POST `/api/admin/users/`, `/api/admin/players/`, or guardian-link endpoints.

Step 1 - UI

Portal `create_account.html` collects account type and role-specific fields. The coordinator is already session-authenticated; server derives the club. Django admin/admin API provides global operations.

Step 2 - State / Controller Layer

Django form or DRF serializer validates. There is no Flutter controller. Portal decorators and `IsAdmin` permission protect their respective entry points.

Step 3 - Service / Repository Layer

Coordinator path calls `portal.services.create_club_account(account_type, coordinator=request.user, data=cleaned_data)`. The service derives `coordinator.club`. Player creation delegates to the single `accounts.services.provision_player` aggregate service; other roles use the appropriate app/web provisioning service.

Step 4 - Backend / API

Portal POST uses Django session + CSRF + role. Mobile-user provisioning calls Firebase Admin server-side to create/link an identity; portal-only staff/coordinator uses Django password. Admin endpoints use Bearer auth plus `IsAdmin`.

Step 5 - Database

Insert/update `accounts_user` (email/names/Firebase UID/role/club), `academy_playerprofile` for players, and `accounts_guardianlink` for families. Club is the Coordinator's trusted server value or the dedicated Admin-player flow's guardian-derived same Club. Firebase temporary passwords are not stored in plaintext in these tables.

Step 6 - Backend Response

Portal success message/temporary credential handling; API serializer JSON. Duplicate email/UID, invalid role/link, Firebase failure, or DB failure produces validation/provisioning error. A newly created Firebase identity is deleted on relational failure when compensation applies.

Step 7 - Model Conversion

Portal uses Django model/form objects. Admin API uses DRF user/player/link serializers. No normal Flutter admin model flow exists.

Step 8 - State Update

Database/Firebase identities become available after successful commit; portal redirects or admin client refreshes.

Step 9 - UI Update

Coordinator roster/account lists show the new user. The user can then sign into their intended channel if active and correctly club/profile provisioned.

Call chain: create-account form -> `create_account()` -> role-specific form -> `create_club_account(coordinator=request.user)` -> server-derived Club -> `provision_player()` or role-appropriate service -> Firebase Admin where applicable -> transaction -> portal success.

Invariant: generic user creation and manual generic Admin creation exclude `PLAYER`; non-Admin member accounts require an active Club. Every Player creation path uses `provision_player`, which requires an active Club and creates exactly one `PlayerProfile` plus an optional same-Club GuardianLink atomically. `seed_users` repairs/assigns the active demo Club to every non-Admin seed and ensures its Player profile; `seed_academy` likewise repairs seeded Player/Guardian/Coach roles, Club membership, and profiles before related data is created.

Workflow 25: Guardian PIN Unlock and Child Selection

Trigger

Guardian dashboard watches `linkedPlayersProvider`; choosing a child sets `selectedChildIdProvider`. `PlayerPrivacyGate` PIN submit triggers the verification controller/provider.

Step 1 - UI

`GuardianDashboardScreen` first displays redacted `PlayerSelectorSerializer` data. The gate collects a 4-6 digit PIN. The selected `playerId` and PIN are passed to the privacy controller.

Step 2 - State / Controller Layer

Privacy providers query PIN status, enter loading for verify, call `verifyPlayerPrivacyPinProvider`, then store the returned unlock token in `PlayerUnlockTokenStore` and mark the player ID unlocked. Token store is memory-only.

Step 3 - Service / Repository Layer

`VerifyPlayerPrivacyPin.call()` -> `ApiPlayerPrivacyPinRepository.verifyPin(playerId,pin)` -> authenticated POST. After success, `selectedChildDetailsProvider` calls `GetPlayerDetails` and API repository attaches `X-Player-Unlock`.

Step 4 - Backend / API

`POST /api/players/{id}/pin/verify/` -> `PlayerPrivacyPinVerifyView.post()` checks player self or linked guardian, then `verify_pin()`. Protected detail calls `GET /api/players/{id}/profile/`, attendance/injury/eligibility endpoints and `require_player_unlock()`.

Step 5 - Database

Read/update `academy_playerprivacypin` (`pin_hash`, `failed_attempts`, `locked_until`) inside transaction/row lock; read `accounts_guardianlink`; read requested child domain tables only after authorization. Signed unlock itself is stateless and not stored in DB.

Step 6 - Backend Response

Success `{verified:true, unlockToken:<signed>}`. Invalid PIN -> 400; locked -> 423 with `lockedUntil`; missing PIN -> validation; wrong link -> 403. Detail success returns serializers; expired/mismatched token is denied.

Step 7 - Model Conversion

PIN response becomes privacy result/token value; child detail JSON becomes `Player.fromJson()`, histories become their respective entity models.

Step 8 - State Update

Token saved under player ID; unlocked-set provider changes; dependent `selectedChildDetailsProvider` refetches. Logout clears tokens; expiration/denial causes a new gate cycle.

Step 9 - UI Update

Gate is replaced by `_GuardianUnlockedContent`; child profile/stat/history cards render. Errors/lock time are shown by the PIN UI.

Call chain: select child -> privacy gate -> verify action -> privacy controller -> `VerifyPlayerPrivacyPin` -> API repository -> `PlayerPrivacyPinVerifyView.post()` -> `verify_pin()` -> `issue_player_unlock()` -> memory store -> detail GET with header -> `require_player_unlock()` + GuardianLink -> child UI.

Variable Trace: Player ID and unlock

Selector `Player.id` -> `selectedChildIdProvider` -> PIN endpoint URL -> signed payload `{user_id,player_id}` -> token store keyed by player ID -> `X-Player-Unlock` -> backend compares request user/URL player and active link.

Failure path: wrong PIN increments failures; fifth triggers 15-minute lock; expired token/detail 403 causes error/reunlock; changing URL player fails binding; removing GuardianLink makes an otherwise signed token insufficient.

Workflow 26: Injury Reporting, Confirmation, Recovery Update, and Archive

Trigger

A Player, linked Guardian, same-Club Coach, or Coordinator starts a report from the injury UI. The Coordinator reviews Pending reports in `CoordinatorInjuriesScreen`. After confirmation, a Player, Guardian, or Coach can request a recovery-status change, and the Coordinator approves/rejects it or archives a Confirmed Recovered report.

Step 1 - UI

`InjuryHistoryScreen` and its form collect the affected player, description, body area, occurrence date, and notes. Reporter-facing cards display Pending/Confirmed/Rejected state. `CoordinatorInjuriesScreen` separates review work from history and exposes Confirm, Reject, Approve update, Reject update, edit, and Archive actions. Rejection actions require a reason.

Step 2 - State / Controller Layer

`InjuryFormController` owns the async mutation state. `.submit()`/`.remove()` handle Pending report edits or withdrawal; `.reviewReport()`, `.requestStatus()`, `.reviewStatus()`, and `.archive()` call the corresponding repository operations. Every success invalidates both `injuriesProvider(playerId)` and `clubInjuriesProvider`.

Step 3 - Service / Repository Layer

`ApiInjuryRepository` uses `AuthenticatedApiClient` for list/create/detail plus workflow endpoints: `POST /api/injuries/{id}/review/`, `POST /api/injuries/{id}/status-updates/`, `POST /api/injuries/{id}/status-updates/{updateId}/review/`, and `POST /api/injuries/{id}/archive/`. Linked-Guardian operations include the player-specific `X-Player-Unlock` header when required.

Step 4 - Backend / API

`InjuryWorkflowListCreateView.post()` derives the Player through `_injury_player_for_report()`: Player self, linked unlocked Guardian, or same-Club Coach/Coordinator. Every new report is forced to Active and Pending. `InjuryReviewView` permits only the same-Club Coordinator to Confirm or Reject a locked Pending row. A Confirmed report accepts one Pending recovery update from a Player, Guardian, or Coach; `InjuryStatusUpdateReviewView` lets only the Coordinator atomically approve/reject it. `InjuryArchiveView` requires same-Club Coordinator plus Confirmed and Recovered state.

Step 5 - Database

The workflow writes `academy_injuryrecord`, `academy_injurystatusupdaterequest`, and `academy_auditlog`. Injury rows store reporter/reviewer, review status, rejection reason, injury/recovery state, dates, and archive time. Update requests preserve the proposer, requested state/date, reviewer, decision, and reason. Transactions and row locks prevent two reviews from independently applying stale Pending state.

Step 6 - Backend Response

Create returns 201; updates/reviews/archive return 200 serialized `InjuryRecord`; withdrawal returns 204. Invalid transition/date/reason returns 400, unauthorized role/Club/link/unlock returns 403, missing rows return 404, and repository/network failures become controller errors.

Step 7 - Model Conversion

JSON converts through `InjuryRecord.fromJson()` and nested `InjuryStatusUpdate.fromJson()`; request drafts use their `toJson()` methods. Enum conversion keeps server values such as `PENDING`, `CONFIRMED`, `REJECTED`, `ARCHIVED`, `ACTIVE`, `RECOVERING`, and `RECOVERED` typed in Dart.

Step 8 - State Update

The controller resolves to `AsyncData` and invalidates player and Club lists. Riverpod refetches the authoritative workflow state rather than locally pretending that a review succeeded.

Step 9 - UI Update

Reporter history and Coordinator review queues rebuild with the server decision. Rejected reports remain visible to their reporter with the reason; care-team visibility excludes unrelated rejected reports. Archived records are hidden from normal views unless the authorized Coordinator/Admin requests archived data.

Call chain: report form -> `InjuryFormController.submit()` -> `SaveInjury` -> `ApiInjuryRepository.saveInjury()` -> `POST /api/injuries/` -> `_injury_player_for_report()` -> serializer -> Pending `InjuryRecord` + audit -> `InjuryRecord.fromJson()` -> invalidate. Review continues Coordinator action -> `.reviewReport()` -> review endpoint -> `select_for_update()` -> transition + reviewer/audit -> refreshed lists.

Failure path: a user cannot choose an unrelated Player because Django resolves link/Club scope; a reporter cannot self-confirm; a second Pending recovery update is rejected; rejection without a reason fails; non-Recovered archive fails; any transaction error leaves the prior authoritative state intact.

Workflow 27: Dispute Raise and Response

Trigger

Coach presses submit in `FlagDisputeScreen` near line 123, invoking `_submit()`. Response action in the dispute list/detail calls `DisputeFormController.respond()`.

Step 1 - UI

Flag form holds category, summary, optional detail, and optional subject `Player.id`. `_submit()` validates required summary and calls the controller.

Step 2 - State / Controller Layer

`DisputeFormController.raise(...)/respond(...)` sets `AsyncLoading`, invokes `raiseDisputeProvider/respondToDisputeProvider`, invalidates `disputesProvider`, and returns `Dispute?` or `AsyncError`.

Step 3 - Service / Repository Layer

`RaiseDispute/RespondToDispute` -> `ApiDisputeRepository.raiseDispute()/respondToDispute()`. JSON includes category/summary/detail/subject player or response body/optional status transition.

Step 4 - Backend / API

Create: `POST /api/disputes/` -> `DisputeListCreateView.post()` requires coach and checks subject player club. Response: `POST /api/disputes/{id}/responses/` -> `DisputeResponseCreateView.post()` applies authorized dispute scope and transactionally appends response/updates status.

Step 5 - Database

Insert `academy_dispute` (raiser/subject/category/status/summary/detail/times) or `academy_disputeresponse` (dispute/author/body/status_change_to/time); response FK cascades with dispute, actor links can become null.

Step 6 - Backend Response

HTTP 201 serialized `Dispute`, including thread responses. Invalid role/club/status/body -> 400/403; missing thread -> 404; network/non-201 -> `DisputeRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `Dispute.fromJson()` and nested response conversion.

Step 8 - State Update

Controller invalidates `disputesProvider`, causing `GetDisputes` to refetch the role-scoped list.

Step 9 - UI Update

Create screen closes/succeeds and dispute list/thread rebuilds; appended response/status appears.

Call chain: flag submit -> `_submit()` -> `DisputeFormController.raise()` -> `RaiseDispute` -> `ApiDisputeRepository.raiseDispute()` -> POST -> `DisputeListCreateView.post()` -> `Dispute` insert -> `Dispute.fromJson()` -> invalidate/list. Response follows controller `.respond()` -> response API -> append row/status -> updated dispute.

Failure path: empty/invalid form -> no request; cross-club subject/non-coach -> 403; invalid transition/body -> 400; controller error retains input. This is an internal dispute, **not a scouting report**.

Workflow 28: Player Session Confirmation

Trigger

In `session_confirmation_button.dart`, a player presses a confirmation option; the local `respond(status)` function calls `SessionConfirmationController.submit(session.id, playerId, status)`.

Step 1 - UI

The widget watches `sessionConfirmationsProvider(playerId)` and controller submitting set. Buttons disable while the session ID is submitting.

Step 2 - State / Controller Layer

`SessionConfirmationController.submit()` adds session ID to its `Set<String>`, calls `confirmSessionProvider(sessionId,playerId,status)`, invalidates confirmations on success, catches errors and returns false, then removes the ID. The controller prevents duplicate concurrent taps.

Step 3 - Service / Repository Layer

`ConfirmSession.call()` -> `ApiSessionConfirmationRepository.confirmSession()`. Although `playerId` crosses client layers, the API payload deliberately contains only `{sessionId,status}`.

Step 4 - Backend / API

`POST /api/session-confirmations/`; `SessionConfirmationView.post()` requires role `PLAYER`, derives player from `request.user`, validates session same club and today, and calls `update_or_create(player=request.user,session=...)`.

Step 5 - Database

Insert/update `academy_sessionconfirmation`, unique `(player_id,session_id)`, with status/responded time. Both FKs cascade on deletion.

Step 6 - Backend Response

HTTP 200/201 serialized confirmation; invalid date/session/status/role/club returns error. Repository converts non-success into an exception.

Step 7 - Model Conversion

JSON -> `SessionConfirmation.fromJson()`.

Step 8 - State Update

Provider invalidation refetches the confirmation list; the submitting set removes the session ID. The widget checks the returned boolean and surfaces a failed submission instead of silently swallowing it.

Step 9 - UI Update

On successful refetch, selected response/confirmation UI changes. On submit failure, buttons re-enable and a SnackBar tells the player that the response could not be updated. An initial confirmation-list load error renders a **Retry RSVP** button.

Call chain: response button -> local `respond()` -> `SessionConfirmationController.submit()` -> `ConfirmSession.call()` -> API repository -> POST -> `SessionConfirmationView.post()` -> `update_or_create()` -> `SessionConfirmation.fromJson()` -> invalidate/refetch -> button state.

Failure path: repository exception -> controller catches/returns false -> visible retry-oriented SnackBar; initial load error -> **Retry RSVP** button. No false optimistic confirmation is shown.

Workflow 29: Password Reset and Change

Trigger

Login "forgot password" calls `_handleForgotPassword()`; change-password screen save calls `ChangePasswordController.submit(...)`.

Step 1 - UI

Forgot flow reads trimmed email and uses login controller reset state. Change flow reads current/new/confirm password fields and provides visibility/loading/error widgets.

Step 2 - State / Controller Layer

`LoginController.sendResetEmail(email)`/`PasswordResetController.send(email)` call `sendPasswordResetProvider`. `ChangePasswordController.submit()` runs `_validate`: current required, new/confirm match, minimum eight, new differs; then calls `changePasswordProvider`.

Step 3 - Service / Repository Layer

`SendPasswordReset` -> `FirebaseAuthRepository.sendPasswordResetEmail(email)`. `ChangePassword` -> repository `changePassword`: get current user/email, build `EmailAuthProvider.credential`, `reauthenticateWithCredential`, then `user.updatePassword(next)`.

Step 4 - Backend / API

These are Firebase Auth client operations, not Django academy endpoints. Django receives the new token/session on later API calls and continues mapping the same UID.

Step 5 - Database

Firebase identity password changes outside Django ORM. Django mobile user password is not used/stored for this login path.

Step 6 - Backend Response

Firebase completes or throws typed exceptions (invalid email/current password, expired session, throttling/network). Repository maps to `AuthException` friendly messages.

Step 7 - Model Conversion

No domain JSON model; success is `void`/boolean controller outcome.

Step 8 - State Update

Controllers set sending/loading/error/success state and prevent duplicate submission.

Step 9 - UI Update

Forgot flow shows feedback that reset was requested; change flow shows success/navigates per screen or error text. Fields/buttons react to controller state.

Call chains: forgot action -> `_handleForgotPassword()` -> `LoginController.sendResetEmail()` -> `SendPasswordReset.call()` -> Firebase repository -> `FirebaseAuth.sendPasswordResetEmail()` -> feedback.

Change button -> `ChangePasswordController.submit()` -> `_validate()` -> `ChangePassword.call()` -> repository reauthenticate -> `updatePassword()` -> success/error UI.

Failure path: local change validation -> no Firebase call; wrong current password -> mapped error; no current user/email -> expired-session error; network/Firebase error -> controller error. There is no Django DB rollback because no Django write occurs.

Workflow 30: Coordinator Creates or Updates a Completed Match

Trigger

The Coordinator opens `CoordinatorMatchesScreen`, chooses an uncompleted tournament fixture or an ad-hoc result, then records both scores. A completed match can later be corrected by the same role.

Step 1 - UI

`EditFootballMatchScreen` creates a `FootballMatchDraft`. For a tournament fixture, opponent, competition, date, and venue come from that fixture and the Coordinator supplies the scores; for an ad-hoc result, the form validates all match fields. Only completed, non-future matches are accepted.

Step 2 - State / Controller Layer

`MatchManagementController.create()` or `.saveMatchChanges()` enters `AsyncLoading`, invokes the appropriate use case, stores `AsyncData` on success, and invalidates `footballMatchesProvider` plus the affected match roster when editing.

Step 3 - Service / Repository Layer

`CreateFootballMatch` or `UpdateFootballMatch` calls `ApiMatchRepository.createMatch()` or `.updateMatch()`. The repository serializes the draft and uses `AuthenticatedApiClient`.

Step 4 - Backend / API

Create uses `POST /api/matches/`; update uses `PUT /api/matches/{matchId}/`. `FootballMatchListCreateView.post()` permits only `COORDINATOR`, rejects an account without a Club, and stamps `club=request.user.club` and `created_by=request.user`. If `fixtureId` is supplied, Django locks and resolves only a same-Club, not-cancelled, not-yet-completed fixture, derives its metadata, creates the match, and links it as `completed_match` atomically. `FootballMatchDetailView.put()` uses `_role_match(..., COORDINATOR)`; a fixture-linked match exposes only score correction so scheduled facts are not silently rewritten.

Step 5 - Database

Create inserts `academy_footballmatch` and `academy_auditlog`; fixture-backed creation also updates `academy_tournamentfixture.completed_match_id` and status. `FootballMatch.save()` calls `full_clean()`. The Club FK uses `PROTECT`, creator uses `SET_NULL`, the fixture-to-match relation is one-to-one, and the `(club, -played_on)` index supports scoped history ordering.

Step 6 - Backend Response

Create returns HTTP 201 and update HTTP 200 with `FootballMatchSerializer` JSON. A future date, blank opponent, invalid venue, or score outside 0-99 returns 400. A wrong role or cross-Club mutation returns 403/404.

Step 7 - Model Conversion

JSON becomes `FootballMatch.fromJson()`. The Dart entity derives display helpers such as score label and win/draw/loss outcome without changing stored data.

Step 8 - State Update

The controller resolves and invalidates affected providers. The list provider refetches the Club-scoped match collection.

Step 9 - UI Update

The form closes on success; `CoordinatorMatchesScreen` shows the new or corrected result. Selecting a match opens `MatchRosterScreen` for objective per-player statistics. Coaches can see Club match records but cannot create/correct the result.

Call chain: Coordinator Matches -> fixture/result action -> `EditFootballMatchScreen._submit()` -> `MatchManagementController.create()` -> `CreateFootballMatch.call()` -> `ApiMatchRepository.createMatch()` -> `POST /api/matches/` -> role/Club/fixture lock -> `FootballMatchSerializer.save()` with server Club/creator -> optional fixture completion -> audit -> `FootballMatch.fromJson()` -> provider invalidation -> result card.

Failure path: malformed result returns local/server validation; a Coach/non-Coordinator or cross-Club request fails before mutation; a cancelled/already-completed fixture fails; the atomic block prevents a match row from being saved without the corresponding fixture link.

Workflow 31: Coordinator Records or Corrects Objective Player Statistics

Trigger

The Coordinator opens a completed match in `MatchRosterScreen`, selects a same-Club Player, enters objective statistics in `EditMatchPerformanceScreen`, and presses Save. The same screen can delete an existing statistics row after confirmation.

Step 1 - UI

The form builds `MatchPerformanceDraft` from position, starter flag, minutes, goals, assists, shots, passing, defending, cards, and goalkeeper fields. Coach rating and subjective notes are deliberately absent because they belong to the Coach-only rating workflow. Goalkeeper-only fields are shown/meaningful for position `GK`. An Active/Recovering injury warning requires explicit acknowledgement before the Coordinator can proceed.

Step 2 - State / Controller Layer

`MatchManagementController.savePerformance(matchId, playerId, draft)` enters loading and calls `saveMatchPerformanceProvider`. Success invalidates both `matchPerformancesProvider(matchId)` and `playerMatchStatisticsProvider(playerId)`. Delete follows the same invalidation rule.

Step 3 - Service / Repository Layer

`SaveMatchPerformance.call()` delegates to `ApiMatchRepository.savePerformance()`, which sends the draft to `PUT /api/matches/{matchId}/performances/{playerId}/`. `DeleteMatchPerformance` uses DELETE on the same URL.

Step 4 - Backend / API

`MatchPerformanceDetailView.put()` resolves the match through `_role_match(..., COORDINATOR)` and a same-Club `PLAYER`. It detects Confirmed Active/Recovering injuries and returns HTTP 409 with `ACTIVE_INJURY_WARNING` until the request acknowledges the override. Inside `transaction.atomic()`, it locks the parent match and existing row, validates the complete objective payload, checks team goal totals and goalkeeper concessions against the match score, then saves server-owned `match`, `player`, and `recorded_by` fields. An acknowledged injury override creates a separate audit event.

Step 5 - Database

The mutation inserts or updates `academy_playermatchperformance` and records `academy_auditlog`. The unique `(match, player)` constraint prevents duplicates. Database checks enforce `shots_on_target <= shots`, `goals <= shots_on_target`, and `passes_completed <= passes_attempted`; model validation also protects player role/Club, position, clean sheet, card, and goalkeeper rules. Existing Coach rating/notes remain role-separated fields.

Step 6 - Backend Response

The first save returns 201; correction returns 200; deletion returns 204. The read serializer returns nested match metadata and normalized camelCase fields. Impossible statistics return 400; an unacknowledged injury warning returns 409; wrong role or Club scope returns 403/404. Deleting a rated row requires `confirmRated=true` because the rating is removed with the row.

Step 7 - Model Conversion

Response JSON becomes `MatchPerformance.fromJson()` with nested `FootballMatch`. Draft and response remain separate types so server IDs and ownership fields are not client-writable.

Step 8 - State Update

The controller resolves, then invalidates the match roster and player statistics providers. Riverpod refetches both authoritative views.

Step 9 - UI Update

The editor closes after a successful save; the match roster shows the recorded player row. The Player's summary/history and the Coach's trend screen reflect the new record after refresh.

Call chain: match card -> `MatchRosterScreen` -> player row -> `EditMatchPerformanceScreen._save()` -> `MatchManagementController.savePerformance()` -> `SaveMatchPerformance.call()` -> `ApiMatchRepository.savePerformance()` -> authenticated PUT -> `MatchPerformanceDetailView.put()` -> scoped match/player lookup -> atomic locks -> serializer/business checks -> `PlayerMatchPerformance.save()` -> audit -> `MatchPerformance.fromJson()` -> invalidate roster/player statistics.

Variable trace: `FootballMatch.id` + `Player.id` -> Riverpod/controller parameters -> URL `matchId/playerId` -> role-specific `_role_match()` and same-Club Player query -> database `(match_id, player_id)` unique key -> serialized IDs -> provider-family cache keys.

Failure path: invalid numeric relationships or goalkeeper rules return 400; an injury requires visible acknowledgement and an audit trail; concurrent save is serialized by row locks and the unique constraint; a Player/Coach/cross-Club Coordinator cannot reach the save; transaction failure leaves no partial row.

Workflow 32: Player, Guardian, and Coach View Match Statistics and Trends

Trigger

A Player opens Match Statistics in `ProgressScreen`; a linked Guardian opens the protected child view after PIN unlock; a same-Club Coach opens a Player trend view from progress/profile.

Step 1 - UI

`PlayerMatchStatisticsView` watches `playerMatchStatisticsProvider(playerId)`. It renders a season summary, Last 5/Last 10/All rating chart, expandable match history, pull-to-refresh, retry state, or a clear empty state.

Step 2 - State / Controller Layer

The family `FutureProvider` calls `getPlayerMatchStatisticsProvider(playerId)`. Its `AsyncValue` controls loading, data, and error widgets. The history-range selector is local UI state and filters already authorized returned rows.

Step 3 - Service / Repository Layer

`GetPlayerMatchStatistics.call()` delegates to `ApiMatchRepository.fetchPlayerStatistics()`, which performs authenticated `GET /api/players/{playerId}/match-statistics/` and converts the returned map.

Step 4 - Backend / API

`PlayerMatchStatisticsView.get()` calls `_may_read_match_statistics()`: Admin may read all, Coach only same-Club Players, Player only self, and Guardian only a linked child. The endpoint then applies `_require_unlock_when_pin_exists()`, so relationship alone does not bypass the household PIN. Optional `from`, `to`, and `limit` query parameters are validated; limit must be 1-100.

Step 5 - Database

The endpoint reads `academy_playermatchperformance` joined to match, Club, and player, filtered by `player_id` and optional match date range, ordered newest first. `build_performance_summary()` calculates totals, pass-completion percentage, clean sheets, and one-decimal average rating from at most 100 authorized rows. It performs no write.

Step 6 - Backend Response

HTTP 200 returns `{playerId, playerName, summary, performances}`. Invalid date/limit gives 400; disallowed identity gives 403; missing player gives 404.

Step 7 - Model Conversion

`PlayerMatchStatistics.fromJson()` creates `MatchPerformanceSummary` and a typed `List<MatchPerformance>`, each containing a nested `FootballMatch`.

Step 8 - State Update

The provider resolves to `AsyncData`. Coordinator statistics saves/deletes and Coach rating saves/deletes invalidate this Player-specific provider; pull-to-refresh explicitly refetches it.

Step 9 - UI Update

Summary tiles show matches, goals, assists, rating, pass rate, and saves/tackles. `PerformanceTrendChart` plots chronological coach ratings; history cards expose match score, date, minutes, attack/passing/defending data, goalkeeper saves when applicable, and notes.

Call chain: Progress/player-profile trend action -> `PlayerMatchStatisticsView` -> `playerMatchStatisticsProvider(playerId)` -> `GetPlayerMatchStatistics.call()` -> `ApiMatchRepository.fetchPlayerStatistics()` -> authenticated GET -> `_may_read_match_statistics()` -> filtered `PlayerMatchPerformance` query -> `build_performance_summary()` + serializer -> `PlayerMatchStatistics.fromJson()` -> summary/chart/history.

Authorization answer: a Player cannot substitute another Player ID because Django compares it with `request.user.id`; a Coach must share the target's Club; a Guardian must have both an active link and, when configured, a player-bound unlock token. Flutter navigation is convenience, not enforcement.

Failure path: provider error renders retry UI; empty history renders an explicit message; malformed range/limit stays a 400 rather than silently broadening access; unauthorized player/guardian/cross-Club Coach receives no statistics.

Workflow 33: Coordinator Publishes a Tournament Schedule and Fixtures

Trigger

The Coordinator signs into the Django portal, opens Tournaments, uploads an official PDF/JPEG/PNG schedule, and adds structured fixtures. Authenticated mobile Club members open `TournamentScheduleScreen` to view the published programme.

Step 1 - UI

The portal uses `TournamentScheduleForm`, `TournamentDocumentForm`, and `TournamentFixtureForm`. Mobile `TournamentScheduleScreen` watches `tournamentSchedulesProvider` and renders schedule cards, fixture status, kickoff, location, venue, and a signed document link.

Step 2 - State / Controller Layer

Portal views use Django forms and messages rather than Riverpod. Mobile uses a `FutureProvider` that calls the tournament schedule use case/repository and displays loading, empty, error/retry, or data states.

Step 3 - Service / Repository Layer

The portal calls `validate_tournament_document()`, `upload_tournament_document()`, signed-URL, replacement, and deletion helpers in `academy/storage.py`. Flutter uses `ApiTournamentScheduleRepository.fetchSchedules()` and `AuthenticatedApiClient.get('/api/tournament-schedules/')`.

Step 4 - Backend / API

Portal routes are session-authenticated and decorated with `portal_role_required(COORDINATOR)`. `_coordinator_schedule()` and fixture queries always filter `club_id=request.user.club_id`. The mobile DRF endpoint is read-only for Coordinator, Coach, Player, Guardian, and Admin; non-admin results are filtered to the authenticated Club and only `is_published=True` schedules are returned.

Step 5 - Database

Create inserts `academy_tournamentschedule`; fixture forms insert/update/delete `academy_tournamentfixture`; every action records `academy_auditlog`. A fixture belongs to one schedule and may point one-to-one to a completed `FootballMatch`. A schedule with completed matches cannot be deleted, and a completed fixture cannot be edited or deleted.

Step 6 - Backend Response

The portal redirects with success/error messages. Mobile returns HTTP 200 with nested `TournamentScheduleSerializer` and `TournamentFixtureSerializer` JSON, including a short-lived `documentUrl` rather than a storage credential or raw private path.

Step 7 - Model Conversion

`TournamentSchedule.fromJson()` creates the schedule plus typed `TournamentFixture` entries. Date/time, venue, fixture status, and optional completed match ID are normalized in Dart.

Step 8 - State Update

Portal redirects reload the authoritative database state. On mobile, the FutureProvider resolves; pull-to-refresh invalidates/refetches schedules.

Step 9 - UI Update

The Coordinator portal shows editable official programme data, while mobile roles receive a read-only Club schedule. A Coordinator can start completed-result entry from an eligible fixture, joining Workflow 30 without retyping scheduled metadata.

Call chain: portal upload -> `tournament_schedules()` -> form/signature validation -> `TournamentSchedule.objects.create()` -> server-side storage upload -> save opaque path + audit -> mobile `TournamentScheduleScreen` -> provider -> `ApiTournamentScheduleRepository` -> GET -> Club-filtered serializer -> typed schedule/fixtures -> cards.

Failure path: wrong role/Club returns portal denial or DRF 403; invalid type/signature/size is rejected before storage; storage failure shows a form error; completed fixture/schedule guards prevent history from being orphaned; Flutter receives no Supabase service key.

Workflow 34: Coach Adds or Clears a Subjective Match Rating

Trigger

A same-Club Coach opens a completed match roster, selects a Player who already has Coordinator-entered objective statistics, enters a 0.0-10.0 rating and optional notes in `EditMatchRatingScreen`, then saves or clears it.

Step 1 - UI

The Coach rating screen contains only `coachRating` and `notes`; it cannot alter goals, minutes, passes, cards, or goalkeeper statistics. Coordinator views receive a serializer representation with rating and notes redacted.

Step 2 - State / Controller Layer

`MatchManagementController.saveRating()/.deleteRating()` invokes the rating use cases and invalidates `matchPerformancesProvider(matchId)`, `matchRosterProvider(matchId)`, and `playerMatchStatisticsProvider(playerId)`.

Step 3 - Service / Repository Layer

`SaveMatchRating/DeleteMatchRating` call `ApiMatchRepository.saveRating()` or `.deleteRating()` at `/api/matches/{matchId}/performances/{playerId}/rating/`.

Step 4 - Backend / API

`MatchPerformanceRatingView` calls `_role_match(..., COACH)`, then locks and resolves an existing performance. `CoachMatchRatingSerializer` accepts only rating and notes. Save stamps `rated_by=request.user` and `rated_at=timezone.now()`; delete clears those four subjective fields without deleting objective statistics.

Step 5 - Database

The endpoint updates the existing `academy_playermatchperformance` row and inserts an audit record. It does not create a performance row, change its Player/match/recorder, or edit Coordinator-owned statistics.

Step 6 - Backend Response

Save returns HTTP 200 serialized performance; clear returns 204. Missing statistics return 404, out-of-range rating returns 400, and non-Coach/cross-Club access returns 403/404.

Step 7 - Model Conversion

Save JSON converts through `MatchPerformance.fromJson()`. A nullable rating represents `AWAITING_RATING`; a numeric rating represents `RATED`.

Step 8 - State Update

The controller resolves and invalidates roster/history providers. The Player's derived average rating is recalculated by the backend on the next statistics GET.

Step 9 - UI Update

Coach roster and Player/Guardian trend views show the refreshed rating and notes. Coordinator screens continue to show objective completion status without receiving subjective fields.

Call chain: Coach roster -> `EditMatchRatingScreen` -> controller `.saveRating()` -> `SaveMatchRating` -> repository PUT -> Coach role/same-Club match -> locked existing performance -> rating serializer -> stamp rater/time + audit -> JSON -> provider invalidation -> trend/history.

Failure path: a Coach cannot rate before objective statistics exist; Coordinator cannot submit a rating through this endpoint; invalid rating/notes fail validation; clearing removes only subjective evaluation, preserving historical statistics.

Workflow 35: Independent-Club Academic Eligibility Suppression

Trigger

A Player or linked Guardian opens a profile/eligibility destination for a Club whose type does not support school academic eligibility.

Step 1 - UI

Cards and profile sections use `academicEligibilityApplicable`; independent Clubs display `Eligibility N/A` or `Not applicable to an Independent club` instead of presenting a misleading academic readiness result.

Step 2 - State / Controller Layer

Existing profile and eligibility providers carry the applicability flag and results. Navigation does not manufacture eligibility history when the feature does not apply.

Step 3 - Service / Repository Layer

The normal player/eligibility API repositories make authenticated reads. No separate client-side database rule is trusted.

Step 4 - Backend / API

`EligibilityHistoryView.get()` first runs identity, Club/link, and PIN authorization, then checks `player.club.allows_academic_eligibility`. For an independent Club it returns `{applicable: false, results: []}`. Portal/admin eligibility controls follow the same Club-type business rule.

Step 5 - Database

The check reads `accounts_club` and the target Player relationship. It performs no eligibility write and does not create a fake history row.

Step 6 - Backend Response

HTTP 200 distinguishes a legitimate not-applicable state from network, authorization, and missing-object errors. This avoids treating absence as either Eligible or Not Eligible.

Step 7 - Model Conversion

Dart converts the applicability boolean separately from the eligibility enum/history list.

Step 8 - State Update

Riverpod stores the typed response, preserving the semantic difference between `not applicable`, `empty history`, and `request failed`.

Step 9 - UI Update

The academic section is hidden or labeled N/A for independent Clubs while school-affiliated Clubs retain normal status/history behavior.

Defense purpose: this prevents the system from imposing a school-specific academic workflow on an independent football Club and demonstrates that Club type affects server-authoritative business rules, not only labels in Flutter.

Failure path: changing Flutter UI or URL parameters cannot enable the feature because the backend rechecks the Player's stored Club type before returning data.

Workflow 36: Coordinator Creates and Publishes Mobile Tournament Brackets

Trigger

A Coordinator opens the mobile Schedule tab, taps **Create tournament** or the edit icon, enters the tournament name/start date, adds U-age brackets with optional schedule times, and publishes the configured tournament.

Step 1 - UI

`TournamentScheduleScreen` exposes management actions only when `canManage` is true. `EditTournamentScreen._saveDetails()`, `_editBracket()`, `_removeBracket()`, and `_publish()` collect the tournament and bracket data, show validation feedback, and disable conflicting actions while a mutation is pending.

Step 2 - State / Controller Layer

`TournamentManagementController` sets `AsyncLoading`, calls `create`, `saveTournament`, `addBracket`, `updateBracket`, `deleteBracket`, or `publish`, records `AsyncData/AsyncError`, and invalidates `tournamentSchedulesProvider` after a successful write.

Step 3 - Service / Repository Layer

`ApiTournamentScheduleRepository` converts dates to the API wire format and calls `AuthenticatedApiClient`: `POST /api/tournament-schedules/`, `PATCH /api/tournament-schedules/{id}/`, `POST /api/tournament-schedules/{id}/brackets/`, `PATCH/DELETE /api/tournament-brackets/{id}/`, and `POST /api/tournament-schedules/{id}/publish/`.

Step 4 - Backend / API

`TournamentScheduleListView`, `TournamentScheduleDetailView`, `TournamentSchedulePublishView`, `TournamentAgeBracketCreateView`, and `TournamentAgeBracketDetailView` require a Coordinator with a Club and resolve every schedule/bracket through that stored Club. Write serializers validate the title, date, unique U-age value, and optional schedule time. Publishing requires at least one bracket.

Step 5 - Database

Django inserts or updates `academy_tournamentschedule` and `academy_tournamentagebracket` inside the authenticated Club and records `academy_auditlog`. The bracket has a database-unique `(schedule_id,max_age)` key. Changing a tournament date or bracket age is rejected if it would make stored squad members overage or incomplete; bracket deletion is blocked after publication or while fixtures/roster entries depend on it.

Step 6 - Backend Response

The API returns the complete nested schedule JSON, including `startsOn`, `isPublished`, `ageBrackets`, optional nested squads, and fixtures. Create/bracket-create returns 201, delete returns 204, and validation or authorization failures remain explicit 400/403/404 responses.

Step 7 - Model Conversion

`TournamentSchedule.fromJson()` creates `TournamentSchedule` and nested `TournamentAgeBracket/TournamentSquad` entities. UTC bracket timestamps become local display values in Flutter while writes send ISO-8601 UTC.

Step 8 - State Update

The controller replaces its mutation state, invalidates `tournamentSchedulesProvider`, and the screen keeps `_current` synchronized with the returned authoritative object rather than assuming the submitted draft was accepted unchanged.

Step 9 - UI Update

The refreshed Schedule tab shows Draft/Published status, bracket chips, bracket times, and roster state. Other Club roles see only published tournaments, while Coordinator and Coach can see drafts required for their management workflow.

Call chain: Coordinator Schedule tab -> `EditTournamentScreen` -> `TournamentManagementController` -> `ApiTournamentScheduleRepository` -> authenticated REST call -> Club-scoped DRF APIView -> serializer/model/transaction -> schedule/bracket/audit rows -> nested JSON -> provider invalidation -> refreshed cards.

Failure path: duplicate U-age, missing title/date, no bracket at publish time, cross-Club ID, invalidating a stored roster, or deleting a bracket with fixtures/entries stops the write; the controller exposes the error and the previous database state remains authoritative.

Workflow 37: Coach Builds and Publishes an Age-Bracket Tournament Squad

Trigger

A Coach opens a tournament's U-age division, searches the candidate list, selects eligible Players, optionally assigns tournament-specific positions, saves a draft, and publishes it after the Coordinator has published the tournament.

Step 1 - UI

`TournamentScheduleScreen` opens `TournamentSquadScreen(canEdit: true)` for Coaches. `_buildCoachEditor()` renders eligibility icons/reasons, search, selection count, and position dropdowns. `_save(publish: false)` saves a draft; `_save(publish: true)` first saves the current selection and then publishes it.

Step 2 - State / Controller Layer

`tournamentRosterCandidatesProvider(bracketId)` loads Coach-only candidate data. `TournamentRosterManagementController.save()/.publish()` owns loading/error state and invalidates both tournament schedules and the bracket candidate provider after success.

Step 3 - Service / Repository Layer

`ApiTournamentRosterRepository.fetchCandidates()`, `saveSquad()`, and `publishSquad()` send authenticated GET/PUT/POST requests to `/api/tournament-brackets/{bracketId}/squad/...`. `TournamentRosterSelection.toJson()` sends only `playerId` and an optional event position.

Step 4 - Backend / API

`TournamentSquadCandidatesView` requires a Coach and returns same-Club active Players with privacy-safe `ELIGIBLE`, `WARNING`, or `BLOCKED` outcomes. `TournamentSquadDetailView.put()` accepts only Coach writes, validates duplicate IDs/positions and same-Club active Player accounts, reruns `roster_eligibility()`, then atomically reconciles added, removed, and changed entries. `TournamentSquadPublishView` requires a published tournament, at least one entry, and no currently blocked member.

Step 5 - Database

`TournamentSquad` is one-to-one with the bracket; `TournamentSquadEntry` is unique per `(squad_id,player_id)`. An atomic transaction and `select_for_update()` serialize concurrent roster changes. Django stores the event position and verified Coach actor, changes status from `DRAFT` to `PUBLISHED`, stamps `published_at`, and records auditable save/publish events.

Step 6 - Backend Response

The nested squad JSON contains bracket/status/publish time and entries. Manager roles receive availability reasons; Player/Guardian output omits those care-sensitive details. A missing manager draft is represented as an empty Draft response, while ordinary roles cannot read an unpublished roster.

Step 7 - Model Conversion

`TournamentSquad.fromJson()`, `TournamentSquadEntry.fromJson()`, and `TournamentRosterCandidate.fromJson()` convert wire status and eligibility strings into Dart enums/typed fields.

Step 8 - State Update

The screen replaces `_squad` with the server response; Riverpod invalidation refetches the tournament card and candidate selections so the status/count cannot drift from the database.

Step 9 - UI Update

The Coach sees Draft or Published confirmation and updated membership. Coordinator/Admin can inspect the roster; Player/Guardian can open only a published tournament/roster and see the selected names and positions.

Call chain: bracket tap -> `TournamentSquadScreen` -> candidates provider -> Coach chooses Players -> controller save/publish -> API roster repository -> DRF roster endpoints -> eligibility policy + atomic reconciliation -> squad/entry/audit rows -> typed JSON -> invalidation -> updated roster card.

Failure path: blocked age/injury/profile candidates cannot be newly selected, cross-Club/inactive/duplicate IDs fail validation, an empty published roster is forbidden, and roster publication waits for the Coordinator's tournament publication. Pending injury is a visible warning, not a silent hard block.

Workflow 38: Tournament Squad Enforcement During Match Statistics Entry

Trigger

A Coordinator opens the roster for a match linked to a tournament age bracket and records objective statistics for a squad member, or deliberately adds an eligible Player who is outside the published squad with a required reason.

Step 1 - UI

`MatchRosterScreen` normally displays published-squad members and existing historical rows. For an age-bracket match, the Coordinator may tap **Add eligible squad exception**; `_addOutOfSquadPlayer()` loads selectable out-of-squad candidates, collects a required reason of at most 500 characters, and passes it to `EditMatchPerformanceScreen`.

Step 2 - State / Controller Layer

`matchRosterProvider(matchId)` requests the normal roster. `outOfSquadMatchCandidatesProvider(matchId)` requests `includeOutOfSquad=true` and retains only rows that require an override, have no existing performance, and remain selectable. `MatchManagementController.savePerformance()` invalidates roster, candidates, match performance, and Player trend providers after success.

Step 3 - Service / Repository Layer

`ApiMatchRepository.fetchMatchRoster()` calls `GET /api/matches/{matchId}/roster/` with the optional query flag. `savePerformance()` sends objective data plus `squadOverrideReason` through `PUT /api/matches/{matchId}/performances/{playerId}/`.

Step 4 - Backend / API

`MatchRosterView.get()` derives the bracket from the match's source fixture. It joins the published squad, eligibility policy, active injury status, and any existing performance. Only a Coordinator may request out-of-squad candidates. `MatchPerformanceDetailView.put()` locks the match/performance, blocks hard-ineligible Players, requires and preserves a reason for an eligible non-squad Player, then runs the existing objective-statistics checks.

Step 5 - Database

Normal writes use the unique `academy_playermatchperformance(match_id,player_id)` row. A squad exception stores `squad_override_reason`, verified `squad_override_by_id`, and `squad_override_at`; a database check requires the reason/timestamp pair to agree. The action also creates a `match.squad_override` audit event.

Step 6 - Backend Response

Roster JSON identifies `inTournamentSquad`, `requiresSquadOverride`, `isSelectable`, availability/reason, tournament position, injury status, and existing performance. The saved performance reports its squad-exception state and returns 200/201; missing reason or blocked eligibility returns structured 400 validation data.

Step 7 - Model Conversion

`MatchRosterPlayer.fromJson()` and `MatchPerformance.fromJson()` preserve membership/override metadata so Flutter can show an **Approved squad exception** chip and keep historical evidence distinguishable from ordinary squad selection.

Step 8 - State Update

Successful save invalidates every affected provider; a failed write leaves the local editor open and the server roster unchanged. Previously recorded historical rows remain visible even if current squad eligibility later changes.

Step 9 - UI Update

The Coordinator sees the saved Player in the match roster with the exception label. Coaches can rate only after objective statistics exist, and authorized Player/Guardian/Coach trend views receive the same historical row without gaining access to the private override reason.

Call chain: match roster -> normal/expanded roster provider -> `MatchRosterView` -> squad + eligibility filter -> exception modal when needed -> `EditMatchPerformanceScreen` -> `MatchManagementController.savePerformance()` -> API PUT -> locked enforcement/audit -> typed response -> provider refresh.

Failure path: Flutter cannot make a blocked Player selectable by changing UI state. Django recomputes age/profile/injury eligibility, checks published membership, requires the Coordinator-only reason, validates statistics, and rolls the transaction back on any failure.

Cross-Cutting Trace: Coordinator Responsive UI Consistency

The newest Coordinator screens share `ResponsiveContent/ConstrainedBox` widths, `AdaptiveFormModal`, `AppStatusChip`, and reusable dashboard loading/error/empty states. Bottom navigation and wider layouts keep the same Tournament, Matches, Injuries, and Account destinations, while form buttons observe Riverpod loading state and `mounted` checks. This is a presentation consistency layer: it improves phone/tablet usability and error clarity, but it never replaces Django's role, Club, eligibility, or transaction checks.

Cross-Cutting Trace: Shared Authenticated API Client and Owner-Scoped Cache

Live academy REST adapters delegate authentication, the 15-second timeout, accepted-status checks, safe server-error extraction, and multipart handling to `data/network/authenticated_api_client.dart` instead of duplicating those policies in each repository. `FirebaseAuthRepository` keeps the `/api/auth/me/` login/restore boundary separate because it creates or restores the identity that the shared client later consumes.

`ApiConfig.baseUrl` selects `http://10.0.2.2:8000` for the Android emulator and localhost for desktop/web development unless `API_BASE_URL` is supplied at build time. Release builds require HTTPS, reject localhost/emulator hosts, and can enforce `API_ALLOWED_HOST`. The client builds normalized URIs, reserves the `Authorization` header so repositories cannot override identity, and refreshes the Firebase token once on an eligible 401 before reporting authentication failure.

Successful GET chain: Riverpod/use case -> domain repository adapter -> `AuthenticatedApiClient.get(path)` -> current `ApiIdentity(uid,getIdToken)` -> Bearer request -> accepted HTTP response -> JSON validation -> `ApiGetCache.put(ownerUid,requestKey,response)` -> repository model conversion -> provider/UI.

Network-only fallback chain: token-network failure, timeout, socket, TLS handshake, or `http.ClientException` before a response -> `_networkFallback()` -> `ApiGetCache.get(currentUid,requestKey)` -> cached response marked with `x-footpath-cache: true`, if a fresh owner-scoped entry exists. An HTTP 401/403/5xx, missing identity, or malformed successful JSON throws its typed auth/HTTP/decode exception and never falls back. Requests carrying `X-Player-Unlock` are neither cached nor served from cache, and writes/multipart uploads are never cached or replayed by this client.

Attendance mutation replay remains a separate, intentionally narrow path: `OfflineFirstAttendanceRepository` queues a complete attendance batch only on `AttendanceNetworkException`; HTTP rejection remains visible. Its session-roll-call fallback likewise reads only the current owner's latest queued batch and only after a network exception. Other successful authenticated GETs may use the owner-scoped response cache, but this does not make every mutation offline-capable.

Repository Production Hardening Trace

The repository contains a production image and service topology (`backend/Dockerfile`, `compose.production.yml`) using Gunicorn, WhiteNoise, PostgreSQL, and Redis; production settings add TLS/HSTS/cookie hardening, structured logs, and optional Sentry without request PII. `/api/health/` is a cheap liveness probe, while `/api/ready/` checks database and cache dependencies and returns 503 when unavailable. CI runs Django tests/checks, Flutter analysis/tests, deploy checks, guarded PostgreSQL backup/restore-script checks, and the backend production-image build.

Live-data safeguards are explicit. Flutter's normal debug/profile/release composition uses Firebase plus `Api*Repository`; mocks require `USE_MOCK=true` for isolated non-release work, while Flutter tests detect their own runtime. Django uses PostgreSQL when `DB_HOST` is configured, SQLite only for DEBUG/testing, and raises `ImproperlyConfigured` in non-test production without `DB_HOST`. These guards prevent an apparently successful production app from silently writing to an unintended local database.

Operational flow: container start -> migrations/static collection -> Gunicorn -> liveness/readiness probes -> platform log/optional Sentry collection. Backup flow: operator/scheduler -> `scripts/postgres_backup.py` -> `pg_dump` custom-format file + retention. Restore-drill flow: explicit isolated database name -> `scripts/postgres_restore.py` confirmation guard -> `pg_restore`.

Evidence boundary: these are repeatable repository artifacts and documented procedures, not evidence of a verified live deployment, exercised monitor/alert, scheduled backup, or successful restore drill. `docs/PRODUCTION-OPERATIONS.md` deliberately leaves those evidence entries unsupplied.

Database Query Traces

QRY-1: Resolve Authenticated User

- **Source File:** `backend/accounts/authentication.py`
- **Function:** `FirebaseAuthentication.authenticate()`
- **Query:** local `User` lookup after Firebase Admin verifies decoded UID.
- **Table:** `accounts_user`.
- **Filter:** `firebase_uid=decoded['uid']`, active user; non-admin club presence checked.
- **Data Sent:** Bearer ID token only; no role/club claim is trusted from Flutter.
- **Data Returned:** Django `User` attached to `request.user`.
- **Error Handling:** invalid/missing/revoked token or local account state raises authentication failure.
- **Allowed Role:** any active configured role; admin may be clubless.

QRY-2: Read Squad

- **Source File:** `backend/academy/views.py`

- **Function:** `SquadListView.get()`.
- **Query:** `PlayerProfile.objects.select_related('user')`, coach adds `user__club=request.user.club`.
- **Table:** `academy_playerprofile` joined to `accounts_user`.
- **Filter:** same club for coach; all profiles for admin.
- **Data Sent:** no body; caller identity in token.
- **Data Returned:** `PlayerSerializer(...,many=True).data`.
- **Error Handling:** non-coach/admin raises `PermissionDenied`.
- **Allowed Role:** coach, admin.

QRY-3: Read Own/Guardian Player Profile

- **Source File:** `backend/academy/views.py`
- **Function:** `MyProfileView.get()` / `PlayerDetailView.get()`.
- **Query:** `get_object_or_404(PlayerProfile.objects.select_related('user'), user=request.user|user_id=player_id)`.
- **Table:** `academy_playerprofile`, `accounts_user`; guardian check reads `accounts_guardianlink`.
- **Filter:** player self or permitted guardian/player ID plus unlock.
- **Data Sent:** URL player ID for guardian, Bearer token, optional `X-Player-Unlock`.
- **Data Returned:** one `PlayerSerializer` map.
- **Error Handling:** 403 role/link/unlock, 404 profile.
- **Allowed Role:** player self; linked unlocked guardian through detail path; coach roster uses QRY-2.

QRY-4: Save Assessment

- **Source File:** `backend/academy/views.py`
- **Function:** `PlayerAssessmentView.put()`.
- **Query:** load profile by `user_id`; serializer update; audit insert.
- **Table:** `academy_playerprofile`, `academy_auditlog`.
- **Filter:** target player ID and equality of player/coach club IDs.
- **Data Sent:** nested 12 ratings + `coachNotes`.

- **Data Returned:** updated `PlayerSerializer` map.
- **Error Handling:** serializer 400, role/club 403, missing 404; notification after commit.
- **Allowed Role:** same-club coach.

QRY-5: Create/List Training Session

- **Source File:** `backend/academy/views.py`
- **Function:** `TrainingSessionListCreateView.get()/post()`.
- **Query:** `_sessions_for(user)` for reads; serializer `.save(created_by=user,club=user.club)` for insert.
- **Table:** `academy_trainingsession`; audit row on create.
- **Filter:** club sessions for normal user; admin helper returns all.
- **Data Sent:** title/date/time/location/tiers/focus JSON.
- **Data Returned:** serialized list or new row.
- **Error Handling:** non-coach POST 403; invalid payload 400; DB error rollback.
- **Allowed Role:** authenticated roles can read their scoped sessions; coach creates.

QRY-6: Replace Session Attendance

- **Source File:** `backend/academy/views.py`
- **Function:** `SessionAttendanceView.post()`.
- **Query:** session lookup; in-club player ID set; atomic `update_or_create`; delete omitted session rows.
- **Table:** `academy_trainingsession`, `accounts_user`, `academy_attendance`.
- **Filter:** session ID, coach club ID, submitted player IDs.
- **Data Sent:** `records[]` with player/status/optional effort/note.
- **Data Returned:** every current attendance row for the session.
- **Error Handling:** date/serializer 400; role/club/player set 403; atomic rollback.
- **Allowed Role:** same-club coach.

QRY-7: Player Attendance History

- **Source File:** `backend/academy/views.py`
- **Function:** `AttendanceListView.get()`.
- **Query:** select related session/recorder/player and filter `player_id`.
- **Table:** `academy_attendance` plus related tables.
- **Filter:** query parameter `player`; authorization/link/unlock first.
- **Data Sent:** `?player=<id>`, token, optional unlock header.
- **Data Returned:** serialized attendance list.
- **Error Handling:** missing parameter 400; unauthorized 403.
- **Allowed Role:** player self, linked unlocked guardian, same-club coach, admin according to helper.

QRY-8: Squad Progress Aggregate

- **Source File:** `backend/academy/views.py`
- **Function:** `SquadProgressView.get()`.
- **Query:** player profile list + attendance `.values('player_id').annotate(Count(...),Avg('effort'))`.
- **Table:** `academy_playerprofile`, `accounts_user`, `academy_attendance`.
- **Filter:** Coach adds `user__club_id=request.user.club_id` and `player__club_id=request.user.club_id`; Super Admin adds no Club filter.
- **Data Sent:** token only.
- **Data Returned:** manually shaped list of per-player aggregate maps.
- **Error Handling:** wrong role 403; DB failure 5xx.
- **Allowed Role:** Coach sees their own Club; Super Admin sees all Clubs.

QRY-9: Verify PIN

- **Source File:** `backend/academy/pin_service.py`, `academy/views.py`.
- **Function:** `verify_pin()` called by `PlayerPrivacyPinVerifyView.post()`.
- **Query:** transactional/select-for-update `PlayerPrivacyPin` row; GuardianLink existence before call.

- **Table:** `academy_playerprivacypin`, `accounts_guardianlink`.
- **Filter:** one player ID and current guardian/user relationship.
- **Data Sent:** PIN candidate; never stored/returned.
- **Data Returned:** verified boolean and signed unlock token; updated failure/lock state.
- **Error Handling:** invalid 400, locked 423, not linked 403, not set 400.
- **Allowed Role:** player self or linked guardian.

QRY-10: Eligibility Change and History

- **Source File:** `backend/portal/services.py`, `academy/signals.py`, `academy/views.py`.
- **Function:** `set_player_eligibility()`, `fire_eligibility_changed()`, `EligibilityHistoryView.get()`.
- **Query:** update profile status; insert history/audit; later filter history by player.
- **Table:** `academy_playerprofile`, `academy_eligibilityhistory`, `academy_auditlog`.
- **Filter:** staff/player same club on update; reader authorization/unlock on GET.
- **Data Sent:** new eligibility enum.
- **Data Returned:** portal redirect; later serialized history.
- **Error Handling:** form/permission errors; signal only logs a real change.
- **Allowed Role:** school staff updates; authorized player/guardian/staff/admin reads.

QRY-11: Injury Confirmation Workflow

- **Source File:** `backend/academy/views.py`.
- **Function:** `InjuryWorkflowListCreateView`, `InjuryWorkflowDetailView`, `InjuryReviewView`, `InjuryStatusUpdateListCreateView`, `InjuryStatusUpdateReviewView`, `InjuryArchiveView`.
- **Query:** scoped list/create/edit/withdraw; locked review transition; create and review recovery update; archive recovered report.
- **Table:** `academy_injuryrecord`, `academy_injurystatusupdaterequest`, `academy_auditlog`.
- **Filter:** self/link/unlock/same-Club care-team scope; Coordinator-only review/archive; pending/confirmed/recovered state guards.
- **Data Sent:** injury report, review action/reason, or requested status/resolution date.
- **Data Returned:** nested workflow row/list or 204.

- **Error Handling:** transition/validation 400, access 403, missing 404; transactions and row locks prevent stale double review.
- **Allowed Role:** Player/Guardian/Coach/Coordinator report by relationship; Player/Guardian/Coach request recovery updates; same-Club Coordinator reviews/archives; scoped care team/Admin reads.

QRY-12: Dispute and Response

- **Source File:** `backend/academy/views.py`.
- **Function:** `DisputeListCreateView.post()`, `DisputeResponseCreateView.post()`.
- **Query:** create dispute; atomic create response and optional status update.
- **Table:** `academy_dispute`, `academy_disputeresponse`.
- **Filter:** user role/club and target dispute scope.
- **Data Sent:** category/summary/detail/subject or response/status transition.
- **Data Returned:** serialized dispute/thread.
- **Error Handling:** invalid/cross-club/role rejected; transaction avoids response/status partial state.
- **Allowed Role:** coach creates; authorized coach/staff/admin respond/read according to view scope.

QRY-13: Register Device

- **Source File:** `backend/academy/views.py`.
- **Function:** `DeviceRegisterView.post()`.
- **Query:** `DeviceToken.objects.update_or_create(token=..., defaults={user,platform})`.
- **Table:** `academy_devicetoken`.
- **Filter:** globally unique token.
- **Data Sent:** token/platform; authenticated actor comes from request.
- **Data Returned:** 204 no body.
- **Error Handling:** blank token 400; auth failure 401.
- **Allowed Role:** any authenticated user.

QRY-14: Account Provisioning

- **Source File:** `backend/accounts/services.py`, `backend/portal/services.py`, `backend/academy/views.py`, and both seed commands.
- **Function:** `provision_user()`, `provision_player()`, `provision_managed_player()`, `create_club_account()`, and seed repair helpers.
- **Query:** user insert/update plus profile/link creation under transactions.
- **Table:** `accounts_user`, `academy_playerprofile`, `accounts_guardianlink`.
- **Filter:** unique email/Firebase UID; guardian/player/club checks.
- **Data Sent:** trusted role-specific form/serializer data; server club.
- **Data Returned:** user/temp-credential metadata or portal result.
- **Error Handling:** generic Player creation is rejected; missing/inactive Club or invalid same-Club guardian is rejected; DB failure rolls back the aggregate and compensation deletes a newly created Firebase identity. Seed commands repair non-Admin Club scope and Player profiles idempotently.
- **Allowed Role:** coordinator for own club or admin for protected global path.

QRY-15: Photo Path

- **Source File:** `backend/academy/storage.py`, `academy/views.py`, `portal/views.py`.
- **Function:** `upload_photo()`, `PlayerPhotoUploadView.post()`, `player_photo()`.
- **Query:** external object upload followed by profile path update; signed URL query for read.
- **Table:** `academy_playerprofile.photo_path`; external private storage object.
- **Filter:** target player ID; same-Club Coordinator in the portal, same-Club Coach through the mobile API, or Super Admin through the API.
- **Data Sent:** validated bytes/content type to storage; object path to DB.
- **Data Returned:** serialized player/success redirect; signed URL on later read.
- **Error Handling:** size/MIME/signature/config/network failures prevent path update.
- **Allowed Role:** same-Club Coach/Super Admin API; same-Club Coordinator portal.

QRY-16: Notification Inbox and Read State

- **Source File:** `backend/academy/notifications.py`, `backend/academy/views.py`.
- **Function:** `_send_to_users()`, `NotificationListView`, `NotificationUnreadCountView`, `NotificationReadView`, `NotificationReadAllView`.
- **Query:** bulk-create a recipient record before FCM; list newest 100; count unread; update one/all `read_at` values.

- **Table:** `academy_notificationrecord` plus `academy_devicetoken` for optional push fanout.
- **Filter:** active recipient IDs when creating; `user=request.user` on every inbox read/update.
- **Data Sent:** server-created event/title/body/data; authenticated request for inbox actions.
- **Data Returned:** serialized `AppNotification` contract, unread count, or updated count/read record.
- **Error Handling:** cross-user record IDs return 404; FCM unavailable leaves the persisted inbox row intact.
- **Allowed Role:** any authenticated user, strictly for their own inbox.

QRY-17: Create or Update Football Match

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`.
- **Function:** `FootballMatchListCreateView.post()`, `FootballMatchDetailView.put()`.
- **Query:** insert/update `FootballMatch`; optional locked fixture link/status update; insert audit row.
- **Table:** `academy_footballmatch`, `academy_tournamentfixture`, `academy_auditlog`.
- **Filter:** Coordinator role and server-owned Club; update resolves `pk=match_id, club_id=request.user.club_id`.
- **Data Sent:** optional same-Club fixture ID or ad-hoc opponent/competition/date/venue plus team scores; no writable Club or creator.
- **Data Returned:** `FootballMatchSerializer` map.
- **Error Handling:** future date/invalid values 400; wrong role 403; cross-Club/missing match 404.
- **Allowed Role:** same-Club Coordinator for writes; Coordinator/Coach/Admin for scoped reads.

QRY-18: Upsert Player Match Performance

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`, `backend/academy/models.py`.
- **Function:** `MatchPerformanceDetailView.put()`.
- **Query:** locked parent/existing-row read; aggregate other player goals; insert/update one performance; insert audit.
- **Table:** `academy_footballmatch`, `academy_playermatchperformance`, `accounts_user`, `academy_auditlog`.
- **Filter:** Coordinator-owned match and same-Club account with role `PLAYER`; unique `(match_id, player_id)`; confirmed injury check.
- **Data Sent:** objective match statistics plus optional injury-warning acknowledgement; match/player/recorder ownership comes from URL plus verified user.
- **Data Returned:** nested `PlayerMatchPerformanceSerializer` map.
- **Error Handling:** serializer/model/score inconsistency 400; unacknowledged injury 409; wrong role/Club 403/404; atomic rollback and row locks protect concurrency.

- **Allowed Role:** same-Club Coordinator.

QRY-19: Read Player Match Summary and History

- **Source File:** `backend/academy/views.py`, `backend/academy/match_statistics.py`.
- **Function:** `PlayerMatchStatisticsView.get()`, `build_performance_summary()`.
- **Query:** select related match/Club/player, filter by player and optional date range, newest-first slice of 1-100 rows.
- **Table:** `academy_playermatchperformance`, `academy_footballmatch`, `accounts_user`, `accounts_club`.
- **Filter:** Player self, same-Club Coach, linked Guardian with PIN unlock when configured, or Admin.
- **Data Sent:** URL player ID; optional `from`, `to`, and `limit`.
- **Data Returned:** player identity, aggregate summary, and serialized historical rows.
- **Error Handling:** malformed/reversed date range or invalid limit 400; disallowed reader 403; missing Player 404.
- **Allowed Role:** Player self, same-Club Coach, linked unlocked Guardian, Admin.

QRY-20: Publish Tournament Schedule and Fixtures

- **Source File:** `backend/portal/views.py`, `backend/academy/storage.py`, `backend/academy/views.py`.
- **Function:** `tournament_schedules()`, `tournament_schedule_detail()`, fixture edit/delete views, `TournamentScheduleListView.get()`.
- **Query:** insert/update/delete Club schedule and fixtures; save opaque document path; read published nested schedules.
- **Table:** `academy_tournamentschedule`, `academy_tournamentfixture`, `academy_footballmatch`, `academy_auditlog`; private Supabase Storage object.
- **Filter:** portal Coordinator's `request.user.club_id`; mobile non-admin Club scope; published schedules only.
- **Data Sent:** validated document bytes and structured fixture fields.
- **Data Returned:** portal redirect or nested schedule/fixture JSON with short-lived signed document URL.
- **Error Handling:** invalid document/signature/size, storage failure, cross-Club access, or completed-history deletion guard stops mutation.
- **Allowed Role:** same-Club Coordinator manages; authenticated Club mobile roles read; Admin may inspect all published schedules.

QRY-21: Save or Clear Coach Match Rating

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`.
- **Function:** `MatchPerformanceRatingView.put()/.delete()`.

- **Query:** locked existing performance update of rating/notes/rater/time; audit insert; clear only subjective fields.
- **Table:** `academy_playermatchperformance`, `academy_auditlog`.
- **Filter:** same-Club Coach match and existing `(match_id, player_id)` performance.
- **Data Sent:** `coachRating` 0.0-10.0 and optional notes.
- **Data Returned:** serialized performance or 204.
- **Error Handling:** invalid 400, wrong role/Club 403/404, missing statistics 404.
- **Allowed Role:** same-Club Coach.

QRY-22: Live Database Selection

- **Source File:** `backend/config/settings.py`.
- **Function:** settings-time `DATABASES` selection.
- **Query:** PostgreSQL configuration when `DB_HOST` exists; SQLite only under DEBUG/testing.
- **Table:** all Django ORM tables through the selected engine.
- **Filter:** environment and `TESTING` state, not a client request.
- **Data Sent:** server-only `DB_*` configuration.
- **Data Returned:** configured Django database connection.
- **Error Handling:** non-test production without `DB_HOST` raises `ImproperlyConfigured` instead of silently using SQLite.
- **Allowed Role:** deployment configuration only; Flutter has no database credentials.

QRY-23: Create, Edit, Delete, or Publish Mobile Tournament Brackets

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`, `backend/academy/models.py`.
- **Function:** `TournamentScheduleListView.post()`, `TournamentScheduleDetailView.patch()`, `TournamentSchedulePublishView.post()`, `TournamentAgeBracketCreateView.post()`, `TournamentAgeBracketDetailView.patch()/.delete()`.
- **Query:** insert/update Club tournament and U-age rows; dependency and current-roster eligibility reads; publish timestamp/status update; audit insert.
- **Table:** `academy_tournamentschedule`, `academy_tournamentagebracket`, `academy_tournamentsquad`, `academy_tournamentsquadentry`, `academy_tournamentfixture`, `academy_auditlog`.
- **Filter:** verified Coordinator role and `schedule__club_id=request.user.club_id`; unique `(schedule_id,max_age)`.

- **Data Sent:** title, start date, bracket maximum age, optional UTC schedule timestamp; no writable Club/actor/status.
- **Data Returned:** complete nested `TournamentScheduleSerializer` map or 204 on safe bracket deletion.
- **Error Handling:** missing/duplicate bracket, no bracket on publish, cross-Club ID, or change/delete that conflicts with fixtures, publication, or roster eligibility returns 400/403/404 and preserves prior state.
- **Allowed Role:** same-Club Coordinator for mutations; role-scoped readers through the schedule GET.

QRY-24: Read and Save Coach Tournament Squad

- **Source File:** `backend/academy/views.py`, `backend/academy/tournament_rosters.py`, `backend/academy/serializers.py`.
- **Function:** `TournamentSquadCandidatesView.get()`, `TournamentSquadDetailView.get()/.put()`.
- **Query:** same-Club active Player candidate read; eligibility/injury lookups; locked create/update of squad; reconcile entry insert/update/delete; audit insert.
- **Table:** `accounts_user`, `academy_playerprofile`, `academy_injuryrecord`, `academy_tournamentsquad`, `academy_tournamentsquadentry`, `academy_auditlog`.
- **Filter:** bracket belongs to authenticated Club; Coach-only candidate/write actions; unique bracket squad and unique squad/player entry.
- **Data Sent:** bracket URL ID plus entry `playerId` and optional tournament position; Club/Coach actor is server-derived.
- **Data Returned:** privacy-safe candidate list or nested typed squad JSON.
- **Error Handling:** duplicate/unknown/inactive/cross-Club/blocked Player or invalid position returns 400; wrong role 403; atomic transaction prevents partial roster replacement.
- **Allowed Role:** same-Club Coach writes/selects; Coordinator/Admin inspect; Player/Guardian read only published tournament rosters.

QRY-25: Publish Coach Tournament Squad

- **Source File:** `backend/academy/views.py`, `backend/academy/tournament_rosters.py`.
- **Function:** `TournamentSquadPublishView.post()`, `roster_eligibility()`.
- **Query:** lock squad, load entries/profiles/injuries, update status/publish time/actor, insert audit row.
- **Table:** `academy_tournamentsquad`, `academy_tournamentsquadentry`, `academy_playerprofile`, `academy_injuryrecord`, `academy_auditlog`.
- **Filter:** same-Club Coach, published parent tournament, non-empty squad, no hard-blocked current member.
- **Data Sent:** bracket URL ID only; publish status, time, and actor are server-controlled.
- **Data Returned:** published `TournamentSquadSerializer` map.
- **Error Handling:** tournament still Draft, missing/empty squad, or newly blocked entry returns 400/404; the row lock protects concurrent publication/change.
- **Allowed Role:** same-Club Coach.

QRY-26: Enforce Tournament Squad During Match-Performance Save

- **Source File:** `backend/academy/views.py`, `backend/academy/tournament_rosters.py`, `backend/academy/models.py`.
- **Function:** `MatchRosterView.get()`, `MatchPerformanceDetailView.put()`, `_match_age_bracket()`.
- **Query:** resolve fixture bracket; read published squad, eligibility, injury, and existing performance; lock match/performance; insert/update performance and optional override metadata/audit.
- **Table:** `academy_tournamentfixture`, `academy_tournamentsquad`, `academy_tournamentsquadentry`, `academy_playermatchperformance`, `academy_injuryrecord`, `academy_auditlog`.
- **Filter:** same-Club role-scoped match; Coordinator-only expanded candidates/write; published membership or eligible exception with required reason.
- **Data Sent:** objective statistics and optional `squadOverrideReason`; membership, eligibility, override actor/time, and Club are server-derived.
- **Data Returned:** membership-aware roster rows or serialized saved performance.
- **Error Handling:** hard-blocked eligibility, missing/oversize reason, cross-Club Player, inconsistent statistics, or constraint failure returns 400/403/404 and rolls back.
- **Allowed Role:** Coordinator enforces/selects/writes; Coach/Admin can read the normal roster; other trend reads retain their existing object/privacy rules.

Variable Traces

Variable: User ID / Firebase UID

`FirebaseAuth.currentUser.uid` -> signed ID token -> Firebase Admin decoded `uid` -> `accounts_user.firebase_uid` -> Django `request.user.id` -> model actor/player foreign keys. The numeric Django ID, not Firebase UID, is used for relational player references.

Variable: Role

`accounts_user.role` -> `UserSerializer` role wire value -> `UserProfile.fromJson()` -> `HomeScreen` portal choice. For APIs the role is read again from `request.user.role`; Flutter never sends an authoritative role.

Variable: Player ID

`PlayerSerializer.id` (Django user PK) -> `Player.fromJson().id` string -> route/provider family parameter -> URL `<player_id>` or request record `playerId` -> backend same-club/ownership/link lookup -> `PlayerProfile.user_id`/related FKs.

Variable: Coach ID

Verified token UID -> local Coach `request.user.id` -> server-assigned `rated_by_id`, attendance `recorded_by_id`, audit `actor_id`, or dispute `raised_by_id`. Match/result and objective-statistics ownership now comes from the Coordinator's separately verified identity. Actor IDs are not accepted as trusted client payload fields.

Variable: Training ID

`TrainingSession.id` returned by Django PK -> schedule card entity -> edit/delete URL or attendance/session-confirmation payload -> `TrainingSession.objects.get(pk=...)` -> attendance/confirmation `session_id` FK -> response `sessionId`.

Variable: Attendance Status

UI enum -> `_marks[playerId]` -> `Attendance.status` -> uppercase `toJson()` -> DRF choice validation -> `academy_attendance.status` -> serializer JSON -> enum `fromWire()` -> chip/aggregate.

Variable: Performance Score

Slider/input integer -> `PlayerRatings` field -> nested JSON -> `AssessmentSerializer` 0-99 validation -> `PlayerProfile` integer column -> nested response -> `PlayerRatings` -> arithmetic `overall` -> card/radar.

Variable: Match ID and Player Match Key

`FootballMatchSerializer.id` -> `FootballMatch.id` -> match-card route -> provider/controller `matchId` -> `/api/matches/{matchId}/...` -> same-Club `FootballMatch` lookup -> `PlayerMatchPerformance.match_id`. Combined with the selected `Player.id`, the database unique `(match_id, player_id)` key identifies exactly one historical performance.

Variable: Coach Rating and Trend

Coach form decimal 0.0-10.0 -> `MatchRatingDraft.coachRating` -> rating-only JSON -> `CoachMatchRatingSerializer` Decimal validation -> `academy_playermatchperformance.coach_rating` plus `rated_by_id/rated_at` -> derived one-decimal mean and serialized row -> Dart `double` -> `PerformanceTrendChart` chronological point.

Variable: Tournament Fixture ID

`TournamentFixtureSerializer.id` -> `TournamentFixture.id` -> Coordinator result-selection UI -> `FootballMatchDraft.fixtureId` -> JSON `fixtureId` -> same-Club locked fixture lookup -> `TournamentFixture.completed_match_id` -> fixture status `COMPLETED`. The server derives opponent/date/venue/competition from the fixture rather than trusting repeated client fields.

Variable: Tournament and Age-Bracket ID

`TournamentScheduleSerializer.id`/nested bracket ID -> Dart `TournamentSchedule.id`/`TournamentAgeBracket.id` -> Schedule card/editor route -> repository REST path -> same-Club schedule/bracket lookup -> `TournamentAgeBracket.schedule_id`. The U-age label is derived from validated `max_age`; Flutter cannot choose the owning Club through either ID.

Variable: Tournament Squad Eligibility

Stored Player date of birth, tournament year/U-age limit, profile existence, and confirmed/pending injury rows -> `roster_eligibility(player,bracket)` -> `ELIGIBLE`, `WARNING`, or `BLOCKED` plus stable code/reason -> candidate JSON -> Dart enum -> enabled/disabled selection and warning icon. The server recomputes the result on save and publish, so the UI value is informative rather than authoritative.

Variable: Squad Override Reason

Coordinator exception form text -> `MatchPerformanceDraft.squadOverrideReason` -> JSON `squadOverrideReason` -> trimmed 1-500 character server validation -> `academy_playermatchperformance.squad_override_reason` with verified `squad_override_by_id` and `squad_override_at` -> audit event and serialized exception flag. The private justification is evidence, not a client-selected permission grant.

Variable: Injury Review Status

New report -> forced `PENDING` -> Coordinator review action `CONFIRM`/`REJECT` -> locked transition to `CONFIRMED`/`REJECTED` -> optional recovery-update approval -> injury state changes -> Confirmed Recovered report -> Coordinator archive -> `ARCHIVED`. Serializer enums become typed Dart workflow states and determine visible actions.

Variable: Academic Eligibility

Portal form enum -> cleaned value -> `set_player_eligibility` -> current profile column + signal old/new rows -> JSON history/current profile -> Dart eligibility enum/change object -> badge/card.

Variable: Guardian Unlock Token

Successful PIN candidate -> `verify_pin` -> `issue_player_unlock(request.user.id, player.id)` -> response string -> `PlayerUnlockTokenStore[playerId]` -> `X-Player-Unlock` -> `require_player_unlock()` signed user/player/max-age check.

Variable: Evaluation ID

There is no assessment/evaluation entity ID. Evaluations overwrite the player profile row keyed by player ID. Per-session effort belongs to an attendance row ID server-side, but the Dart entity identifies it through player/session rather than exposing an attendance PK.

Variable: Scout ID / Scouting Report ID

`NOT IMPLEMENTED IN CURRENT REPOSITORY`. No variable, role, model, endpoint, or table carries these IDs.

Object / Model Traces

Object: UserProfile

- **Created from:** `/api/auth/me/ UserSerializer` JSON.
- **Converted using:** `UserProfile.fromJson()`.
- **Stored in:** local widget route/bootstrap state, passed to `HomeScreen`/coach portal; Firebase maintains identity session separately.
- **Consumed by:** role routing, coach header/profile, logout flows.
- **Key fields:** local identity, display name/email, role, club.

Object: Player

- **Created from:** `PlayerSerializer` JSON from squad/me/detail/assessment/position/photo.
- **Converted using:** `Player.fromJson()` with nested rating/tier/position/eligibility converters.
- **Stored in:** Riverpod `FutureProvider` results and route constructor arguments.
- **Consumed by:** roster cards, profile, assessment editor, guardian/player dashboards.
- **Key fields:** ID/name/age/class/tier/position/12 ratings/eligibility/photo/notes.

Object: TrainingSession

- **Created from:** schedule form draft or API JSON.
- **Converted using:** `TrainingSession.toJson()/fromJson()`.
- **Stored in:** `academy_trainingsession`; transient Riverpod session list.
- **Consumed by:** schedule cards, edit form, attendance screen, player confirmation.
- **Key fields:** ID/title/tiers/date/time/location/focus/attendee count.

Object: Attendance

- **Created from:** attendance marks or API JSON.

- **Converted using:** `Attendance.toJson()/fromJson()`.
- **Stored in:** `academy_attendance`; queued batch JSON in device `outbox_attendance` on network failure.
- **Consumed by:** log screen, histories, dashboard summaries/streak, progress aggregate.
- **Key fields:** player/session/status/updated time/coach/effort/note.

Object: FootballMatch

- **Created from:** Coordinator `FootballMatchDraft`, optional tournament fixture, or `FootballMatchSerializer` JSON.
- **Converted using:** `FootballMatchDraft.toJson()` and `FootballMatch.fromJson()`.
- **Stored in:** `academy_footballmatch`; transient `footballMatchesProvider` list and route arguments.
- **Consumed by:** Coordinator result/statistics screens, Coach roster/rating screens, and nested Player/Guardian performance history.
- **Key fields:** ID, opponent, competition, played date, venue, both scores; Club/creator remain server-owned.

Object: MatchPerformance

- **Created from:** Coordinator objective `MatchPerformanceDraft`, Coach `MatchRatingDraft`, or `PlayerMatchPerformanceSerializer` JSON.
- **Converted using:** draft `toJson()` and `MatchPerformance.fromJson()` with nested `FootballMatch`.
- **Stored in:** unique `academy_playermatchperformance` row; transient match/player family providers.
- **Consumed by:** Coordinator statistics roster/editor, Coach rating roster/editor, and Player/Guardian/Coach summary, trend chart, and match-history cards.
- **Key fields:** player/match IDs, position/start/minutes, attacking/passing/defending/card/goalkeeper statistics, coach rating, and notes.

Object: PlayerMatchStatistics

- **Created from:** `/api/players/{playerId}/match-statistics/` aggregate response.
- **Converted using:** `PlayerMatchStatistics.fromJson()` with `MatchPerformanceSummary.fromJson()` and performance-list conversion.
- **Stored in:** Riverpod `playerMatchStatisticsProvider(playerId)` only; the summary is derived, not persisted.
- **Consumed by:** Player Match Statistics, protected linked-Guardian child view, and Coach player-trend routes.
- **Key fields:** authorized player identity, totals/rates/average, and historical performance rows.

Object: EligibilityChange

- **Created from:** eligibility signal-created backend row returned by API.

- **Converted using:** `EligibilityChange.fromJson()`.
- **Stored in:** `academy_eligibilityhistory`; Riverpod family provider list.
- **Consumed by:** `EligibilityHistoryScreen._ChangeCard`.
- **Key fields:** old/new status, changer representation, timestamp.

Object: TournamentSchedule

- **Created from:** Coordinator portal forms or `/api/tournament-schedules/` JSON.
- **Converted using:** `TournamentSchedule.fromJson()` with nested `TournamentAgeBracket.fromJson()`, `TournamentSquad.fromJson()`, and `TournamentFixture.fromJson()`.
- **Stored in:** `academy_tournamentschedule`, `academy_tournamentagebracket`, `academy_tournamentfixture`, private object storage path; transient Riverpod provider.
- **Consumed by:** Coordinator portal/mobile management, Coach roster workflow, Club schedule cards, and fixture-backed result entry.
- **Key fields:** Club-owned title/start date/private document path/publish timestamps, U-age brackets/times/squads, and fixture stage/opponent/kickoff/venue/location/status/completed match.

Object: TournamentSquad and TournamentRosterCandidate

- **Created from:** Coach candidate/squad APIs and Coach `TournamentRosterSelection` input.
- **Converted using:** `TournamentRosterCandidate.fromJson()`, `TournamentSquad.fromJson()`, `TournamentSquadEntry.fromJson()`, and selection `toJson()`.
- **Stored in:** one `academy_tournamentsquad` per bracket plus unique `academy_tournamentsquadentry` rows; provider/screen selection state is transient.
- **Consumed by:** Coach roster editor, Coordinator/Admin inspection, published Player/Guardian roster view, and match-roster enforcement.
- **Key fields:** bracket ID, Draft/Published status/time, selected Player ID/name/event position, and manager-visible privacy-safe availability state/code/reason.

Object: InjuryRecord

- **Created from:** player form draft or API JSON.
- **Converted using:** `InjuryRecord.toJson()/fromJson()`.
- **Stored in:** `academy_injuryrecord` with nested `academy_injurystatusupdaterequest`; player and Club provider lists.
- **Consumed by:** care-team report/history views and Coordinator confirmation/recovery/archive queue.
- **Key fields:** Player, reporter, report review state/reason/reviewer, injury state/dates, archive time, and optional Pending recovery update.

Object: Dispute

- **Created from:** coach flag form or API list/thread response.
- **Converted using:** `Dispute.fromJson()`.
- **Stored in:** `academy_dispute` with `academy_disputeresponse` children.
- **Consumed by:** dispute list/thread/form providers.

Object: SessionConfirmation

- **Created from:** player response API JSON.
- **Converted using:** `SessionConfirmation.fromJson()`.
- **Stored in:** unique `academy_sessionconfirmation` player/session row.
- **Consumed by:** `SessionConfirmationButton` and coach confirmation views/providers.

Object: AppNotification

- **Created from:** current-user `NotificationRecordSerializer` JSON.
- **Converted using:** `AppNotification.fromJson()`.
- **Stored in:** `academy_notificationrecord`; transient Riverpod inbox lists and unread count.
- **Consumed by:** `NotificationBell`, `NotificationInboxScreen`, `NotificationNavigationController`, foreground/opened-push handling, role-aware destination routing, and mark-read actions.
- **Key fields:** ID/type/title/body/neutral data/read state/timestamp; FCM is a delivery channel, not the history store.

Button-to-Database/API Traces

#	Button/action	Callback	Controller/use case	API/service	Store/result	UI change
1	Login	`LoginScreen._handleSignIn()`	`LoginController.signIn` -> `SignIn`	Firebase sign-in + `GET /api/auth/me/`	reads `accounts_user`	`HomeScreen` or error
2	Forgot password	`_handleForgotPassword()`	reset controller/use case	Firebase `sendPasswordResetEmail`	Firebase identity service	feedback/error
3	Logout	role screen `_signOut()`	`UnregisterDevice` -> `SignOut`	current-token `DELETE /api/devices/` -> Firebase `signOut`	scoped device row removed best-effort; local session/cache/unlock cleared	login route even if unregister fails
4	Save session	`ScheduleSessionScreen._submit()`	schedule controller/use case	`POST /api/training-sessions/`	insert session/audit	pop + refreshed list
5	Edit session	same `_submit()` with `existing`	`.saveChanges` -> update use case	`PUT /api/training-sessions/{id}/`	update session/audit	pop + refresh
6	Cancel session	`_cancelSession()`	controller `.cancel`	`DELETE /api/training-sessions/{id}/`	delete session; on-commit cancellation inbox/push; attendance retains null FK	list refresh
7	Finalize attendance	`LogAttendanceScreen._finalize()`	attendance controller/use case	POST attendance or local outbox	upsert/prune attendance / queue	pop + history refresh
8	Save assessment	`EditPerformanceDataScreen._save()`	assessment controller/use case	`PUT /api/players/{id}/assessment/`	update player profile/audit	updated card/profile
9	Assign position	position picker selection	position controller/use case	`PUT /api/players/{id}/position/`	update profile/audit	position label refresh
10	Verify PIN	privacy gate submit	PIN controller/use case	`POST /api/players/{id}/pin/verify/`	PIN counter/lock; signed token	unlocked child content
11	Confirm session	confirmation option `respond()`	confirmation controller/use case	`POST /api/session-confirmations/`	upsert confirmation	selected response or visible retry feedback
12	Report injury	injury form `_save()`	injury controller/use case	`POST /api/injuries/`	insert Pending report/audit	reporter and Club queues refresh
13	Confirm/reject injury	Coordinator review action	controller `.reviewReport`	`POST /api/injuries/{id}/review/`	locked report transition/audit	review queue refresh
14	Request/review recovery	care-team/Coordinator action	`.requestStatus` / `.reviewStatus`	status-update POST/review endpoints	insert request; approved transition/audit	history/Club queues refresh
15	Flag dispute	`FlagDisputeScreen._submit()`	dispute controller/use case	`POST /api/disputes/`	insert dispute	list/thread refresh
16	Respond dispute	response submit	controller `.respond`	`POST /api/disputes/{id}/responses/`	insert response/update status	thread refresh

#	Button/action	Callback	Controller/use case	API/service	Store/result	UI change
17	Staff eligibility save	portal form submit	portal service	Django `set_player_eligibility`	profile + history + audit	portal/mobile refresh
18	Upload photo	Coach picker or portal multipart submit	photo controller/use case or portal service	Django multipart -> Supabase Storage REST	object + profile path	immediate/refetched avatar or visible error
19	Coordinator create account	portal form submit	`create_club_account`	Firebase Admin/service/ORM	user/profile/link	roster/account list
20	Notification bell / row / push open	bell or `_openMessage()` / `_openNotification()`	notification providers + `NotificationNavigationController`	current-user notification and profile APIs	read/update `NotificationRecord`; protected destination data remains API-scoped	badged inbox or role-aware schedule/profile/eligibility destination
21	Record match	Coordinator `EditFootballMatchScreen._submit()`	`MatchManagementController.create` -> `CreateFootballMatch`	`POST /api/matches/`	insert Club-owned match, optional fixture link, audit	refreshed results/fixtures
22	Save objective statistics	Coordinator statistics editor	controller -> `SaveMatchPerformance`	`PUT /api/matches/{matchId}/performances/{playerId}/`	atomic upsert unique match/player row + injury override audit when used	roster and Player trends refresh
23	Save Coach rating	Coach rating editor	controller -> `SaveMatchRating`	PUT rating subresource	update rating/notes/rater/time + audit	roster and trends refresh
24	Publish tournament	Coordinator portal form	Django form/storage helpers	portal POST + Supabase Storage REST	schedule/private path/audit	portal redirect; mobile schedule available
25	Add fixture	Coordinator portal fixture form	Django ModelForm/view	portal POST	structured fixture/audit	schedule detail refresh
26	Archive injury	Coordinator confirmation	controller `.archive`	`POST /api/injuries/{id}/archive/`	Confirmed Recovered -> Archived + audit	removed from normal queue
27	Create/edit mobile tournament	`EditTournamentScreen._saveDetails()`	`TournamentManagementController`	POST/PATCH tournament schedule API	Club schedule/audit row	nested Draft schedule refresh
28	Add/edit/remove U-age bracket	`_editBracket()` / `_removeBracket()`	tournament controller	bracket POST/PATCH/DELETE APIs	bracket/dependency validation/audit	chips, time, and divisions refresh
29	Publish mobile tournament	`EditTournamentScreen._publish()`	controller `.publish()`	schedule publish POST	status/time/audit update	visible to Club roles; Coach can publish roster
30	Save/publish tournament squad	`TournamentSquadScreen._save()`	`TournamentRosterManagementController`	squad PUT then optional publish POST	atomic squad/entry reconciliation + audit	Draft/Published membership refresh
31	Add eligible squad exception	`MatchRosterScreen._addOutOfSquadPlayer()`	roster candidate provider -> performance controller	expanded roster GET + performance PUT	reason/verified actor/time + objective row/audit	approved-exception chip and trends refresh

Screen-to-Screen Navigation Traces

- App launch -> `MaterialApp.home` -> `SessionBootstrapScreen` -> conditional `LoginScreen`/`HomeScreen` by widget build.

- Login -> `Navigator.pushReplacement(MaterialPageRoute(HomeScreen(profile)))` -> role portal.
- `HomeScreen` -> direct widget return `CoordinatorPortalScreen`, `CoachPortalScreen`, `PlayerPortalScreen`, or `GuardianPortalScreen` according to the server-returned role.
- Portal bottom navigation -> local tab index -> `IndexedStack` child; no route push and state is retained.
- Training list add -> `_openScheduleForm()` -> `Navigator.push(MaterialPageRoute(ScheduleSessionScreen))`; boolean result triggers refresh behavior.
- Training card tap -> `_logAttendance(session)` -> `Navigator.push(LogAttendanceScreen(session,...))`; full session object is passed.
- Player card -> `Navigator.push` player-profile screen; `Player` object/ID passed.
- Player profile edit assessment -> `Navigator.push(EditPerformanceDataScreen(player))`; saved `Player?` returns through pop.
- Coordinator adaptive shell -> Tournament tab (`TournamentScheduleScreen`) / Matches (`CoordinatorMatchesScreen`) / Injuries (`CoordinatorInjuriesScreen`) / Account. Fixture result or match card opens `EditFootballMatchScreen/MatchRosterScreen`; selecting a Player opens `EditMatchPerformanceScreen`.
- Coordinator Tournament tab -> **Create tournament** or edit icon -> `EditTournamentScreen`; save/add bracket/publish returns to an invalidated schedule list. Coach/authorized Club-role bracket tap -> `TournamentSquadScreen`; only Coach receives edit controls, while published-roster readers receive the read-only list.
- Coordinator age-bracket match roster -> **Add eligible squad exception** -> `AdaptiveFormModal` candidate/reason sheet -> `EditMatchPerformanceScreen`; the saved result returns to the same roster and refetches normal/expanded providers.
- Coach match roster -> Player with existing statistics -> `EditMatchRatingScreen`; Coach Progress player card/profile trend -> `PlayerMatchStatisticsScreen(playerId,...)`.
- Player Progress tab -> embedded `PlayerMatchStatisticsView(playerId: currentPlayer.id)`; linked Guardian protected child destinations can open the same authorized statistics view after PIN unlock.
- Eligibility tile -> `Navigator.push(EligibilityHistoryScreen(playerId,...))`.
- Attendance history button -> `Navigator.push(AttendanceHistoryScreen(playerId,...))`.
- Injury history tile -> `Navigator.push(InjuryHistoryScreen(playerId,readOnly...))`.
- Notification bell -> `Navigator.push(NotificationInboxScreen())`. A known inbox row, foreground SnackBar **View**, `onMessageOpenedApp`, or `getInitialMessage()` -> `NotificationNavigationController` -> current profile/role resolution -> schedule tab (`session_*`), Player/authorized Guardian profile (`assessment_saved`), or Player/authorized Guardian eligibility history (`eligibility_changed`). Unknown events or profile-resolution failures open/focus the inbox. Player IDs cannot be overridden by payload data, Guardian IDs are checked against linked children, and protected destinations retain PIN/privacy gates.
- Sign out -> navigation replacement/removal to `LoginScreen`.

Navigation uses Flutter `Navigator` with `MaterialPageRoute`, plus `IndexedStack` for tabs. No GoRouter, AutoRoute, GetX, or named-route table was found.

Async Code Traces

Firebase/app bootstrap

`main() async` awaits Firebase plugin setup before constructing repositories that depend on it. Without `await`, the widget tree could request Auth/Messaging before the default app exists. The UI thread is not synchronously blocked; Flutter's event loop resumes the future.

Login and restore

Async operations wait for auth state, credential sign-in, token issuance, network HTTP, and JSON results. Controllers set loading before `await` and state/error after. Removing `await` would navigate before profile authorization or let exceptions escape the intended catch.

API read providers

Riverpod `FutureProvider` runs HTTP asynchronously and exposes loading/data/error. Widgets rebuild on completion instead of blocking frames. Live repository reads cross `AuthenticatedApiClient`; only a pre-response network failure may resolve from a fresh current-owner cache entry. Disposing a screen releases `autoDispose` provider state, although an underlying HTTP call is not automatically a database transaction cancellation.

Mutations

Async controllers disable save buttons while awaiting to reduce duplicate writes. `mounted`/`context.mounted` checks prevent navigation or `setState` on disposed UI after completion.

Tournament mutation ordering is intentional. `TournamentSquadScreen._save(publish: true)` awaits the squad PUT before the publish POST, so publication validates the exact persisted entries. Each success invalidates schedules/candidates; a partial outcome is reported explicitly as "draft saved, publish failed" rather than pretending the two requests were one client-side action.

Attendance synchronization

Network and sqflite operations are awaited sequentially so queue order is preserved. Running all retries in parallel would break last-write intent for multiple batches targeting a session.

Unawaited device registration

Login/bootstrap uses `unawaited(registerDeviceProvider...)` because push registration is secondary and should not delay portal entry. The tradeoff is reduced immediate error visibility.

Foreground and opened notifications

`FirebaseMessaging.onMessage` and `onMessageOpenedApp` stream subscriptions live for the app state lifetime; `getInitialMessage()` handles a push that launched the app, and `onTokenRefresh` re-registers the current token. Handlers invalidate inbox/count providers and use navigator/messenger keys without requiring a screen-local context. Open actions pass through the same tested destination controller as inbox-row taps, while unknown/unsafe requests fall back to the persistent inbox.

Django `transaction.on_commit`

This callback is asynchronous in event timing, not a Flutter `Future`: it defers notification persistence/FCM side effects until the relational commit succeeds. Session cancellation snapshots recipient IDs before deleting, but registers its notification only after `session.delete()` succeeds; a failed delete cannot announce a cancellation. Notification helpers must not change the already returned domain result.

Defense answers

- **Why async/await?** Firebase, HTTP, sqflite, and platform operations finish later; `await` preserves readable ordered control flow and targeted error handling.
- **Does it block the UI?** Await yields control to Dart's event loop; rendering continues unless CPU-heavy synchronous work is performed, which these flows do not do.
- **What if `await` is removed?** Code receives a `Future` rather than its value, loading flags may reset early, navigation can race, and errors may bypass the catch.

Failure / Error Path Matrix

Workflow	Validation failure	Auth/authorization	Network/database/external failure	UI/result
Restore	malformed profile -> parse/auth error	401/403 signs out	network/5xx -> restore error	retry/continue/login, never cached role authorization
Login	empty not prechecked; Firebase rejects	local missing/inactive/clubless rejects	Firebase/HTTP error	controller error text, button re-enabled
Schedule	empty/tier/date or paired/ordered time validation	non-coach/cross-club edit 403	HTTP/DB error	field/API error; form stays + SnackBar
Attendance	unmarked/dialog/window/serializer	non-coach/cross-club/player set 403	network queues; HTTP/DB does not	queued/success pop; other error SnackBar
Assessment	malformed/range serializer	non-coach/cross-club 403	transport/DB error	form remains, old current values
Match record	blank/future/venue/score or fixture conflict	only same-Club Coordinator writes	transport/DB error	form remains; no partial match/fixture link
Objective match statistics	inconsistent shots/goals/passes/GK/team score	only same-Club Coordinator writes; injury override explicit	atomic lock/constraint/DB error	warning/editor remains; providers stay authoritative
Coach rating	rating range/notes invalid	only same-Club Coach; existing statistics required	lock/DB error	rating editor remains; objective fields unchanged
Match statistics read	bad date range/limit	Player self, same-Club Coach, linked unlocked Guardian, Admin	network/DB error	empty/retry/error state; no broadened data
Profile read	optional photo null is valid	role/link/unlock rejects	network-only may use current-owner GET cache; HTTP/DB errors remain visible	cached data or AsyncError/retry; avatar fallback
PIN	format/incorrect -> 400	missing link 403	transaction error	failure count/lock feedback; no child detail
Eligibility	invalid form/choice	staff/club/read permission	DB/notification failure	DB success survives push failure; refresh required
Confirmation	invalid today/status/session	non-player/cross-club	network/server	submit SnackBar or **Retry RSVP**; no false selected response
Injury	invalid fields/dates/transition/rejection reason	relationship-scoped report; Coordinator-only review/archive	network/transaction	controller error, prior workflow state retained
Tournament	invalid document/signature/size/fixture	Coordinator own Club manages; mobile reads own Club	storage/network/DB	form error/retry; completed history protected
Mobile tournament/bracket	title/date/U-age duplicate/no bracket/dependent roster or fixture	only same-Club Coordinator mutates	transport/transaction	editor retains authoritative result; no partial invalid bracket state
Tournament squad	duplicate/blocked/cross-Club/inactive/empty publish	only same-Club Coach writes; unpublished data hidden from ordinary roles	transport/row-lock/DB	Draft remains or prior Published roster survives; candidate error/retry shown

Workflow	Validation failure	Auth/authorization	Network/database/external failure	UI/result
Match squad enforcement	blocked Player or missing/long exception reason	expanded candidates and objective write require same-Club Coordinator	transaction/constraint	editor stays open; no unapproved performance row or false roster membership
Dispute	summary/category/status invalid	role/club/scope	network/transaction	controller error, thread unchanged
Photo	missing/size/MIME/signature	same-Club Coordinator portal; same-Club Coach/Super Admin API	storage config/network/DB	no path update; visible error/fallback
Provisioning	duplicate/invalid role/link	coordinator/admin gate	Firebase/DB	compensation attempts Firebase cleanup
Notification	malformed inbox action/missing token/unknown event	every inbox query is current-user scoped; current profile + linked-child/privacy gates protect destinations	FCM invalid/unavailable, profile/inbox network error	business + inbox record remain; known safe route or focused-inbox fallback; device push may be absent

Authorization Trace

```
Firebase email/password or restored user
  -> Firebase ID token
  -> Authorization: Bearer
  -> Firebase Admin verify(check_revoked=True)
  -> decoded uid
  -> accounts_user(firebase_uid, active)
  -> reject non-admin without club
  -> request.user.role check
  -> queryset/object club/owner check
  -> GuardianLink check where relevant
  -> PIN unlock signature/user/player/10-minute check where relevant
  -> serializer/business validation
  -> ORM query/mutation
```

Access control is a combination of backend authentication, business/view authorization, and relational filters. Flutter role routing is UX only. There is no direct client database access and no Supabase RLS authorization layer. The backend prevents a user from pretending to be another role because the verified UID maps to the immutable-for-request local user role; client-supplied role/Club/actor fields are ignored, and each endpoint compares `request.user.role` with its explicit capability.

Match-performance authorization is role-separated. The Coordinator creates results and objective statistics using a same-Club match and Player, while the Coach can change only rating/notes on an existing row. Tournament ownership is also separated: the Coordinator defines/publishes the tournament and brackets; the Coach selects/publishes each bracket squad; the Coordinator may add only an eligible out-of-squad match exception with an auditable reason. None accepts writable ownership. Statistics reads separately permit Admin, same-Club Coach, Player self, or linked Guardian; Guardian access still passes the Player PIN unlock check when configured. Tournament and injury workflows apply the same principle: the URL ID is always combined with stored role, Club, relationship, state transition, eligibility policy, and sometimes a row lock.

Data Lifecycle Traces

1. User / Account

- **Create:** coordinator/admin form -> provisioning service -> Firebase identity where applicable + `accounts_user`.
- **Validate:** form/serializer, unique email/UID, allowed role, club rules.
- **Store:** Firebase identity and Django user; a Player is never a bare generic user-the dedicated aggregate also creates exactly one profile and optional same-Club guardian link. Seed commands repair the same Club/profile invariant for demo data.
- **Retrieve:** `/api/auth/me/`, portal/admin lists; token UID mapping.
- **Display:** role portal/profile/account lists.
- **Update:** admin role/status/account operations; password handled by Firebase for mobile or Django for web.
- **Delete/archive:** admin detail/delete behavior and Firebase linking require coordinated care; no soft-delete lifecycle was found.

2. Player Profile / Performance

- **Create:** trusted player provisioning creates `User` + one-to-one `PlayerProfile` + optional link.
- **Validate:** birth/tier/defaults and account rules; assessments 0-99 through serializer.
- **Store:** current fields in `academy_playerprofile`.
- **Retrieve:** squad/me/detail endpoints -> `Player`.
- **Display:** cards, dashboard, radar, notes, eligibility.
- **Update:** Coach assessment/position/mobile photo; School Staff eligibility; Coordinator portal photo; Super Admin photo.
- **Delete/archive:** player user deletion cascades profile/dependents; no assessment archive/history exists.

2A. Football Match / Historical Performance

- **Create:** Coordinator records a fixture-backed or ad-hoc completed match; Django stamps Club/creator. Coordinator then upserts objective statistics per same-Club Player; Coach separately rates an existing row.
- **Validate:** fixture state/date/venue/scores; statistic relationships, goalkeeper fields, rating, Club/Player role, team-goal/opponent-score consistency, and confirmed injury warning acknowledgement.
- **Store:** `academy_footballmatch` plus unique historical `academy_playermatchperformance` rows and audit events.

- **Retrieve:** Coordinator/Coach/Admin match roster endpoints; Player self/same-Club Coach/linked unlocked Guardian/Admin statistics endpoint with optional date/limit filter.
- **Display:** Coordinator results/statistics, Coach rating roster, and Player/Guardian/Coach totals, rating trend, and match history.
- **Update:** same-Club Coordinator corrects results/objective statistics; same-Club Coach saves/clears subjective rating/notes; providers refetch affected views.
- **Delete/archive:** Coordinator can delete a performance row with rated-row confirmation; Coach can clear only rating; deleting a match cascades its rows at model level, but the mobile API exposes no match-delete endpoint.

2B. Tournament Schedule / Fixture

- **Create:** Coordinator portal can upload one official document and create structured fixtures; the mobile Coordinator can create a Draft tournament and flexible U-age brackets in the authenticated Club.
- **Validate:** title/start date, unique U-age 3-25, optional bracket time, file size/MIME/signature, fixture date/time/venue/status, Club ownership, and current-roster eligibility after date/age changes.
- **Store:** opaque private object path in `academy_tournamentschedule`; bracket rows in `academy_tournamentagebracket`; fixtures in `academy_tournamentfixture`; file bytes in Supabase Storage or development/test fallback.
- **Retrieve:** portal Club query or role-aware Club-scoped mobile API; ordinary roles receive only Published tournaments and private documents use signed URLs.
- **Display:** portal management pages plus mobile Draft/Published tournament cards, bracket schedules/roster counts, and fixtures.
- **Update:** mobile Coordinator edits name/date/brackets and publishes; portal can replace the document or edit an uncompleted fixture; result entry links exactly one completed match.
- **Delete/archive:** a Draft bracket can be removed only without fixtures/roster entries; completed-history and Published-state guards prevent unsafe delete/edit.

2C. Tournament Squad / Match Exception

- **Create:** same-Club Coach saves one Draft squad per U-age bracket and unique selected Player entries; Coach publishes only after the parent tournament is Published.
- **Validate:** active same-Club Player role, unique IDs, valid event position, age/profile/injury eligibility, non-empty publication, and current tournament state.
- **Store:** `academy_tournamentsquad`, `academy_tournamentsquadentry`, publish metadata, verified actor, and audit rows; eligible match exceptions add reason/actor/time to the historical performance row.
- **Retrieve:** Coach-only candidate list; manager-visible Draft/Published squad; ordinary Club roles see only published tournament/roster data; match roster is filtered through published membership.
- **Display:** Coach search/selection/position editor, manager/read-only roster list, and Coordinator approved-exception chip on the match roster.
- **Update:** atomic entry reconciliation supports add/remove/position changes; publication reruns eligibility; Coordinator exception uses the existing objective-statistics save with a required reason.

- **Delete/archive:** removing a Draft selection deletes its entry; published squads cannot be emptied; there is no separate archive endpoint, and historical match exceptions remain attached to their performance evidence.

2D. Injury Report / Recovery Workflow

- **Create:** authorized care-team relationship submits a Pending report; Player/Guardian/Coach can later submit one Pending recovery update on a Confirmed injury.
- **Validate:** Player relationship/Club/PIN, dates, states, rejection reasons, and one-Pending-update rule.
- **Store:** `academy_injuryrecord`, `academy_injurystatusupdaterequest`, and audit rows.
- **Retrieve:** role/Club/link/unlock-scoped care-team lists; Coordinator can request archived records.
- **Display:** reporter history and Coordinator confirmation/recovery queue.
- **Update:** same-Club Coordinator confirms/rejects reports, approves/rejects status requests, or edits confirmed records.
- **Delete/archive:** Pending reporter/Coordinator can withdraw; only Confirmed Recovered report can be archived by same-Club Coordinator.

3. Training Session

- **Create:** coach form/API -> session insert with server club/creator.
- **Validate:** required fields/choices/tiers/date plus paired 12-hour start/end values with start strictly before end, enforced by serializer and model.
- **Store:** `academy_trainingsession`.
- **Retrieve:** club-scoped training-session GET -> `TrainingSession` list.
- **Display:** schedule cards/tabs and player confirmation.
- **Update:** same-club coach PUT.
- **Delete/archive:** coach DELETE; confirmations cascade; attendance survives with null session; no soft archive.

4. Attendance

- **Create:** coach finalizes complete batch; upsert unique player/session; offline network failure queues locally.
- **Validate:** UI marks/window plus server role/club/date/serializer.
- **Store:** `academy_attendance`; temporary local `outbox_attendance` if unsent.
- **Retrieve:** session/player endpoints and progress aggregate.
- **Display:** chips, histories, streaks, percentages, effort cards.

- **Update:** re-finalizing batch updates same unique rows and prunes omissions.
- **Delete/archive:** omitted rows deleted; session deletion preserves rows with null session; successful sync deletes outbox copy.

5. Eligibility

- **Create:** first status is part of player profile; every real change creates history.
- **Validate:** portal choice, School Staff role, same Club, and Club type supports academic eligibility.
- **Store:** current `academy_playerprofile.eligibility` + append-only `academy_eligibilityhistory`.
- **Retrieve:** player profile and history endpoint.
- **Display:** badge/history cards for school-affiliated Clubs; explicit N/A/hidden academic status for independent Clubs.
- **Update:** school-staff portal save, signals/audit/FCM.
- **Delete/archive:** player deletion cascades history; no normal history edit/delete UI.

6. Notification

- **Create:** committed business event -> `notify_*` -> `_send_to_users()` -> one `NotificationRecord` for each active recipient, followed by optional FCM fanout.
- **Validate/scope:** event payload is server-created; recipient IDs come from Club/player/guardian relationships rather than a client-selected inbox owner.
- **Store:** authoritative history/read state in `academy_notificationrecord`; optional delivery endpoints in `academy_devicetoken`.
- **Retrieve:** current-user list and unread-count endpoints -> `AppNotification` list/count providers.
- **Display:** foreground SnackBar, badged bell, and retryable inbox; known opened pushes/rows route to the role-appropriate schedule, profile, or eligibility destination, while unknown events remain in the inbox.
- **Update:** current-user mark-one/mark-all endpoints set `read_at`; cross-user IDs are not addressable.
- **Delete/archive:** deleting a user cascades their records; there is no normal inbox delete action.

Scouting Report Lifecycle

`NOT IMPLEMENTED IN CURRENT REPOSITORY` at every lifecycle stage.

Top 31 Important Function Call Chains

1. **Startup:** `main()` -> `Firebase.initializeApp()` -> `runApp(ProviderScope)` -> `_FootPathAppState.initState()` messaging listeners -> `FootPathApp.build()` -> `SessionBootstrapScreen`.

2. **Restore:** `initState()` -> `_restore()` -> `RestoreSession.call()` -> `FirebaseAuthRepository.restoreSession()` -> `authStateChanges().first` -> `/api/auth/me/` -> `UserProfile.fromJson()` -> `HomeScreen`.
3. **Login:** `_handleSignIn()` -> `LoginController.signIn()` -> `SignIn.call()` -> `signInAndFetchProfile()` -> Firebase sign-in -> `/api/auth/me/` -> role portal.
4. **Logout:** `_signOut()` -> `UnregisterDevice.call()` -> current FCM token -> scoped `DELETE /api/devices/` (best-effort) -> `SignOut.call()` -> Firebase sign-out + current-owner GET-cache clear -> clear unlocks -> `LoginScreen`.
5. **Role authorization:** API call -> Bearer token -> `FirebaseAuthentication.authenticate()` -> local `User` -> view role check -> club/object/link check -> response/403.
6. **Player self profile:** `PlayerDashboardScreen` -> `myProfileProvider` -> `GetMyProfile.call()` -> `fetchMyProfile()` -> `MyProfileView.get()` -> `PlayerSerializer` -> `Player.fromJson()`.
7. **Guardian child profile:** select child -> PIN verify controller -> API verify -> `verify_pin()` -> `issue_player_unlock()` -> token store -> `fetchPlayerDetails()` with header -> guarded detail view -> `Player.fromJson()`.
8. **Squad list:** coach screen -> `squadProvider` -> `GetSquad.call()` -> `fetchSquad()` -> `SquadListView.get()` -> club query -> `List<Player>`.
9. **Position:** picker -> `PlayerPositionController` -> `SavePlayerPosition.call()` -> `savePosition()` -> `PlayerPositionView.put()` -> serializer save/audit -> `Player.fromJson()` -> invalidate.
10. **Assessment:** save button -> `_save()` -> `EditPerformanceController.submit()` -> `SavePlayerAssessment.call()` -> `saveAssessment()` -> `PlayerAssessmentView.put()` -> profile/audit/on-commit inbox + FCM -> `Player.fromJson()`.
11. **Create session:** add -> form `_submit()` -> `ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()` -> `createSession()` -> `TrainingSessionListCreateView.post()` -> session/audit/on-commit inbox + FCM -> refresh.
12. **Edit session:** form `_submit()` -> controller `.saveChanges()` -> `UpdateTrainingSession.call()` -> repository `updateSession()` -> `TrainingSessionDetailView.put()` -> refresh.
13. **Cancel session:** `_cancelSession()` -> controller `.cancel()` -> `CancelTrainingSession.call()` -> repository `deleteSession()` -> `TrainingSessionDetailView.delete()` -> snapshot recipients/audit/delete -> on-commit inbox + FCM -> refresh.
14. **Attendance online:** `_finalize()` -> `AttendanceLogController.save()` -> `LogSessionAttendance.call()` -> offline decorator -> API repository POST -> `SessionAttendanceView.post()` -> atomic upsert/prune -> decode/invalidate.
15. **Attendance offline replay:** API transport exception -> decorator -> `AttendanceOutbox.enqueue()` -> `AttendanceSyncService` owner query -> API repository retry -> backend save -> outbox delete.
16. **Squad progress:** progress screen -> `squadProgressProvider` -> `GetSquadProgress.call()` -> API progress repository -> `SquadProgressView.get()` -> Coach Club filter or Super Admin all-club query -> ORM Count/Avg -> `PlayerProgress.fromJson()`.
17. **Eligibility change:** staff form -> `staff_eligibility()` -> `set_player_eligibility()` -> `PlayerProfile.save()` -> pre/post signals -> history/audit -> on-commit inbox + FCM.
18. **Injury report/review:** form -> `InjuryFormController.submit()` -> `SaveInjury` -> API POST -> `InjuryWorkflowListCreateView.post()` -> Pending row/audit -> Coordinator `.reviewReport()` -> locked Confirm/Reject transition -> invalidate.

19. **Dispute response:** response action -> `DisputeFormController.respond()` -> `RespondToDispute.call()` -> API POST responses -> `DisputeResponseCreateView.post()` -> response/status transaction -> `Dispute.fromJson()`.
20. **Notification delivery/open:** mutation commit -> `NotificationRecord` persistence + best-effort FCM -> foreground/opened handler, bell, or inbox row -> notification providers/API + `NotificationNavigationController` -> current-profile role mapping -> protected schedule/profile/eligibility destination or focused inbox fallback. Device registration supplies the optional FCM delivery token.
21. **Publish tournament:** Coordinator portal form -> `tournament_schedules()` -> document validation -> schedule row -> private storage upload -> opaque path/audit -> nested mobile API -> typed schedule cards.
22. **Create match:** Coordinator fixture/result action -> match form -> `MatchManagementController.create()` -> `CreateFootballMatch` -> API POST -> role/Club/fixture lock -> server Club/creator + optional fixture completion/audit -> `FootballMatch.fromJson()` -> refresh.
23. **Save objective statistics:** Coordinator roster Player -> performance editor -> controller -> `SaveMatchPerformance` -> API PUT -> `_role_match(COORDINATOR)` + same-Club Player + injury gate -> atomic validation/upsert/audit -> `MatchPerformance.fromJson()` -> invalidate.
24. **Save Coach rating:** Coach roster -> rating editor -> controller/use case -> rating API PUT -> `_role_match(COACH)` -> locked existing row -> rating/notes/rater/time + audit -> refresh.
25. **Read match trends:** Player, linked unlocked Guardian, or Coach route -> `playerMatchStatisticsProvider` -> API GET -> `_may_read_match_statistics()` + PIN gate -> scoped query -> derived summary -> typed chart/history.
26. **Recovery request:** care-team action -> controller `.requestStatus()` -> status-update API -> Confirmed injury/relationship/one-Pending validation -> request row/audit -> Coordinator review -> atomic injury transition.
27. **Live database selection:** Django settings load -> `DB_HOST` present -> PostgreSQL/Supabase connection; DEBUG/testing -> SQLite; non-test production without `DB_HOST` -> `ImproperlyConfigured` before serving traffic.
28. **Mobile tournament/bracket management:** Coordinator Schedule tab -> `EditTournamentScreen` -> `TournamentManagementController` -> `ApiTournamentScheduleRepository` -> schedule/bracket DRF endpoint -> same-Club serializer/model/audit -> nested schedule -> provider invalidation.
29. **Coach tournament squad:** bracket row -> `TournamentSquadScreen` + candidates provider -> `TournamentRosterManagementController.save/publish` -> roster repository -> Coach-only endpoint -> eligibility policy + atomic entry reconciliation/publish -> typed squad -> refresh.
30. **Match squad filtering:** `MatchRosterScreen` -> `matchRosterProvider` -> `GetMatchRoster` -> API roster GET -> `_match_age_bracket()` -> published squad + eligibility + injury + performance join -> membership-aware rows -> roster cards.
31. **Eligible squad exception:** Coordinator add-exception action -> expanded candidates provider -> reason modal -> performance editor -> `savePerformance` -> API PUT -> eligibility recompute + locked membership check -> reason/actor/time + audit/performance save -> approved-exception UI.

Code-Tracing Defense Questions (50)

T1. What happens after the user presses Login?

`_handleSignIn` reads controllers, calls `LoginController.signIn`, then `SignIn` and `FirebaseAuthRepository`; Firebase authenticates, Django `/api/auth/me/` authorizes, JSON becomes `UserProfile`, and `HomeScreen` routes by role. Failure updates controller error.

T2. Which file handles authentication?

Client orchestration is `firebase_auth_repository.dart`; trusted API authentication is `backend/accounts/authentication.py`. The former obtains tokens; the latter verifies them and maps a local user.

T3. Where is the returned user session stored?

Firebase Auth's SDK maintains the mobile identity session. The Django profile is a `UserProfile` passed through bootstrap/login routes, not a custom database token stored by Flutter. Guardian unlock tokens are separate and memory-only.

T4. How is the user role obtained?

Django loads `accounts_user` by verified Firebase UID, `UserSerializer` returns its role, and `UserProfile.fromJson` parses it. Backend views re-read `request.user.role` for every authorization decision.

T5. How does the application decide which screen to show?

`HomeScreen.build()` switches on `UserProfile.role`: Coordinator, Coach, Player, and Guardian enter their dedicated portals; other authenticated roles use their supported web/admin surface or fallback.

T6. Trace a scouting report from UI to database.

`NOT IMPLEMENTED IN CURRENT REPOSITORY`: no role, screen, controller, endpoint, model, migration, or table. The dispute flow is distinct and must not be relabeled.

T7. Trace attendance from button click to database.

Finalize -> `_finalize` builds records -> attendance controller/use case -> offline decorator -> API repository POST -> `SessionAttendanceView.post` validates coach/club/date/players -> atomic `update_or_create`/prune in `academy_attendance` -> serialized records -> Dart models/provider refresh. Network-only failure queues locally.

T8. How is player performance retrieved?

Player/squad provider -> get-player use case -> `ApiPlayerRepository` -> player endpoint -> `PlayerProfile` query/`PlayerSerializer` -> `Player.fromJson`/`PlayerRatings.fromJson` -> Riverpod data -> card/radar.

T9. Where does JSON become a Dart object?

At entity factories such as `UserProfile.fromJson`, `Player.fromJson`, `TrainingSession.fromJson`, `Attendance.fromJson`, `PlayerProgress.fromJson`, and others, called by concrete API repositories.

T10. How does database data reach a Flutter widget?

ORM queryset/model -> DRF serializer -> HTTP JSON -> API repository decode/entity factory -> use case future -> Riverpod `FutureProvider` `AsyncData` -> `ConsumerWidget` rebuild.

T11. What happens if the database request fails?

Django returns a non-success/connection closes; repository throws a domain-specific exception; controller or `FutureProvider` becomes error and UI shows retry/error. Only a network-classified attendance write is queued optimistically.

T12. Why is the login function asynchronous?

Firebase credential validation, token creation, and HTTP profile lookup complete later. Await keeps them ordered and lets loading/error state bracket the whole operation without blocking UI rendering.

T13. What data is passed between assessment functions?

Selected `Player.id`, a `PlayerRatings` object, and `coachNotes`; repository transforms them into nested ratings JSON. Coach/club/actor are derived server-side, not passed as trusted values.

T14. Why does `EditPerformanceController` exist?

It owns mutation loading/error, calls the use case, invalidates squad state, and keeps widgets independent of HTTP/Firebase mechanics.

T15. Why doesn't the UI query the database directly?

It would expose infrastructure/policy, duplicate authorization, and bypass Django transactions/audit. Repository/API and Django boundaries preserve security and replaceability.

T16. Where is validation performed?

Convenience checks occur in Flutter forms; authoritative checks occur in DRF serializers/views, Django forms/services, and some model/database constraints.

T17. Where is authorization performed?

Primarily in Django: Firebase authentication class, permission classes, role comparisons, club-filtered querysets, ownership, GuardianLink, and signed PIN unlock. UI routing is not trusted.

T18. How does the backend know which user made the request?

It verifies the Firebase Bearer token, extracts UID, finds the active `accounts_user` with that `firebase_uid`, and sets `request.user`.

T19. What happens when the user logs out?

The screen first calls `UnregisterDevice`: it obtains the current FCM token and sends a current-user-scoped `DELETE /api/devices/`; absence, timeout, or HTTP failure is logged and cannot block logout. `SignOut` then calls `FirebaseAuthRepository._signOutAndClearCache()`: it captures the owner UID, signs out of Firebase, clears that owner's durable API GET cache, clears guardian/player unlock memory, and replaces the authenticated portal with login. Only the matching device-token row is deleted; no training/player domain row is removed.

T20. How are sessions restored after app restart?

`SessionBootstrapScreen.initState` calls restore; repository waits for Firebase auth state, gets a fresh ID token, and calls `/api/auth/me/`. Only a valid local profile reaches `HomeScreen`.

T21. How does offline attendance reach the database later?

Outbox stores owner/session/record JSON. Sync loads current-owner rows oldest first, converts JSON back to `Attendance`, calls the same live repository, and removes only after server success.

T22. What stops offline data crossing accounts?

Each outbox row has `owner_uid`; current sync requests rows for only the active Firebase UID. The general GET cache uses the same owner UID in its composite key and is cleared for that owner on sign-out; unlock-protected reads are excluded entirely.

T23. What happens if the attendance API returns 403 while offline logic is enabled?

It becomes a general attendance repository exception, not `AttendanceNetworkException`, so it is not queued. The controller shows failure.

T24. How does an assessment return to the exact screen?

Backend returns updated player JSON; repository creates `Player`; controller returns it to `_save`; screen `Navigator.pop(saved)` supplies it to the previous route and invalidates the squad for a refetch.

T25. How is the guardian unlock token produced and consumed?

Successful transactional PIN verification calls `issue_player_unlock(user,player)`; Flutter stores returned token under player ID; detail repositories add `X-Player-Unlock`; backend verifies signature, age, binding, and link.

T26. What is stored in the PIN table?

Player one-to-one key, `pin_hash`, failed-attempt count, lock-until timestamp, and update time-not plaintext PIN and not guardian unlock tokens.

T27. How does eligibility update reach mobile?

Staff portal saves current profile; signals add history/audit and send FCM on commit. Mobile does not receive a state stream; its provider refetches profile/history on open/refresh/invalidation conditions.

T28. How is progress generated?

One endpoint groups attendance by player and annotates status counts and average effort, combines it with player profiles, returns JSON, and Flutter maps to `PlayerProgress` cards.

T29. How does file upload avoid exposing a Supabase service key?

Browser or Flutter sends multipart to Django; the Flutter path uses `AuthenticatedApiClient.postMultipart()` and never receives the storage credential. Django validates bytes, calls Supabase REST using server environment credentials, stores only the object path, and returns signed read URLs.

T30. What happens when a session is canceled?

Coach confirms UI -> delete use case/repository -> same-club `TrainingSessionDetailView.delete` -> snapshot recipients and audit -> model delete -> register on-commit cancellation inbox/FCM -> session list invalidation. Attendance session FK becomes null; confirmations cascade. If deletion fails, the on-commit callback is never registered/executed as a successful cancellation.

T31. Why can the player ID be trusted in a URL?

It is not trusted by itself. Django combines it with verified `request.user`, role, club/ownership/link, and optional unlock checks before querying/returning data.

T32. Where is a new account's club assigned?

Coordinator portal passes the authenticated Coordinator's Club into `create_club_account`/provisioning. Generic Admin user creation requires an active selected Club and excludes `PLAYER`; the dedicated Admin-player path derives the Player Club from the required active same-Club guardian and calls `provision_player`. Seed commands also repair every non-Admin seed's Club and ensure each seeded Player has exactly one profile.

T33. How is a duplicate RSVP prevented?

Backend uses `update_or_create` on the database-unique `(player,session)` pair. A new response updates rather than stacks another row.

T34. What happens while a `FutureProvider` is waiting?

Riverpod exposes loading; the `ConsumerWidget` renders a progress state. On resolution it stores data/error and rebuilds only consumers of that provider.

T35. Where is notification receiving traced?

Backend event helpers persist current-user `NotificationRecord` rows before optional FCM. Flutter traces `onMessage` to a foreground SnackBar and provider invalidation, and traces its **View** action plus `onMessageOpenedApp`, `getInitialMessage()`, and inbox-row taps through `NotificationNavigationController`. The controller re-resolves `/api/auth/me/`, suppresses duplicate active opens, marks the matching row read best-effort, and maps session events to Schedule, assessments to the Player/authorized Guardian child Profile, and eligibility changes to the protected eligibility history; unknown/unsafe requests fall back to the inbox. Local tests verify the inbox/router policy, but signed physical-device FCM/APNs delivery remains environment-dependent and unverified.

T36. Who enters match-performance data?

The Coordinator records the completed result and objective Player statistics. The Coach can add only the subjective rating and notes after statistics exist. Django verifies each role independently, derives Club/actor from the token-mapped local user, and restricts the match and Player to that Club. This separation prevents one role from controlling both evidence and evaluation.

T37. How are impossible match statistics rejected?

The write serializer and model check field ranges and relationships such as goals <= shots on target <= shots and completed passes <= attempted passes. Goalkeeper-only fields require `GK`; clean sheet conflicts with goals conceded; the transactional view also rejects combined Player goals above the team score and goalkeeper concessions above the opponent score. Database constraints provide a final guard for key numeric relationships and duplicate match/player rows.

T38. How can a Player see trends without reading another Player's data?

Flutter sends the selected/current Player ID, but Django does not trust it. `_may_read_match_statistics()` compares a Player URL ID with `request.user.id`, checks same-Club scope for a Coach, permits Admin, or requires an active Guardian link. The endpoint also enforces PIN unlock when configured before querying and building the summary.

T39. How do Flutter and Django communicate?

Flutter's API repository calls `AuthenticatedApiClient`, which sends HTTPS REST requests using JSON or multipart form data. It attaches a Firebase ID token as a Bearer header. Django REST Framework maps the URL to an APIView, verifies the token with Firebase Admin, authorizes the local role/Club/object, validates with serializers, uses the ORM, and returns JSON that Dart converts with `fromJson()`.

T40. Why does Flutter not connect directly to Supabase/PostgreSQL?

Direct access would expose credentials and distribute authorization logic into a modifiable client. Django is the trusted boundary: it owns service keys, roles, Club tenancy, privacy rules, validation, transactions, audits, and signed-file URLs. Flutter therefore receives only the data each verified user may see.

T41. What API is used between the applications?

The project uses a REST-style HTTP API built with Django REST Framework and Dart's `http` package. Payloads are normally JSON; photos use multipart form data. Authentication is Firebase Bearer-token based for mobile API requests, while the Django portal uses session cookies plus CSRF protection.

T42. What is the purpose of Riverpod and Clean Architecture?

Screens watch Riverpod providers/controllers; controllers call domain use cases; use cases depend on repository interfaces; concrete API/Firebase/mock/local repositories live in the data layer. This keeps HTTP/Firebase details out of UI/business entities, makes async state consistent, and lets tests replace infrastructure without changing screens.

T43. How is a tournament document kept private?

The Coordinator uploads it to Django, which validates size, declared type, and signature, then uploads server-side to private Supabase Storage. Django stores only an opaque path. An authorized schedule GET creates a short-lived signed URL; neither Flutter nor the browser receives the service key.

T44. Why is injury confirmation a separate workflow?

An initial report may come from several care-team roles, so it begins Pending rather than immediately changing the authoritative injury state. A same-Club Coordinator confirms/rejects it. Recovery updates also require Coordinator approval and an audit trail, reducing accidental or unilateral status changes.

T45. How do we know normal builds use live shared data?

Flutter defaults to Firebase plus `Api*Repository`; mocks require an explicit non-release `USE_MOCK=true` or the Flutter test runtime. Django uses PostgreSQL when `DB_HOST` is configured and refuses non-test production startup without it. SQLite is limited to DEBUG/testing, preventing silent local persistence in production.

T46. Who controls each part of the tournament workflow?

The Coordinator owns the tournament identity, start date, U-age brackets, bracket schedule times, and tournament publication. The Coach owns Player selection, event positions, and squad publication for each bracket. During a bracket-linked match, the Coordinator owns objective statistics and may add only an eligible out-of-squad exception with a required audited reason. Django uses separate role-gated endpoints for these capabilities.

T47. How is a Player's tournament eligibility calculated?

`roster_eligibility()` loads the stored profile and bracket. Missing profile/DOB, an overage birth year, or a confirmed Active/Recovering injury is `BLOCKED`; a Pending injury is `WARNING`; otherwise the Player is `ELIGIBLE`. Candidate output is privacy-safe, and Django reruns the rule on squad save, squad publish, tournament/bracket edits, match roster reads, and performance writes.

T48. What prevents duplicate or cross-Club squad members?

The roster serializer rejects duplicate submitted IDs; the view queries only active `PLAYER` accounts from `request.user.club_id`; model validation rechecks Club/role; and the database has a unique `(squad,player)` constraint. The squad itself is one-to-one with its bracket, and save reconciliation runs atomically under a row lock.

T49. Why allow a tournament squad exception?

It handles legitimate late replacement or operational decisions without silently weakening eligibility. Only the Coordinator can request expanded candidates, hard-blocked Players remain forbidden, and an eligible out-of-squad Player requires a 1-500 character reason. Django stores the verified actor/time and audit event with the historical performance, making the exception visible and accountable.

T50. Can Flutter bypass tournament or squad rules?

No. Hidden buttons and disabled candidates improve UX, but Django verifies the Firebase-mapped role and Club, resolves the bracket and published squad, recomputes profile/age/injury eligibility, locks affected rows, validates the exception reason/statistics, and applies model/database constraints. A modified client receives 400/403/404 rather than an unauthorized write.

Memorization Cards - Eighteen Critical Workflows

Card 1: Login

What I click

Login button.

First function called

`LoginScreen._handleSignIn()`.

Next function

`LoginController.signIn()` -> `SignIn.call()`.

Service used

`FirebaseAuthRepository.signInAndFetchProfile()`.

Database/API operation

Firebase email/password; then Bearer `GET /api/auth/me/`; `accounts_user` UID/active/club lookup.

Data returned

Serialized local user profile.

State change

`LoginState` loading/error; profile returned; device/sync start.

Screen result

`HomeScreen` and role portal.

20-second defense explanation

Firebase authenticates the credentials, but Django then verifies the token and returns the active local role/club profile. Flutter parses it and routes to the correct portal.

60-second technical explanation

The button reads trimmed email and raw password into `LoginController`. It sets loading, calls `SignIn`, then `FirebaseAuthRepository`, which signs in through Firebase, obtains an ID token, and calls `/api/auth/me/`. Django verifies revocation, maps UID to an active user, enforces club assignment, and serializes the profile. Dart converts it to `UserProfile`; login starts device registration/sync and replaces the route. Any Firebase/API exception becomes controller error text.

Card 2: Session Restoration

What I click

Nothing; app startup triggers it.

First function called

`SessionBootstrapScreen.initState()` -> `_restore()`.

Next function

`RestoreSession.call()`.

Service used

`FirebaseAuthRepository.restoreSession()`.

Database/API operation

Firebase auth-state read + `GET /api/auth/me/` local user query.

Data returned

`UserProfile?`.

State change

Bootstrap profile/completed/error local state.

Screen result

Home, login, or restore error.

20-second defense explanation

The app never trusts a cached role. It restores Firebase identity, obtains a token, and rechecks the Django account before showing an authenticated portal.

60-second technical explanation

`initState` calls `_restore`, which invokes the restore use case. The repository awaits the first Firebase auth state. No user yields login. A user yields a fresh ID token and `/api/auth/me/`; Django checks revocation, active local UID mapping, and club. JSON becomes `UserProfile`. The screen sets local state, starts device registration/attendance sync, and builds `HomeScreen`. A 401/403 signs out; recoverable network failure shows retry.

Card 3: Player Profile Retrieval

What I click

Open player dashboard/profile.

First function called

Widget watches `myProfileProvider` or related player provider.

Next function

`GetMyProfile.call()` / `GetPlayerDetails.call()`.

Service used

`ApiPlayerRepository.fetchMyProfile()` / `fetchPlayerDetails()`.

Database/API operation

GET player endpoint; read `academy_playerprofile` joined to `accounts_user`.

Data returned

`PlayerSerializer` JSON.

State change

`FutureProvider` loading -> data/error.

Screen result

Player card, ratings, notes, eligibility, photo.

20-second defense explanation

The provider calls a player use case and authenticated API repository. Django applies role/object rules, serializes the current profile, and Riverpod rebuilds the dashboard from a typed `Player`.

60-second technical explanation

The player dashboard watches `myProfileProvider`, which invokes `GetMyProfile` and the API repository. The shared client sends a Bearer GET to `/api/players/me/`; Django requires player role and queries the profile related to `request.user`. Guardians use the detail route plus unlock header, which disables persistent caching for that request. `PlayerSerializer` returns nested ratings and optional photo URL; `Player.fromJson` creates enums/nested `PlayerRatings`; the provider changes from loading to data and the consumer renders cards. A pre-response network failure may use the current owner's cached successful GET; HTTP/auth/decode errors become `AsyncError`.

Card 4: Guardian Unlock

What I click

Select child and submit PIN.

First function called

Privacy gate verification callback/controller.

Next function

`VerifyPlayerPrivacyPin.call()`.

Service used

`ApiPlayerPrivacyPinRepository.verifyPin()`.

Database/API operation

POST PIN verify; row-locked PIN hash/failure state + GuardianLink read; signed token issuance.

Data returned

`verified` and `unlockToken`.

State change

Token stored in memory for player ID; unlocked provider changes.

Screen result

Full child content fetch/render.

20-second defense explanation

A valid guardian login and link are not enough for private detail. The server verifies the hashed household PIN, throttles guesses, and issues a ten-minute token bound to guardian and child.

60-second technical explanation

The redacted selector supplies player ID. The gate posts the PIN with Firebase auth. Django verifies the current GuardianLink, locks the PIN row, checks lock time and hash, updates failures, then signs a user/player payload. Flutter keeps it only in memory and adds it as `X-Player-Unlock` to detail/history requests. The server rechecks signature age/binding and the current link. Five failures lock for 15 minutes; a different player ID or expired token fails.

Card 5: Create Training Session

What I click

Add session, then Save.

First function called

`ScheduleSessionScreen._submit()`.

Next function

`ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()`.

Service used

`ApiTrainingRepository.createSession()`.

Database/API operation

`POST /api/training-sessions/`; insert `academy_trainingsession` with server club/creator.

Data returned

Serialized `TrainingSession`.

State change

Controller loading/data/error; session provider refreshed.

Screen result

Form pops and session card appears.

20-second defense explanation

The form builds a typed session, the controller/use case posts it, and Django validates coach role then assigns the trusted club and creator before saving and returning it.

60-second technical explanation

`_submit` validates required fields and tiers, constructs `TrainingSession`, and calls the async controller. The use case delegates to `ApiTrainingRepository`, which sends `toJson` with Firebase Bearer token. `TrainingSessionListCreateView.post` requires coach, validates the serializer, saves with `created_by=request.user` and `club=request.user.club`, audits, and schedules FCM on commit. Response becomes `TrainingSession.fromJson`; state resolves and the route pops/refetches. Errors keep the form and show a SnackBar.

Card 6: Attendance Online/Offline

What I click

Finalize Attendance.

First function called

`LogAttendanceScreen._finalize()`.

Next function

`AttendanceLogController.save()` -> `LogSessionAttendance.call()`.

Service used

`OfflineFirstAttendanceRepository`, then `ApiAttendanceRepository` or `AttendanceOutbox`.

Database/API operation

POST complete batch -> atomic attendance upsert/prune; network failure -> local outbox insert.

Data returned

Saved records or optimistic queued records.

State change

Controller resolves; provider invalidates; queue sync state persists when offline.

Screen result

Success/queued pop; other errors remain visible.

20-second defense explanation

The coach submits the complete marked roster. Django validates and replaces the session attendance atomically. A network-only failure stores the owner-scoped batch locally and replays it later in order.

60-second technical explanation

`_finalize` validates the time window/unmarked players and builds typed records, clearing effort/note for non-present. Controller/use case calls an offline decorator that tries the API. Django checks coach, same-club session, 0-2-day window, record schema, and every player club; a transaction upserts submitted rows and deletes omissions. JSON returns to `Attendance.fromJson` and providers refresh. If transport fails, the decorator serializes the full batch with Firebase UID in sqflite. Sync replays oldest first, deletes success, and stops with retry metadata on failure.

Card 7: Assessment Save

What I click

Save on performance editor.

First function called

`EditPerformanceDataScreen._save()`.

Next function

`EditPerformanceController.submit()` -> `SavePlayerAssessment.call()`.

Service used

`ApiPlayerRepository.saveAssessment()`.

Database/API operation

PUT assessment; update 12 rating columns and notes in `academy_playerprofile`; audit/push.

Data returned

Updated `Player` JSON.

State change

Controller returns `Player`; squad invalidated.

Screen result

Editor pops; updated rating/card appears.

20-second defense explanation

The coach sends 12 ratings and notes; Django verifies same-club coach access, validates 0-99, updates the current profile, audits, and returns a typed player.

60-second technical explanation

The form builds `PlayerRatings` and notes, controller enters loading, and `SavePlayerAssessment` delegates to the API repository. The PUT carries nested ratings and no trusted actor/club. Django's view requires coach, compares club IDs, validates/ flattens through `AssessmentSerializer`, saves the profile, records audit, and schedules notification after commit. `PlayerSerializer` response becomes `Player.fromJson`; Riverpod invalidates squad and the editor returns the saved object. Current data overwrites; no assessment-history row exists.

Card 8: Performance Dashboard

What I click

Coach Progress tab.

First function called

`CoachProgressScreen` watches `squadProgressProvider`.

Next function

`GetSquadProgress.call()`.

Service used

`ApiProgressRepository.fetchSquadProgress()`.

Database/API operation

GET progress; ORM groups attendance and computes Count/Avg.

Data returned

List of `PlayerProgress` maps.

State change

FutureProvider loading/data/error.

Screen result

Squad summary and per-player cards.

20-second defense explanation

Opening the tab triggers one authenticated aggregate request. Django computes attendance counts and average effort per club player, and Flutter converts the list into progress cards.

60-second technical explanation

`CoachProgressScreen` consumes `squadProgressProvider`. The get-progress use case calls its API adapter through `AuthenticatedApiClient`. `SquadProgressView` checks Coach/Super Admin, Club-filters both profile and attendance querysets only for a Coach, and leaves them all-club for Super Admin before the filtered `Count/Avg` group-by. It shapes a JSON list with zero/null defaults. `PlayerProgress.fromJson` converts each row; Riverpod resolves and rebuilds summary/card widgets. This is deterministic analytics, not AI.

Card 9: Eligibility Update and Read

What I click

Staff submits eligibility; player/guardian opens history.

First function called

Portal `staff_eligibility()`; read screen watches `eligibilityHistoryProvider`.

Next function

`set_player_eligibility()`; read `GetEligibilityHistory.call()`.

Service used

Portal service/signals; `ApiEligibilityHistoryRepository` for read.

Database/API operation

Update profile, insert eligibility history/audit; later GET history.

Data returned

Portal success and serialized `EligibilityChange` list.

State change

Server current/history state; mobile FutureProvider refetch.

Screen result

Updated badge/history cards.

20-second defense explanation

School staff changes a same-club eligibility status. Signals preserve old/new history and audit after the profile save; authorized player or unlocked guardian later retrieves the history.

60-second technical explanation

The CSRF/session-protected staff form validates a club player and status, then calls `set_player_eligibility`. Saving `PlayerProfile` triggers `pre_save` to stash the prior value and `post_save` to insert `EligibilityHistory`, audit, and on-commit FCM only when changed. Mobile history uses a player-keyed Riverpod provider and authenticated endpoint; Django checks role/link/unlock, serializes rows, and Dart maps them to `EligibilityChange` cards. Grades are never stored.

Card 10: Injury Report and Confirmation

What I click

Report Injury, then the Coordinator opens the review queue and confirms/rejects it.

First function called

Injury form `_save()`, followed by the Coordinator review action.

Next function

`InjuryFormController.submit()`, then `.reviewReport()`.

Service used

`ApiInjuryRepository`.

Database/API operation

`POST /api/injuries/`, then `POST /api/injuries/{id}/review/`; insert Pending report, then locked Coordinator decision and audit.

Data returned

Serialized `InjuryRecord`.

State change

Controller loading/data/error; Player and Club injury providers invalidated.

Screen result

Reporter history and Coordinator queue refresh with Pending/Confirmed/Rejected state.

20-second defense explanation

An authorized care-team member submits a relationship-scoped Pending report. Only the same-Club Coordinator can confirm/reject it, and every decision is audited before Flutter refetches authoritative state.

60-second technical explanation

The form creates an `InjuryRecord` draft and calls the controller/use case/API repository. Django resolves the affected Player from self, Guardian link/PIN, or same-Club Coach/Coordinator scope and forces Active/Pending state. The Coordinator's review endpoint locks that same row, rechecks Club and Pending state, requires a rejection reason when needed, stamps reviewer/time, and records an audit action. JSON returns through `InjuryRecord.fromJson`; invalidated Player/Club providers refetch. Recovery changes follow the same request-then-Coordinator-review rule.

Card 11: Record Match and Objective Player Statistics

What I click

Coordinator Matches -> fixture/ad-hoc result -> match roster -> Player -> Save statistics.

First function called

`EditFootballMatchScreen._submit()`, then `EditMatchPerformanceScreen._save()` for the Player row.

Next function

`MatchManagementController.create()` / `.savePerformance()` -> corresponding use case.

Service used

`ApiMatchRepository` through `AuthenticatedApiClient`.

Database/API operation

`POST /api/matches/`, then `PUT /api/matches/{matchId}/performances/{playerId}/`; insert Club-owned match and atomic unique Player performance.

Data returned

Typed `FootballMatch`, then typed `MatchPerformance` with nested match data.

State change

Controller loading/data/error; match list, roster, and affected Player statistics providers invalidated.

Screen result

New result and recorded roster row; trends become available to authorized Player/Guardian/Coach readers.

20-second defense explanation

The Coordinator records the completed result and objective statistics. Django derives Club/actor from the verified Coordinator, validates fixture/Player/statistic consistency, saves atomically, audits, and returns typed data for refresh.

60-second technical explanation

The match form posts through controller/use case/repository. Django accepts only a Coordinator, stamps Club/creator, and optionally locks/links a same-Club tournament fixture. Selecting a Player sends match/Player IDs plus objective statistics. Django re-resolves both inside the Coordinator's Club, checks confirmed injuries, locks the match/existing row, validates relationships and team totals, then inserts/replaces the unique historical row with server-owned recorder. Coach rating is deliberately handled by a separate endpoint.

Card 12: View Match Statistics and Trends

What I click

Player Progress -> Match Statistics; linked Guardian protected child view; or Coach -> Player trends.

First function called

`PlayerMatchStatisticsView.build()` watches `playerMatchStatisticsProvider(playerId)`.

Next function

`GetPlayerMatchStatistics.call()`.

Service used

`ApiMatchRepository.fetchPlayerStatistics()`.

Database/API operation

`GET /api/players/{playerId}/match-statistics/`; authorized historical query plus derived summary.

Data returned

`PlayerMatchStatistics` containing `MatchPerformanceSummary` and chronological history rows.

State change

Family FutureProvider loading -> data/error; pull-to-refresh refetches.

Screen result

Season summary tiles, Last 5/10/All rating chart, and expandable match history.

20-second defense explanation

Django permits only Player self, same-Club Coach, linked unlocked Guardian, or Admin, then summarizes historical rows. Flutter converts the response into totals, a rating trend, and match details.

60-second technical explanation

The view watches a Player-keyed FutureProvider, which invokes the use case and authenticated repository GET. Django checks verified role, Club/self/link, and PIN unlock before querying. It validates optional date/limit filters, loads at most 100 newest rows, calculates pass rate, totals, clean sheets, and one-decimal average rating, then serializes summary/history. Dart constructs typed nested objects; Riverpod resolves; the UI renders tiles, chronological rating chart, and expandable cards. Unauthorized IDs never enter aggregation.

Card 13: Flutter-to-Django API Communication

What I click

Any live-data action, such as opening schedule or saving attendance.

First function called

The screen callback or Riverpod provider/controller.

Next function

Domain use case -> repository interface -> concrete `Api*Repository`.

Service used

`AuthenticatedApiClient` with Dart `http`, plus Firebase Auth for the ID token.

Database/API operation

HTTPS REST request with Bearer token and JSON/multipart -> DRF APIView -> serializer/authorization -> Django ORM.

Data returned

HTTP status plus JSON converted by a Dart `fromJson()` factory.

State change

Riverpod changes loading -> data/error and invalidates/refetches affected providers after mutations.

Screen result

Widget rebuilds from authorized server data or shows a retryable error.

20-second defense explanation

Flutter never queries the database directly. It sends a Firebase-authenticated REST request to Django; Django verifies identity, enforces role/Club/privacy, validates, persists, and returns JSON for Riverpod to display.

60-second technical explanation

The UI calls a controller/provider, then a domain use case and repository interface. A concrete API repository serializes the request and delegates to the shared authenticated client. That client gets the current Firebase ID token, reserves the Bearer header, builds the configured HTTPS URI, applies timeout/error/cache rules, and sends JSON or multipart data. DRF verifies the token with Firebase Admin, loads the local user, applies permissions and scoped queries, validates through serializer/model/database rules, and uses the ORM. The response becomes typed Dart data and Riverpod rebuilds only its consumers.

Card 14: Publish Tournament Schedule

What I click

Coordinator portal -> Tournaments -> upload official schedule -> add fixture.

First function called

`tournament_schedules()` or `tournament_schedule_detail()`.

Next function

Django form validation -> storage helper -> ORM/audit.

Service used

Supabase Storage REST through server-side `httpx`; DRF read API for mobile.

Database/API operation

Insert Club schedule/fixtures and opaque document path; `GET /api/tournament-schedules/` returns published nested data.

Data returned

Portal redirect/messages and mobile typed schedule/fixture list with short-lived signed URL.

State change

Portal reloads DB state; mobile FutureProvider resolves/refetches.

Screen result

Official programme and fixtures appear for authenticated Club roles; eligible fixture can start result entry.

20-second defense explanation

Only the Club Coordinator manages official schedules. Django validates and privately stores the file, stores structured fixtures for real workflows, and gives mobile users only authorized Club data and a short-lived link.

60-second technical explanation

Session-authenticated portal views require Coordinator role and filter every schedule/fixture by `request.user.club_id`. Upload validation checks size, MIME type, and signature. Django uploads with the server-only Supabase key and stores an opaque path. Structured fixtures are relational rows; completed ones link one-to-one to match results and cannot be casually edited/deleted. The mobile DRF endpoint returns only published same-Club schedules with nested fixtures and a signed URL, which Flutter converts through `TournamentSchedule.fromJson()`.

Card 15: Coach Match Rating Separation

What I click

Coach match roster -> Player with statistics -> enter rating/notes -> Save.

First function called

Rating editor save callback.

Next function

`MatchManagementController.saveRating()` -> `SaveMatchRating`.

Service used

`ApiMatchRepository.saveRating()` through `AuthenticatedApiClient`.

Database/API operation

`PUT /api/matches/{matchId}/performances/{playerId}/rating/`; update only subjective fields on an existing row.

Data returned

Typed `MatchPerformance` with rating state.

State change

Roster, match performances, and Player statistics providers invalidate/refetch.

Screen result

Coach sees rated state; authorized trend/history views show refreshed rating.

20-second defense explanation

The Coordinator owns objective statistics; the Coach owns rating and notes. Separate endpoints and serializers enforce that responsibility split instead of relying on hidden buttons.

60-second technical explanation

The rating call resolves a same-Club match through `_role_match(..., COACH)` and requires an existing objective performance. A rating-only serializer accepts 0.0-10.0 plus notes, locks the row, stamps the verified Coach and time, and audits the change. It cannot change goals, minutes, Player, match, or recorder. Clearing the rating nulls only subjective fields, preserving the historical evidence.

Card 16: Mobile Tournament and Age-Bracket Management

What I click

Coordinator Schedule tab -> **Create tournament** or edit icon -> save details/add bracket/publish.

First function called

`EditTournamentScreen._saveDetails()`, `_editBracket()`, or `_publish()`.

Next function

`TournamentManagementController` matching mutation method.

Service used

`ApiTournamentScheduleRepository` through `AuthenticatedApiClient`.

Database/API operation

Schedule/bracket POST/PATCH/DELETE/publish endpoints; write `TournamentSchedule`, `TournamentAgeBracket`, and audit rows under the verified Club.

Data returned

Complete nested `TournamentSchedule` JSON with age brackets, squads, and fixtures.

State change

Controller loading/error resolves and `tournamentSchedulesProvider` invalidates.

Screen result

Draft/Published tournament card, bracket chips/times, and management actions reflect server state.

20-second defense explanation

The Coordinator can now create and publish tournament brackets from Flutter. Every request is Firebase-authenticated, Club-scoped in Django, validated against dependencies/current rosters, audited, and returned as typed nested schedule data.

60-second technical explanation

The screen calls an AsyncNotifier controller, which delegates to the API repository and shared authenticated client. DRF maps the route to a Coordinator-only schedule/bracket APIView and resolves the object through `request.user.club_id`. Serializers validate title, date, U3-U25 uniqueness, and optional bracket time. Transactions reject edits that would invalidate stored roster members, while deletion guards protect Published brackets, linked fixtures, and entries. Django writes schedule/bracket/audit rows and returns the full nested object; Riverpod invalidates and rebuilds the cards.

Card 17: Coach Tournament Squad

What I click

Coach Schedule tab -> U-age division -> select Players/positions -> **Save draft** or **Publish roster**.

First function called

`TournamentSquadScreen._save()` after candidates load through `tournamentRosterCandidatesProvider`.

Next function

`TournamentRosterManagementController.save()` and optional `.publish()`.

Service used

`ApiTournamentRosterRepository` through `AuthenticatedApiClient`.

Database/API operation

`PUT /api/tournament-brackets/{id}/squad/` then optional publish POST; atomic reconcile of `TournamentSquadEntry` and status/audit update.

Data returned

Typed `TournamentSquad` with Draft/Published state and selected Player entries.

State change

Schedule and candidate providers invalidate; `_squad` takes the server response.

Screen result

Selected roster, event positions, membership count, and publication state refresh for permitted roles.

20-second defense explanation

Only the Coach selects and publishes each age-bracket squad. Django computes age/profile/injury eligibility, blocks invalid Players, stores one unique entry per Player, and exposes unpublished details only to manager roles.

60-second technical explanation

The candidates provider calls a Coach-only endpoint that queries active same-Club Players and returns privacy-safe eligibility. The save request contains only Player IDs and optional event positions. Django rejects duplicates, cross-Club/inactive accounts, invalid positions, and hard-blocked candidates, then uses a transaction and row lock to add/remove/update entries and audit the differences. Publication requires a Published tournament, a non-empty squad, and a fresh eligibility pass. Typed JSON flows back through Riverpod invalidation to the Schedule and roster screens.

Card 18: Tournament Match Squad Enforcement

What I click

Coordinator age-bracket match -> roster; either select a published-squad Player or tap **Add eligible squad exception**, choose a Player, enter a reason, and save statistics.

First function called

`MatchRosterScreen._addOutOfSquadPlayer()` for an exception, otherwise the Player tile opens the statistics editor.

Next function

Expanded roster provider -> reason modal -> `MatchManagementController.savePerformance()`.

Service used

`ApiMatchRepository.fetchMatchRoster()/savePerformance()` through the shared client.

Database/API operation

Roster GET joins bracket/published squad/eligibility; performance PUT locks and saves objective statistics plus optional reason/verified actor/time and audit.

Data returned

Membership-aware `MatchRosterPlayer` rows and saved `MatchPerformance` with squad-exception metadata.

State change

Roster, expanded candidates, match performances, and Player trend providers invalidate.

Screen result

Normal squad members remain the default roster; an approved eligible exception appears with a clear status chip.

20-second defense explanation

Tournament matches default to the Coach's Published squad. An eligible late exception is possible only for the Coordinator with a written reason; age/profile/confirmed-injury blocks cannot be overridden, and the server records the actor, time, and audit event.

60-second technical explanation

`MatchRosterView` derives the bracket from the source fixture and normally returns Published squad members plus existing historical rows. Only a Coordinator can request expanded candidates. On save, `MatchPerformanceDetailView` locks the match and row, recomputes `roster_eligibility`, checks current Published membership, and requires a trimmed reason for a non-member. It stores override reason/by/at under a database consistency constraint, audits the exception, then validates all objective statistics. A failed rule rolls back; success returns typed data and Riverpod refetches every dependent screen.

Final Trace Verification Notes

- Every connected chain above was checked against the referenced screen/provider/use-case/repository/view/model files.
- Table names follow Django defaults verified from model/migration naming; `PlayerEligibility` is a proxy and has no table.
- No secret values, tokens, database passwords, or service credentials are included.
- Scout/scouting and AI paths are marked `NOT IMPLEMENTED IN CURRENT REPOSITORY` because no executable connection exists.
- Notification persistence, current-user inbox/read APIs, device registration/unregister, foreground SnackBar handling, bell navigation, duplicate suppression, best-effort read marking, and role-aware session/profile/eligibility destinations are traceable and locally tested. Real Firebase/APNs delivery and presentation on a signed physical device are not claimed as verified.
- Container/compose, health/readiness, optional Sentry, backup/restore scripts, runbook, and CI image-build paths are traceable repository artifacts; no live deployment, executed alert, scheduled backup, or successful restore drill is claimed.
- The code comment in `PlayerAssessmentView` still says "six ratings," while the executable entity/profile supports twelve (six outfield plus six goalkeeper); this trace follows the actual fields.
- Current tracing reflects commit `6586553`, including mobile Coordinator tournament/bracket management, Coach age-bracket squad selection/publication, server-enforced match-squad eligibility and auditable exceptions, responsive Coordinator UI consistency, role-separated match evidence/ratings, injury workflows, independent-Club eligibility suppression, and enforced live persistence.
- Verification on 2026-08-29: GitHub CI passed all three jobs. The Django job passed 352 tests, deployment checks, and migration-drift checks; the Flutter job passed 295 tests and reported no analysis issues; the production-container job passed. Production environment/TLS configuration and real-device/deployed-service evidence remain separate responsibilities rather than claimed live evidence.